ATLAS VISUEL OBSIDIA / X108 + SRL

Gouvernance décisionnelle, preuve, publication maîtrisée et mémoire sessionnelle

Sources fusionnées	audit memoire lot(2).docx audit memoire ect(2).docx
Règle de fabrication	Reprise source-only : mots, formulations, variantes, boundaries et listes reprises depuis les deux documents. Aucun “etc.” ajouté, aucun arbitrage, aucune invention.
Statut	Document PDF de compréhension visuelle. Non-runtime. Non-code. Non-publication brute.

Phrase verrou : X108 autorise. SRL rappelle. X108 gouverne l’action. SRL gouverne la continuité contextuelle.

Sommaire de fabrication

Bloc	Contenu	Source
P78	Classification documentaire public / privé / preuve / investisseur / démo / archive	audit memoire lot(2)
Schémas 05 à 27	Gouvernance X108, domaines Banque / Trading / GPS / E-commerce, AgentVote, Aggregate, Sigma, Envelope, Audit, OS/runtime, Lean, synthèse kernel	audit memoire lot(2)
SRL	Mémoire sessionnelle, Active / Semi-active / Cold / Ghost, session card, symbolic hash, recall, audit SRL, métriques et boundaries readonly	audit memoire ect(2)
Synthèse	Séparation finale : X108 = autorité ; SRL = continuité mémoire	fusion 2 docs

Schémas repris du lot : 05 — PRIORITÉ BLOCK > HOLD > ALLOW, 06 — HOLD / TEMPORAL LOCK, 07 — NON-CONTOURNEMENT, 08 — ENTRÉE VARIABLE → IR / ALPHABET INTERNE, Schéma 05 validé si :, Schéma 06 validé si :, Schéma 07 validé si :, Schéma 08 validé si :, 09 — BANQUE : FLUX COMPLET, 10 — BANQUE : MÉTRIQUES ET CAS X108, 11 — TRADING / BOURSE : FLUX COMPLET, 12 — TRADING : MÉTRIQUES ET CAS X108, Schéma 09 validé si :, Schéma 10 validé si :, Schéma 11 validé si :, Schéma 12 validé si :, 13 — GPS / DÉFENSE / AVIATION : FLUX COMPLET, 14 — GPS : MÉTRIQUES ET CAS X108, 15 — E-COMMERCE : ANNEXE DOMAINE, 16 — AGENTVOTE : UNITÉ ATOMIQUE COMMUNE, Schéma 13 validé si :, Schéma 14 validé si :, Schéma 15 validé si :, Schéma 16 validé si :, 17 — DOMAINAGGREGATE, 18 — META-AGENTS, 19 — SIGMA / PROOF SIGNALS, 20 — TRIPLE CONFIDENCE, Schéma 17 validé si :, Schéma 18 validé si :, Schéma 19 validé si :, Schéma 20 validé si :, 21 — CANONICAL DECISION ENVELOPE, Schéma complet, 22 — AUDIT / HASH / REPLAY, Schéma complet, 23 — EXPOSITION HUMAINE READONLY, Schéma complet, 24 — OS / RUNTIME / BUS, Schéma complet, Schéma 21 validé si, Schéma 22 validé si, Schéma 23 validé si, Schéma 24 validé si, 25 — LEAN 57 THÉORÈMES, Schéma complet, 26 — LEAN → RUNTIME KERNEL, Schéma complet, 27 — SYNTHÈSE FINALE : PREUVE VISUELLE DU KERNEL, Schéma complet, Schéma 25 validé si, Schéma 26 validé si, Schéma 27 validé si

PARTIE I - P78 + Gouvernance X108

Reprise complète et mise en forme visuelle de audit memoire lot(2).docx

SOURCE 1 - audit memoire lot(2).docx

MODE AUDIT + DOCS CONTROLLED — P78 PRESENTATION PROOF PUBLIC PRIVATE SPLIT

RÔLE

Tu es Claude Code en mode classification documentaire.

Objectif : séparer proprement les documents de preuve, présentation, public, investisseur, démo, privé/propriétaire, archives et source packs locaux.

REPO

C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs_REMOTE_A5F21C6B

BRANCHE DE DÉPART

p77-canon-wording-targeted-cleanup

Créer une nouvelle branche :

p78-presentation-proof-public-private-split

CONTEXTE VALIDÉ

```
P56 terminé : Sigma post-Guard veto-only, gamma proof = 1.0.
```

P57 terminé : core machinery inventory.

P58 terminé : path-aware triage.

P59 terminé : audit tooling importé.

P60 terminé : tests proof importés.

P61 terminé : bus dry-run adapté.

P62 terminé : 77 fichiers lourds classifiés.

P63 terminé : global fusion reality audit.

P64 terminé : fusion continuity ledger.

P65 terminé : OS adapters readonly/dry-run + bus registry.

P66 terminé : SRL readonly memory layer.

P67 terminé : boundary semantic split.

P68 terminé : API auth/route exposure.

P69 terminé : filesystem/path exposure.

P70 terminé : network egress/connectors.

P71 terminé : source runtime/source packs.

P72 terminé : invariant graph/formal proof alignment.

P73 terminé : agents complémentaires readonly.

P74 terminé : Sigma safe evolution, NO_SIGMA_CHANGE_REQUIRED.

P75 terminé : runtime core risk review, NO_RUNTIME_IMPORT.

P76 terminé : GPS terrain portable reconciliation.

P77 terminé : canon wording targeted cleanup.

Commits connus :

P65 : 759b6d9

P66 : da51e45

P67 : 4a54438

P68 : 754fc74

P69 : c72dea7

P70 : 8d1bd3d

P71 : b5ab1da

P72 : 1d61ebb

P73 : d7b22d8

P74 : c526fa5

P75 : ba2a154

P76 : 7bd1019

P77 : 3f80037

OBJECTIF P78

Auditer et classifier les documents selon leur rôle réel.

P78 doit répondre :

1. Quels fichiers sont des preuves techniques ?

2. Quels fichiers sont des docs d’audit ?

3. Quels fichiers sont des supports publics ?

4. Quels fichiers sont des supports investisseurs ?

5. Quels fichiers sont des narratifs / doctrine / pitch ?

6. Quels fichiers sont des démos ?

7. Quels fichiers sont des archives ?

8. Quels fichiers sont source packs locaux ?

9. Quels fichiers doivent rester privés/propriétaires ?

10. Quels fichiers peuvent être publiés ?

11. Quels fichiers ne doivent pas être inclus dans un pack public ?

12. Quels fichiers peuvent nourrir un RSSI evidence pack ?

13. Quels fichiers doivent être séparés avant freeze P80 ?

P78 ne doit pas supprimer.

P78 ne doit pas déplacer massivement.

P78 ne doit pas modifier runtime/Sigma/routes/connectors.

P78 doit produire une matrice de classification et, si safe, créer uniquement des index docs.

PRINCIPE

```
Support présentation ≠ preuve technique.
Public proof ≠ moteur propriétaire complet.
Source pack local ≠ canon public.
Narratif investisseur ≠ audit RSSI.
Démo ≠ runtime souverain.
```

> Archive ≠ runtime load.

SURFACES À AUDITER

docs/

docs/core_import/

docs/freeze/

docs/source_packs/

docs/public/

docs/investor/

docs/demo/

docs/proof/

specs/

specs/_invariant_graph/

README.md

LIMITS.md

KERNEL_OVERVIEW.md

GLOSSAIRE.md

_source_packs/

_tmp_core_import/

_freezes/

.local_audits/

audit/

proofs/

apps/obsidia_api/

runtime_wiring/

sigma/

periphery/

connectors/

NE PAS TOUCHER

- sigma/
- runtime_wiring/
- apps/obsidia_api/routes/
- apps/obsidia_api/*.py
- connectors/
- periphery/
- proofs/V18_3_1/
- proofs/lean/
- _tmp_core_import/
- _source_packs/
- _freezes/
- .local_audits/
- audit/world_action_bus.jsonl
- proofs/PROOFKIT_REPORT.json

CLASSIFICATIONS P78

DOC_PROOF_TECHNICAL

DOC_AUDIT_TRAIL

DOC_CORE_IMPORT_LEDGER

DOC_FORMAL_PROOF_MAP

DOC_RSSI_EVIDENCE_CANDIDATE

DOC_PUBLIC_SAFE

DOC_PUBLIC_NEEDS_SOFTENING

DOC_INVESTOR_NARRATIVE

DOC_DEMO_ONLY

DOC_PRIVATE_PROPRIETARY

DOC_SOURCE_PACK_LOCAL_ONLY

DOC_ARCHIVE_ONLY

DOC_RUNTIME_INTERNAL

DOC_API_INTERNAL

DOC_MEMORY_INTERNAL

DOC_CONNECTOR_INTERNAL

DOC_DO_NOT_PUBLISH

DOC_UNKNOWN_REVIEW

DÉCISIONS POSSIBLES

KEEP_AS_TECHNICAL_PROOF

KEEP_AS_AUDIT_LEDGER

KEEP_AS_PUBLIC_SAFE

MOVE_LATER_PUBLIC

MOVE_LATER_INVESTOR

MOVE_LATER_DEMO

KEEP_PRIVATE_PROPRIETARY

KEEP_ARCHIVE_ONLY

KEEP_SOURCE_PACK_LOCAL_ONLY

DO_NOT_PUBLISH

REQUIRES_REVIEW

RÈGLES

1. Les docs P56→P77 sont audit ledger.

2. Les specs/_invariant_graph sont preuve/architecture, pas pitch.

3. Les source packs locaux restent local-only tant que non canonisés.

4. Les fichiers contenant chemins locaux, extracts source, connecteurs, routes internes ne sont pas public safe.

5. Les docs narratifs peuvent être classés investor/public, mais doivent être séparés du proof.

6. Les preuves techniques doivent rester liées à tests, commits, manifests, hashes.

7. Les docs investisseurs ne doivent pas prétendre plus que P72/P77 autorise.

8. Les archives restent archives, pas runtime.

9. Les fichiers privés/propriétaires doivent être marqués DO_NOT_PUBLISH.

10. Aucun déplacement massif dans P78 sauf création d'index de classification.

SORTIES À CRÉER

docs/core_import/P78_PRESENTATION_PROOF_PUBLIC_PRIVATE_SPLIT.md

docs/core_import/P78_PRESENTATION_PROOF_PUBLIC_PRIVATE_SPLIT.json

tests/test_p78_presentation_proof_public_private_split.py

Optionnel :

scripts/audit_presentation_proof_public_private_split_p78.py

Optionnel si safe docs-only :

docs/public/README_PUBLIC_BOUNDARY.md

docs/investor/README_INVESTOR_BOUNDARY.md

docs/proof/README_PROOF_BOUNDARY.md

docs/demo/README_DEMO_BOUNDARY.md

JSON attendu :

{
"audit_id": "P78",
"status": "P78_PRESENTATION_PROOF_PUBLIC_PRIVATE_SPLIT_READY",
"mode": "AUDIT_AND_DOC_INDEX_ONLY",
"source_patch_applied": false,
"files_imported_count": 0,
"split_decision": "CLASSIFY_AND_INDEX_ONLY",
"split_model": {
"technical_proof": "",
"audit_trail": "",
"formal_proof_map": "",
"rssi_evidence_candidate": "",
"public_safe": "",
"public_needs_softening": "",
"investor_narrative": "",
"demo_only": "",
"private_proprietary": "",
"source_pack_local_only": "",
"archive_only": "",
"do_not_publish": ""
},
"files_scanned_count": 0,
"doc_matrix": [],
"category_counts": {},
"decision_counts": {},
"technical_proof": [],
"audit_trail": [],

"formal_proof_map": [],
"rssi_evidence_candidates": [],
"public_safe": [],
"public_needs_softening": [],
"investor_narrative": [],
"demo_only": [],
"private_proprietary": [],
"source_pack_local_only": [],
"archive_only": [],
"do_not_publish": [],
"unknown_review": [],
"created_indexes": [],
"focus_findings": [],
"p72_claim_constraints_applied": [],
"p77_wording_constraints_applied": [],
"runtime_modified": false,
"sigma_modified": false,
"routes_modified": false,
"srl_modified": false,
"connectors_modified": false,
"source_packs_modified": false,
"proofs_modified": false,
"lean_proofs_modified": false,
"act_enabled": false,
"memory_write_enabled": false,
"graphiti_write_enabled": false,
"neo4j_write_enabled": false,
"kernel_mutation_enabled": false,

"x108_merge_enabled": false,

"next_step": "P79_RSSI_EVIDENCE_PACK_AND_GITHUB_SECURITY_AUDIT"

}

LOT 2 — SÉCURITÉ / GOUVERNANCE DU KERNEL

LOT 2 =

05. Priorité BLOCK > HOLD > ALLOW

06. HOLD / temporal lock

07. Non-contournement

08. Entrée variable → IR / alphabet interne

Objectif du lot :

Montrer que X108 n’est pas un simple score.

C’est un verrou de gouvernance :

il priorise le refus,

il suspend l’action incertaine,

il empêche le contournement,

il ne traite que de la structure gouvernable.

> `Le doc fourni insiste sur cette séparation : agents = signalent, agrégation = synthétise, Sigma = surveille, X108 = décide, OS3 = prouve.`

SCHÉMA 05 — PRIORITÉ BLOCK > HOLD > ALLOW

Rôle du schéma

Montrer que la décision X108 suit une hiérarchie souveraine, pas une moyenne.

Question à laquelle il répond

Pourquoi ALLOW n'est jamais le défaut ?

Contenu obligatoire

Les trois états doivent être affichés dans cet ordre :

1. BLOCK

2. HOLD

3. ALLOW

Structure visuelle recommandée

```
┌──────────────────────────────────────────┐
│ PRIORITÉ 1 — BLOCK                       │
│ Risque critique                          │
│ Fraude                                   │
│ Contradictions fortes                    │
│ Bypass / frontière cassée                │
│ → Action interdite                       │
└──────────────────┬───────────────────────┘
                   │
                   │ si rien ne bloque
                   ▼
┌──────────────────────────────────────────┐
│ PRIORITÉ 2 — HOLD                        │
│ Incertitude                              │
│ Contexte manquant                        │
│ Confiance trop basse                     │
│ Risque instable                          │
```

```
| Délai X108 non satisfait                      |
| → Action suspendue                            |
└──────────────────────┬────────────────────────┘
| si rien ne suspend
```

▼

```
┌──────────────────────────────────────────────┐
| PRIORITÉ 3 — ALLOW                           |
| Structure cohérente                          |
| Risque acceptable                            |
| Preuve suffisante                            |
| Confiance suffisante                         |
| → Action autorisée                           |
└──────────────────────────────────────────────┘
```

Règles à afficher

```
contradictions >= 2 → BLOCK
FRAUD_PATTERN → BLOCK
```

```
unknowns > 1 → HOLD
confidence < 0.45 → HOLD
risk_flags >= 2 + confidence < 0.72 → HOLD
```

```
sinon → ALLOW
```

Message visuel

ALLOW arrive seulement quand BLOCK et HOLD n’ont rien trouvé.

Ce qu’il faut faire ressortir

Un bon score métier ne force jamais ALLOW.

Une seule condition critique peut bloquer.

Une zone grise ne passe pas : elle HOLD.

À ne pas faire

Ne pas représenter les trois états comme équivalents.

Mauvais :

ALLOW / HOLD / BLOCK

Bon :

BLOCK > HOLD > ALLOW

SCHÉMA 06 — HOLD / TEMPORAL LOCK

Rôle du schéma

Faire comprendre que HOLD est une décision active, pas une absence de décision.

Question à laquelle il répond

Pourquoi le kernel suspend au lieu d’autoriser ?

Contenu obligatoire

Afficher les déclencheurs de HOLD :

unknowns > 1

confidence < 0.45

```
risk_flags >= 2 + confidence < 0.72
```

elapsed_s < min_required_elapsed_s

stabilité insuffisante

preuve / attestation / trace insuffisante

Structure visuelle recommandée

```
┌──────────────────────────────────────────┐
│ ACTION CANDIDATE                         │
│ demande d’action possible                │
└──────────────────┬───────────────────────┘
                   ▼
┌──────────────────────────────────────────┐
│ X108 TEMPORAL / COHERENCE CHECK          │
│ elapsed_s                                │
│ min_required_elapsed_s                   │
│ confidence                               │
│ unknowns                                 │
│ risk_flags                               │
│ sigma_report                             │
└──────────────────┬───────────────────────┘
```

▼

```
┌──────────────────────────────────────────────┐
│ CONDITION NON SATISFAITE                     │
│ temps insuffisant                            │
│ contexte incomplet                           │
│ trajectoire instable                         │
│ confiance basse                              │
│ preuve incomplète                            │
└──────────────────────┬───────────────────────┘
```

▼

```
┌──────────────────────────────────────────────┐
│ HOLD                                         │
│ action non autorisée maintenant              │
│ aucune exécution réelle                      │
│ attente / contexte / réévaluation            │
└──────────────────────────────────────────────┘
```

Version arbre

Action candidate

```
↓
```

Action critique / irréversible ?

```
↓
```

Oui

```
↓
elapsed_s >= min_required_elapsed_s ?
├─ non → HOLD
└─ oui
↓
```

confidence suffisante ?

```
├─ non → HOLD
└─ oui
↓
```

unknowns / risk_flags acceptables ?

```
├─ non → HOLD
└─ oui → peut continuer vers ALLOW
```

Message visuel

HOLD est le frein temporel et structurel.

Formulation à mettre dans le support

Une action possible n’est pas une action autorisée.

X108 peut suspendre pour attendre, vérifier, compléter ou rejouer.

À ne pas faire

Ne pas présenter HOLD comme :

erreur

échec

indécision

bug

Présenter HOLD comme :

verrou de sécurité

délai actif

refus temporaire d’action réelle

SCHÉMA 07 — NON-CONTOURNEMENT

Rôle du schéma

Montrer que toutes les couches peuvent produire des signaux, mais qu’aucune ne peut court-circuiter X108.

Question à laquelle il répond

Qui a le droit d’autoriser une action ?

Contenu obligatoire

Afficher les couches sans autorité d’action :

Agents domaine

DomainAggregate

Meta-agents

Sigma

Projection française

Terminal / API

Mémoire / contexte

Interface humaine

Afficher la seule couche souveraine :

Kernel X108

Structure visuelle recommandée

```
┌──────────────────────────────┬──────────────────────────────┐
│ COUCHE                       │ DROIT D’AUTORISER ACT ?      │
├──────────────────────────────┼──────────────────────────────┤
│ Agents domaine               │ NON                          │
│ AgentVote                    │ NON                          │
│ DomainAggregate              │ NON                          │
│ Meta-agents                  │ NON                          │
│ Sigma                        │ NON                          │
│ Projection française         │ NON                          │
```

```
| Terminal / API                | NON                          |
| Audit / replay                | NON                          |
| Kernel X108                   | OUI                          |
└──────────────────────────────┴──────────────────────────────┘
```

Version flux

Agents

```
→ peuvent signaler
→ cannot ACT
```

Agrégation

```
→ peut synthétiser
→ cannot ACT
```

Sigma

```
→ peut alerter
→ cannot ACT
```

Projection française

```
→ peut exposer
→ cannot ACT
```

KERNEL X108

```
→ ALLOW / HOLD / BLOCK
```

Champs / boundaries à afficher

```
decision_authority = KX108_ONLY
emits_act = false
memory_write = false
readonly = true
```

no-bypass

no-write

Message visuel

Tout peut signaler.

Seul X108 autorise.

À ne pas faire

Ne pas mettre Sigma après X108 comme s’il pouvait revalider.

Ne pas mettre la projection française comme “réponse finale décisionnelle”.

Ne pas montrer l’audit comme une autorité : l’audit prouve après coup, il ne décide pas.

SCHÉMA 08 — ENTRÉE VARIABLE → IR / ALPHABET INTERNE

Rôle du schéma

Expliquer la “compréhension” structurelle sans parler de LLM.

Question à laquelle il répond

Comment le système traite plusieurs langues, payloads et signaux terrain ?

Contenu obligatoire

Entrées variables :

français

anglais

JSON

transaction bancaire

signal marché

télémétrie GPS

capteur

payload API

Transformation :

normalisation structurelle

IR

alphabet interne

risk_flags

constraints

domain markers

boundary markers

Sorties vers domaine :

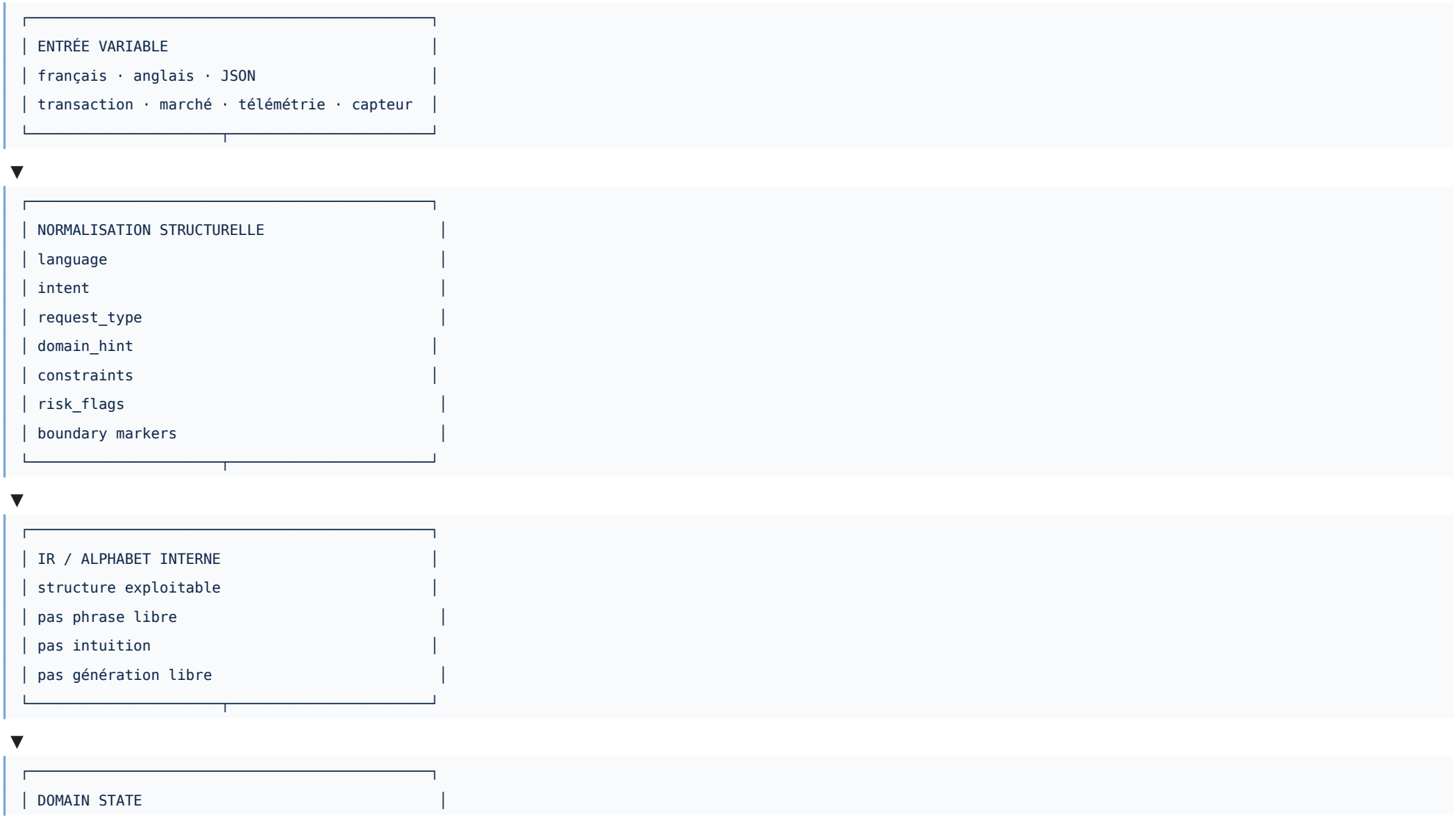
BankState

TradingState

GpsDefenseAviationState

EcomState

Structure visuelle recommandée



```
| BankState                                  |
| TradingState                                |
| GpsDefenseAviationState                    |
| EcomState                                  |
└──────────────────────┬─────────────────────┘
```

▼

```
┌────────────────────────────────────────────┐
| PIPELINE DOMAINE                            |
| agents → votes → aggregate → X108          |
└────────────────────────────────────────────┘
```

Message visuel

La langue est variable.

L’alphabet interne est stable.

Formulation support

Le kernel ne comprend pas le français comme un chatbot.

Il reçoit une structure gouvernable.

La langue, le JSON ou le signal terrain sont ramenés dans un alphabet interne.

À ne pas faire

Ne pas dire :

le kernel parle français

le kernel est un LLM

le kernel comprend n’importe quoi directement

Dire :

l’entrée est normalisée

l’entrée devient structure

la structure devient gouvernable

MINI-CHECKLIST DE VALIDATION DU LOT 2

Schéma 05 validé si :

BLOCK apparaît avant HOLD.

HOLD apparaît avant ALLOW.

ALLOW est visiblement le dernier cas.

Les seuils sont présents.

Schéma 06 validé si :

HOLD est présenté comme décision active.

elapsed_s / min_required_elapsed_s apparaissent.

Aucune action réelle ne sort pendant HOLD.

Schéma 07 validé si :

```
agents, aggregate, meta-agents, Sigma, projection = cannot ACT.
Kernel X108 = seule autorité.
decision_authority = KX108_ONLY visible.
```

Schéma 08 validé si :

français / anglais / JSON / capteur / transaction sont visibles.

IR / alphabet interne est central.

DomainState est la sortie.

Pas de LLM / pas de génération libre.

ORDRE DE FABRICATION RECOMMANDÉ DANS LE LOT 2

1. Schéma 07 — Non-contournement

2. Schéma 05 — Priorité BLOCK > HOLD > ALLOW

3. Schéma 06 — HOLD / temporal lock

4. Schéma 08 — Entrée variable → IR

Pourquoi :

on fixe d’abord l’autorité,

puis la priorité,

puis le frein temporel,

puis l’entrée structurelle.

RÉSULTAT ATTENDU APRÈS LE LOT 2

Après le lot 2, on doit pouvoir dire :

Le système accepte des entrées variables,

mais ne gouverne que des structures.

Aucune couche périphérique ne peut autoriser.

X108 applique une priorité stricte :

BLOCK avant HOLD, HOLD avant ALLOW.

HOLD est un verrou actif, pas une erreur.

Phrase verrou du lot :

La structure entre.

Les couches signalent.

X108 seul autorise.

LOT 3 — DOMAINES PRINCIPAUX : BANQUE + TRADING

LOT 3 =

09. Banque — flux complet

10. Banque — métriques et cas X108

11. Trading / Bourse — flux complet

12. Trading — métriques et cas X108

Objectif du lot :

Montrer que Banque et Trading ne sont pas des exemples décoratifs.

Ce sont deux chemins complets qui prouvent que le kernel reçoit des états métiers,

pas du texte libre,

pas une IA,

pas un simple signal.

Chaque domaine produit :

DomainState

```
→ agents spécialisés
→ AgentVote[]
→ agrégation métier
→ verdict candidat
→ meta-agents
→ Sigma / proof signals
→ X108
→ Envelope
```

> Le document d’audit confirme que Banque passe par BankState → build_bank_agents → aggregate_bank → meta-agents → GuardX108 → CanonicalDecisionEnvelope, et Trading par TradingState → build_trading_agents → aggregate_trading → meta-agents → GuardX108 → CanonicalDecisionEnvelope.

SCHÉMA 09 — BANQUE : FLUX COMPLET

Rôle du schéma

Montrer le chemin complet d’une transaction bancaire jusqu’au kernel.

Question à laquelle il répond

Que devient une transaction bancaire quand elle entre dans Obsidia/X108 ?

Contenu obligatoire

Le schéma doit montrer 8 niveaux :

1. Payload bancaire

2. BankState

3. 12 agents banque

4. AgentVote[]

5. aggregate_bank()

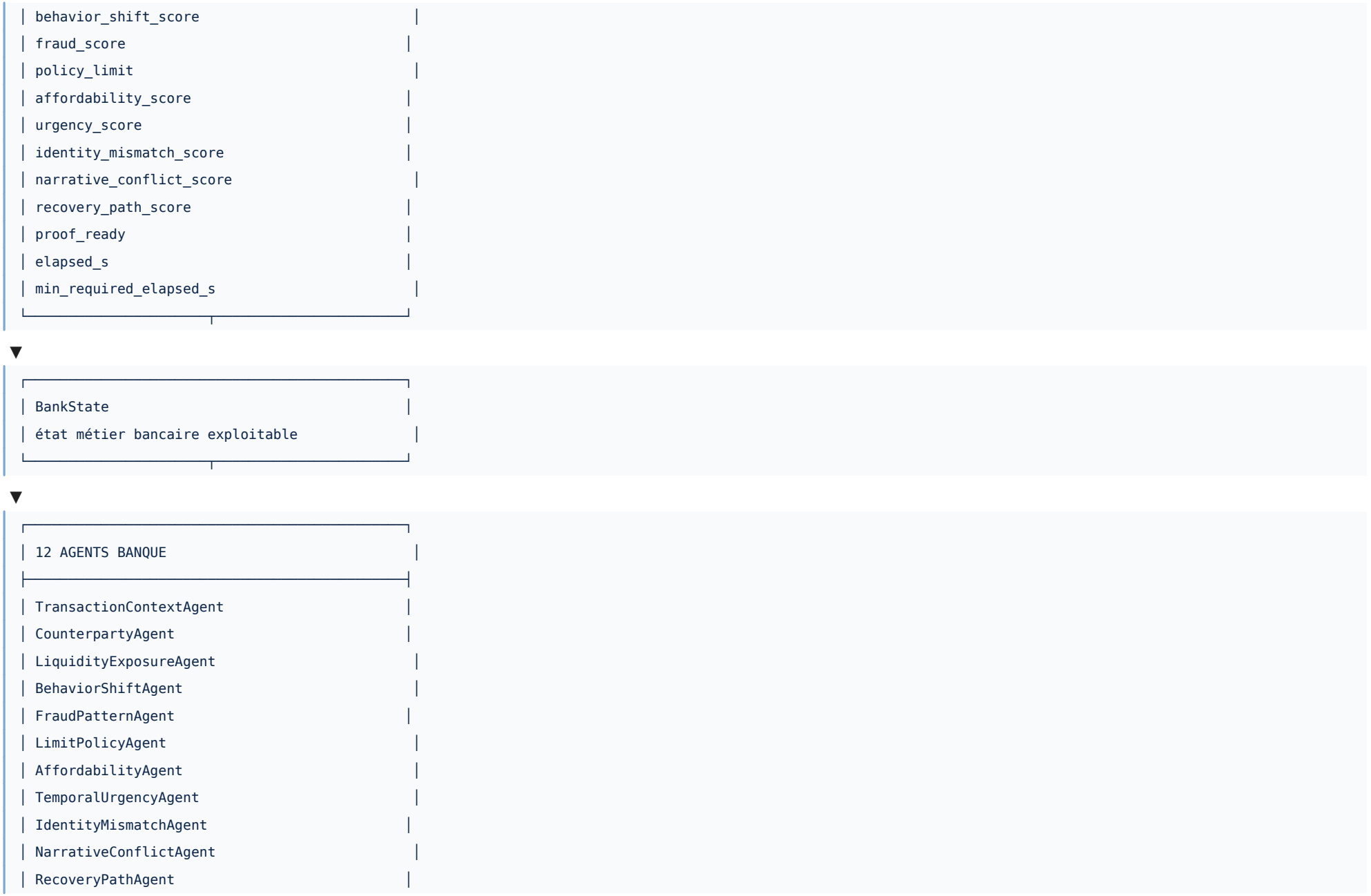
6. verdict métier candidat

7. Meta-agents + Sigma

8. Kernel X108 → Envelope → Audit

Structure visuelle complète

```
┌──────────────────────────────────────────┐
│ PAYLOAD BANCAIRE                         │
├──────────────────────────────────────────┤
│ transaction_type                         │
│ channel                                  │
│ amount                                   │
│ account_balance                          │
│ counterparty_known                       │
│ counterparty_age_days                    │
```


behavior_shift_score
fraud_score
policy_limit
affordability_score
urgency_score
identity_mismatch_score
narrative_conflict_score
recovery_path_score
proof_ready
elapsed_s
min_required_elapsed_s
BankState
état métier bancaire exploitable
12 AGENTS BANQUE
TransactionContextAgent
CounterpartyAgent
LiquidityExposureAgent
BehaviorShiftAgent
FraudPatternAgent
LimitPolicyAgent
AffordabilityAgent
TemporalUrgencyAgent
IdentityMismatchAgent
NarrativeConflictAgent
RecoveryPathAgent


BankProofAgent
AgentVote[]
claim · confidence · severity_hint
unknowns · contradictions · risk_flags
proposed_verdict · evidence_refs
aggregate_bank()
scores AUTHORIZE / ANALYZE / BLOCK
confidence
unknowns
contradictions
risk_flags
evidence_refs
proof_ready
deterministic
VERDICT MÉTIER CANDIDAT
AUTHORIZE / ANALYZE / BLOCK
≠ autorisation finale
META-AGENTS + SIGMA / PROOF SIGNALS

```
| trace · ticket · attestation · replay        |
| proof consistency · stability · readiness    |
└──────────────────────┬──────────────────────┘
                       ▼
┌──────────────────────┴──────────────────────┐
| KERNEL X108 — DERNIER MUR                    |
| BLOCK > HOLD > ALLOW                         |
└──────────────────────┬──────────────────────┘
                       ▼
┌──────────────────────┴──────────────────────┐
| CanonicalDecisionEnvelope                    |
| x108_gate · reason_code · severity           |
| decision_id · trace_id · attestation_ref     |
└──────────────────────┬──────────────────────┘
                       ▼
┌──────────────────────┴──────────────────────┐
| Audit / hash / replay                        |
└──────────────────────────────────────────────┘
```

Métriques Banque obligatoires

authorize_score

analyze_score

block_score

confidence

unknowns

contradictions

risk_flags

evidence_refs

proof_ready

deterministic

L’audit confirme que aggregate_bank() additionne les votes AUTHORIZE, ANALYZE, BLOCK, puis sort authorize_score, analyze_score, block_score, proof_ready et deterministic.

Message visuel

La banque ne demande pas un oui/non direct.

Elle produit un verdict métier candidat.

X108 décide ensuite si l’action est autorisée, suspendue ou bloquée.

Erreurs à éviter

Ne pas écrire :

```
BankState → ALLOW / BLOCK
```

Écrire :

BankState

```
→ agents banque
→ AgentVote[]
→ aggregate_bank
→ AUTHORIZE / ANALYZE / BLOCK candidat
→ X108
→ ALLOW / HOLD / BLOCK
```

Ne pas confondre :

BLOCK métier

avec :

BLOCK kernel

SCHÉMA 10 — BANQUE : MÉTRIQUES ET CAS X108

Rôle du schéma

Montrer comment les signaux bancaires déclenchent ALLOW, HOLD ou BLOCK.

Question à laquelle il répond

Quels cas bancaires activent le mur X108 ?

Structure visuelle recommandée

CAS BANQUE	SIGNALS / MÉTRIQUES	VERDICT MÉTIER CANDIDAT	SORTIE X108
Transaction cohérente	confidence haute	AUTHORIZE	ALLOW
Contrepartie inconnue	unknowns	ANALYZE	HOLD
Solde / liquidité fragile	affordability / liquidity	ANALYZE	HOLD
Fraude forte	FRAUD_PATTERN / fraud_score	BLOCK	BLOCK
Identité incohérente	identity_mismatch_score	ANALYZE / BLOCK	HOLD ou BLOCK selon signaux
Contradictions multiples	contradictions >= 2	BLOCK	BLOCK
Confiance trop basse	confidence < 0.45	ANALYZE	HOLD
Risques multiples	risk_flags >= 2 + conf < .72	ANALYZE	HOLD

Métriques à afficher autour du tableau

authorize_score

analyze_score

block_score

confidence

unknowns

contradictions

risk_flags

severity_hint

reason_code

x108_gate

Règles X108 à relier au tableau

```
contradictions >= 2
→ BLOCK
reason_code = CONTRADICTION_THRESHOLD_REACHED
```

FRAUD_PATTERN

```
→ BLOCK
reason_code = CONTRADICTION_THRESHOLD_REACHED
```

unknowns > 1

```
→ HOLD
reason_code = UNKNOWNS_OR_CONFIDENCE_LOW
```

confidence < 0.45

```
→ HOLD
reason_code = UNKNOWNS_OR_CONFIDENCE_LOW
```

```
risk_flags >= 2 et confidence < 0.72
→ HOLD
reason_code = RISK_FLAGS_REQUIRE_DELAY
```

sinon

```
→ ALLOW
reason_code = GUARD_ALLOW
```

Version “mur banque”

BankAggregate

```
↓
```

authorize_score / analyze_score / block_score

```
↓
```

unknowns / contradictions / risk_flags

```
↓
```

GuardX108

```
↓
```

ALLOW / HOLD / BLOCK

Message visuel

AUTHORIZE n’est pas ALLOW.

ANALYZE n’est pas HOLD.

BLOCK métier n’est pas forcément le même objet que BLOCK kernel.

Le domaine parle métier.

X108 parle autorité.

À ne pas faire

Ne pas faire un schéma bancaire qui finit directement par :

```
AUTHORIZE = action autorisée
```

Correction obligatoire :

```
AUTHORIZE = verdict métier candidat
ALLOW = autorisation kernel
```

SCHÉMA 11 — TRADING / BOURSE : FLUX COMPLET

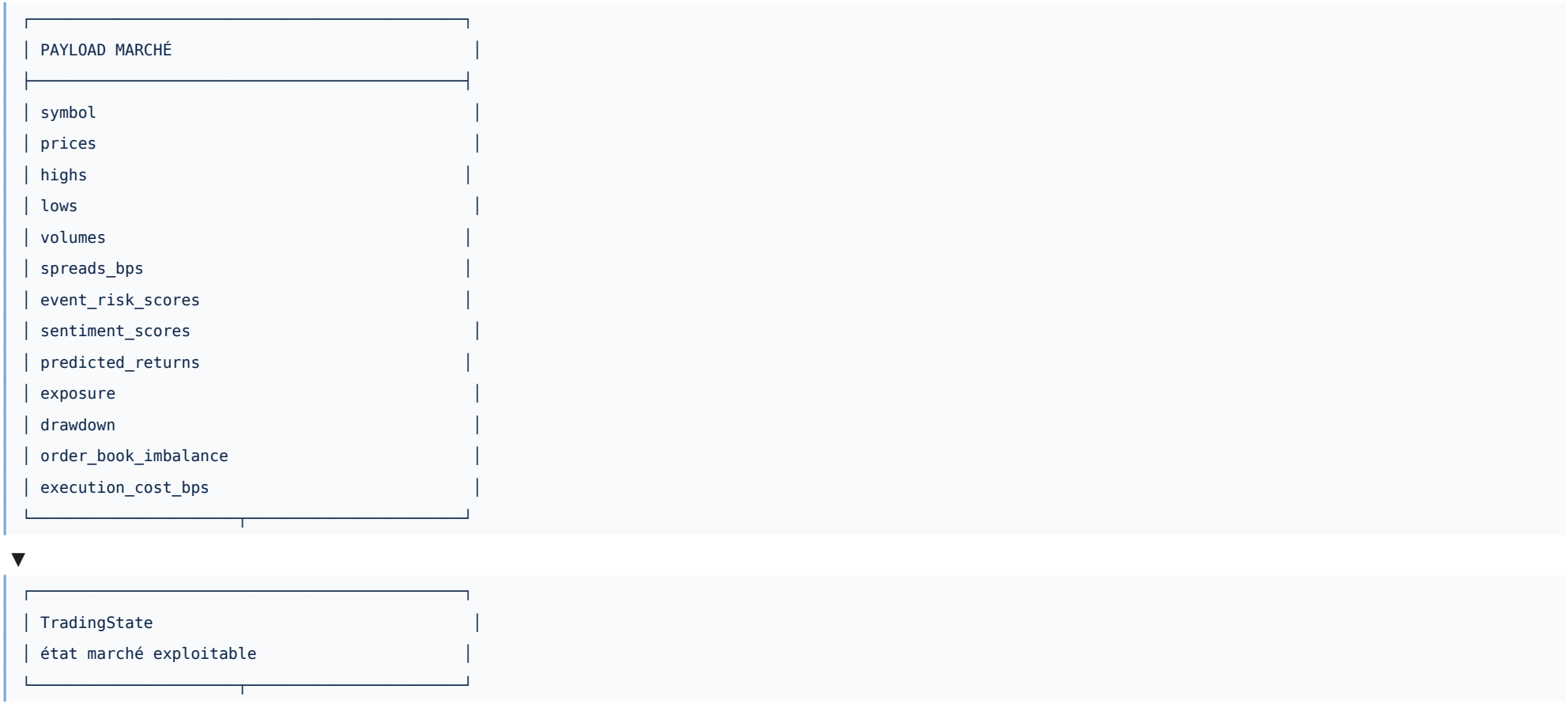
Rôle du schéma

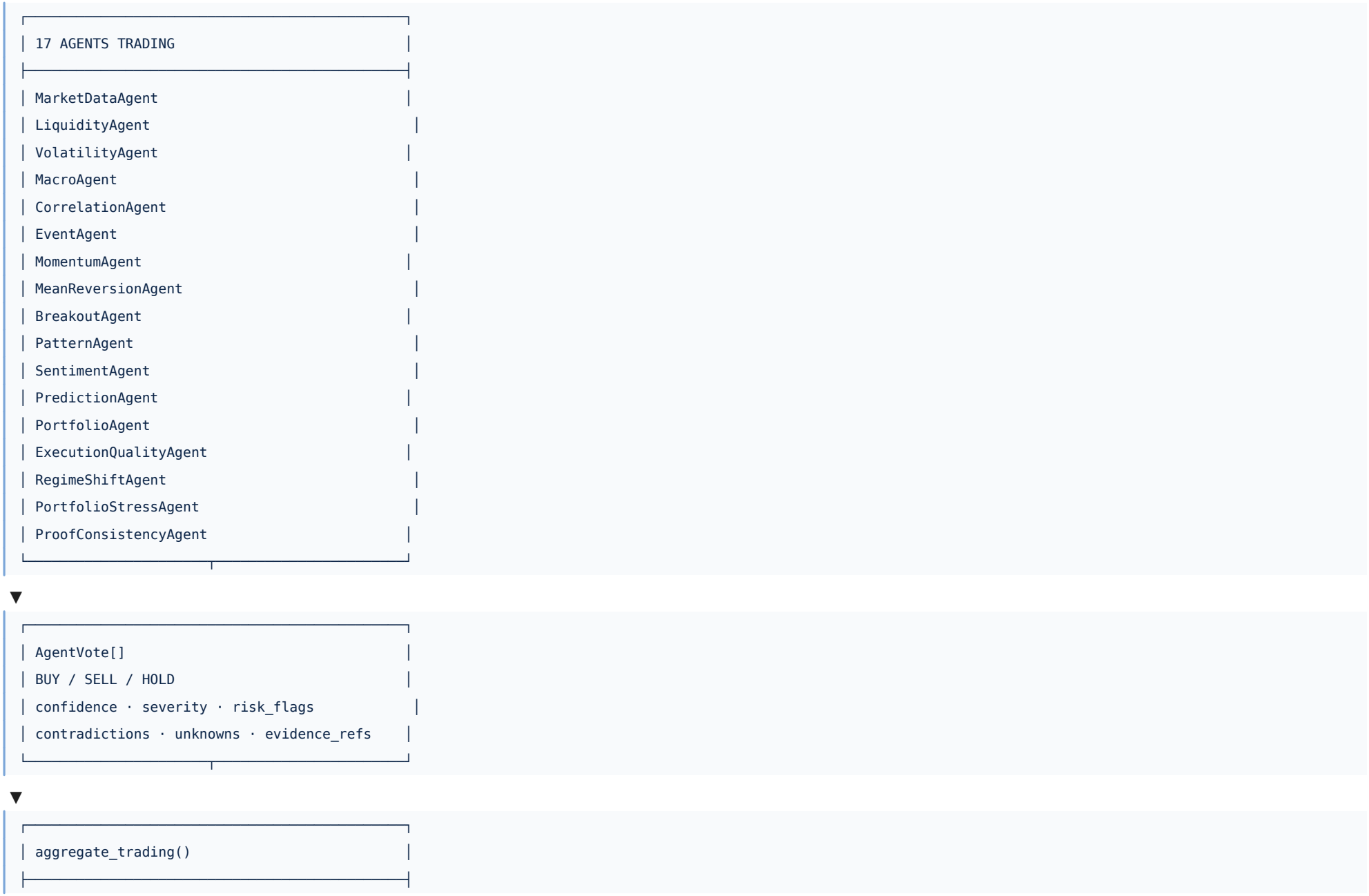
Montrer que le trading ne passe pas d’un signal marché à une action.

Question à laquelle il répond

Que devient un signal de marché dans Obsidia/X108 ?

Structure visuelle complète




17 AGENTS TRADING
MarketDataAgent
LiquidityAgent
VolatilityAgent
MacroAgent
CorrelationAgent
EventAgent
MomentumAgent
MeanReversionAgent
BreakoutAgent
PatternAgent
SentimentAgent
PredictionAgent
PortfolioAgent
ExecutionQualityAgent
RegimeShiftAgent
PortfolioStressAgent
ProofConsistencyAgent
AgentVote[]
BUY / SELL / HOLD
confidence · severity · risk_flags
contradictions · unknowns · evidence_refs
aggregate_trading()


buy_score
sell_score
hold_score
confidence
unknowns
contradictions
risk_flags
proof_ready
deterministic
SIGNAL MÉTIER
BUY / SELL / HOLD
VERDICT MÉTIER CANDIDAT
EXECUTE_LONG / EXECUTE_SHORT / REVIEW
≠ autorisation finale
META-AGENTS + SIGMA / PROOF SIGNALS
risk trajectory · proof consistency
stability · severity · readiness
KERNEL X108 — DERNIER MUR
BLOCK > HOLD > ALLOW

```
└──────────────────────┬──────────────────────┘
                       ▼
┌──────────────────────────────────────────────┐
│ CanonicalDecisionEnvelope                    │
└──────────────────────┬──────────────────────┘
                       ▼
┌──────────────────────────────────────────────┐
│ Audit / replay                               │
└──────────────────────────────────────────────┘
```

Métriques Trading obligatoires

buy_score

sell_score

hold_score

confidence

unknowns

contradictions

risk_flags

evidence_refs

proof_ready

deterministic

L’audit confirme que aggregate_trading() additionne BUY, SELL, HOLD, produit EXECUTE_LONG, EXECUTE_SHORT ou REVIEW, et sort buy_score, sell_score, hold_score, proof_ready, deterministic.

Message visuel

Un signal BUY n’est pas une autorisation d’achat.

EXECUTE_LONG doit encore survivre à X108.

Erreurs à éviter

Ne pas écrire :

```
BUY → EXECUTE_LONG → action
```

Écrire :

BUY

```
→ EXECUTE_LONG candidat
→ Sigma / meta-agents
→ X108
→ ALLOW / HOLD / BLOCK
```

SCHÉMA 12 — TRADING : MÉTRIQUES ET CAS X108

Rôle du schéma

Montrer comment le kernel traite un signal de marché selon stabilité, risque et preuve.

Question à laquelle il répond

Quand un trade est-il autorisé, suspendu ou bloqué ?

Structure visuelle recommandée

CAS TRADING	SIGNALS / MÉTRIQUES	VERDICT MÉTIER CANDIDAT	SORTIE X108
Signal clair + risque bas	buy/sell_score dominant	EXECUTE_LONG / SHORT	ALLOW
Signal faible	hold_score dominant	REVIEW	HOLD
Confiance basse	confidence < 0.45	REVIEW	HOLD
Risques multiples	risk_flags >= 2 + conf < .72	REVIEW	HOLD
Contradictions fortes	contradictions >= 2	REVIEW / blocage métier	BLOCK
Proof consistency fail	proof inconsistency	REVIEW	HOLD / BLOCK selon signaux
Régime instable	sigma_report / regime shift	REVIEW	HOLD

Métriques à afficher autour du tableau

buy_score

sell_score

hold_score

confidence

risk_flags

contradictions

unknowns

severity

sigma_report

sigma_override

reason_code

x108_gate

Mapping important à afficher

BUY / SELL / HOLD

```
= signaux métier
```

EXECUTE_LONG / EXECUTE_SHORT / REVIEW

```
= verdict métier candidat
```

ALLOW / HOLD / BLOCK

```
= décision kernel
```

Version “mur trading”

TradingAggregate

```
↓
```

buy_score / sell_score / hold_score

```
↓
```

EXECUTE_LONG / EXECUTE_SHORT / REVIEW

```
↓
```

risk_flags / contradictions / confidence / sigma_report

```
↓
```

GuardX108

```
↓
```

ALLOW / HOLD / BLOCK

Message visuel

Le marché peut proposer une direction.

Le kernel décide si l’action peut sortir.

À ne pas faire

Ne pas présenter le trading comme :

```
signal → trade
```

Présenter :

```
signal → verdict candidat → contrôle stabilité/gouvernance → X108 → décision
```

MINI-CHECKLIST DE VALIDATION DU LOT 3

Schéma 09 validé si :

- le payload bancaire est visible
- BankState est visible
- les 12 agents banque sont visibles
- aggregate_bank est visible
- authorize_score / analyze_score / block_score sont visibles
- AUTHORIZE / ANALYZE / BLOCK sont séparés de ALLOW / HOLD / BLOCK

Schéma 10 validé si :

- les cas fraude / inconnues / contradictions / confiance basse sont visibles
- les reason_code X108 sont présents ou réservés dans la légende
- on comprend que X108 tranche après le verdict métier

Schéma 11 validé si :

- le payload marché est visible
- TradingState est visible
- les 17 agents trading sont visibles
- buy_score / sell_score / hold_score sont visibles
- EXECUTE_LONG / EXECUTE_SHORT / REVIEW sont séparés de ALLOW / HOLD / BLOCK

Schéma 12 validé si :

- les cas signal clair / signal faible / risques multiples / contradictions sont visibles
- Sigma peut apparaître comme surveillance, pas autorité
- X108 reste le dernier mur

ORDRE DE FABRICATION RECOMMANDÉ DANS LE LOT 3

1. Schéma 09 — Banque flux complet

2. Schéma 11 — Trading flux complet

3. Schéma 10 — Banque cas X108

4. Schéma 12 — Trading cas X108

Pourquoi :

on construit d’abord les deux chemins complets,

puis on détaille les cas de décision.

RÉSULTAT ATTENDU APRÈS LE LOT 3

Après ce lot, on doit pouvoir dire :

Banque et Trading sont deux démonstrateurs complets du kernel.

La banque transforme une transaction en BankState,

puis en votes AUTHORIZE / ANALYZE / BLOCK,

puis X108 décide ALLOW / HOLD / BLOCK.

Le trading transforme un marché en TradingState,

puis en votes BUY / SELL / HOLD,

puis en EXECUTE_LONG / EXECUTE_SHORT / REVIEW,

puis X108 décide ALLOW / HOLD / BLOCK.

Phrase verrou du lot :

Le domaine propose.

Le kernel autorise.

LOT 4 — GPS / DÉFENSE / AVIATION + ECOM ANNEXE + AGENTVOTE

LOT 4 =

13. GPS / Défense / Aviation — flux complet

14. GPS — métriques et cas X108

15. E-commerce — annexe domaine

16. AgentVote — unité atomique commune

Objectif du lot :

Montrer la branche critique GPS / aviation comme démonstrateur profond du kernel.

Puis ajouter E-commerce en annexe sans le mettre au même niveau stratégique.

Et terminer par AgentVote, parce que c’est l’unité minimale qui relie tous les domaines au kernel.

Le doc fourni confirme que GPS doit rester une branche proof/Sigma confirmée, avec ses 6 agents, ses métriques propres et ses cas de sortie nominal/source manquante/source conflict/brownout/time skew.

SCHÉMA 13 — GPS / DÉFENSE / AVIATION : FLUX COMPLET

Rôle du schéma

Montrer comment un état de navigation critique devient une décision gouvernée.

Question à laquelle il répond

Que devient une trajectoire GPS / aviation quand elle entre dans Obsidia/X108 ?

Contenu obligatoire

Le schéma doit montrer 8 niveaux :

1. Payload GPS / aviation

2. GpsDefenseAviationState

3. 6 agents GPS

4. AgentVote[]

5. aggregate_gps_defense_aviation()

6. verdict métier candidat

7. Sigma / proof signals

8. Kernel X108 → Envelope → Audit

Structure visuelle complète

```
┌──────────────────────────────────────────┐
│ PAYLOAD GPS / DÉFENSE / AVIATION         │
├──────────────────────────────────────────┤
│ mission_id                               │
│ gps_available                            │
│ inertial_available                       │
│ radio_available                          │
│ elapsed_s                                │
│ min_required_elapsed_s                   │
```


position_confidence
trajectory_drift_score
source_conflict_score
brownout_score
time_skew_score
environment_risk_score
rollback_possible
attestation_ready
GpsDefenseAviationState
état critique navigation / défense / aviation
6 AGENTS GPS / DÉFENSE / AVIATION
SourceAvailabilityAgent
TrajectoryIntegrityAgent
SourceConflictAgent
TimeSkewAgent
BrownoutAgent
GpsAttestationReadinessAgent
AgentVote[]
claim · confidence · severity_hint
unknowns · contradictions · risk_flags
proposed_verdict · evidence_refs


aggregate_gps_defense_aviation()
trajectory_valid_score
recalc_score
degraded_score
abort_score
truth_score
sigma_score
mismatch_gap
proof_ready
deterministic
VERDICT MÉTIER CANDIDAT
TRAJECTORY_VALID
RECALC_TRAJECTORY
DEGRADED_NAVIGATION
ABORT_TRAJECTORY
≠ autorisation finale
META-AGENTS + SIGMA / PROOF SIGNALS
stability · integrity · governance
readiness · attestation · replay
KERNEL X108 — DERNIER MUR

```
| BLOCK > HOLD > ALLOW                          |
└──────────────────────┬──────────────────────┘
```

▼

```
┌──────────────────────────────────────────────┐
| CanonicalDecisionEnvelope                    |
| x108_gate · reason_code · severity           |
| decision_id · trace_id · attestation_ref     |
└──────────────────────┬──────────────────────┘
```

▼

```
┌──────────────────────────────────────────────┐
| Audit / hash / replay                        |
└──────────────────────────────────────────────┘
```

Agents GPS à afficher avec leur rôle

SourceAvailabilityAgent

```
→ vérifie GPS / inertiel / radio
```

TrajectoryIntegrityAgent

```
→ vérifie dérive trajectoire
```

SourceConflictAgent

```
→ détecte conflit entre sources
```

TimeSkewAgent

```
→ détecte désalignement temporel
```

BrownoutAgent

```
→ détecte instabilité énergie / brownout
```

GpsAttestationReadinessAgent

```
→ vérifie attestation + rollback
```

Métriques GPS obligatoires

trajectory_valid_score

recalc_score

degraded_score

abort_score

truth_score

sigma_score

mismatch_gap

proof_ready

deterministic

Message visuel

GPS ne veut pas dire “position OK”.

GPS signifie :

sources,

trajectoire,

temps,

énergie,

attestation,

rollback,

preuve.

Erreurs à éviter

Ne pas faire :

```
GPS → OK / pas OK → X108
```

Faire :

GPS / inertiel / radio / drift / conflict / time skew / brownout / attestation

```
→ GpsDefenseAviationState
→ 6 agents
→ aggregate_gps
→ verdict candidat
→ X108
```

Ne pas confondre :

TRAJECTORY_VALID

avec :

ALLOW

TRAJECTORY_VALID est un verdict métier candidat. ALLOW est une sortie kernel.

SCHÉMA 14 — GPS : MÉTRIQUES ET CAS X108

Rôle du schéma

Montrer les cas concrets où X108 autorise, suspend ou bloque une trajectoire.

Question à laquelle il répond

Comment X108 traite une navigation incertaine ?

Structure visuelle recommandée

CAS GPS	VERDICT MÉTIER CANDIDAT	SIGNAUX / MÉTRIQUES	X108 GATE	RAISON
Nominal	TRAJECTORY_VALID	sources OK / proof OK	ALLOW	GUARD_ALLOW
Source manquante	RECALC_TRAJECTORY	GPS / inertiel / radio miss	HOLD	UNKNOWNS_OR_CONFIDENCE_LOW
Source conflict fort	ABORT_TRAJECTORY	SOURCE_CONFLICT	BLOCK	CONTRADICTION_THRESHOLD
Brownout fort	DEGRADED_NAVIGATION	BROWNOUT / power uncertain	HOLD	RISK_FLAGS_REQUIRE_DELAY
Time skew fort	RECALC_TRAJECTORY	TIME_SKEW	HOLD	RISK_FLAGS_REQUIRE_DELAY
Attestation manquante	RECALC_TRAJECTORY	ATTESTATION_NOT_READY	HOLD	UNKNOWNS_OR_CONFIDENCE_LOW
Rollback impossible	RECALC / ABORT selon contexte	rollback_possible = false	HOLD/BLOCK	proof / safety boundary

Métriques à afficher autour du tableau

gps_available

inertial_available

radio_available

position_confidence

trajectory_drift_score

source_conflict_score

brownout_score

time_skew_score

environment_risk_score

rollback_possible

attestation_ready

Scores agrégés à afficher

trajectory_valid_score

recalc_score

degraded_score

abort_score

truth_score

sigma_score

mismatch_gap

Version “mur GPS”

GpsDefenseAviationState

↓

sources / trajectory / time / power / proof / rollback

↓

aggregate_gps_defense_aviation()

↓

TRAJECTORY_VALID / RECALC / DEGRADED / ABORT

↓

truth_score / sigma_score / mismatch_gap

↓

GuardX108

↓

ALLOW / HOLD / BLOCK

Cas à rendre visuellement très clairs

Cas 1 — nominal

```
gps_available = true
inertial_available = true
```

```
radio_available = true
```

source_conflict_score bas

time_skew_score bas

brownout_score bas

```
attestation_ready = true
rollback_possible = true
↓
```

TRAJECTORY_VALID

```
↓
```

ALLOW

Cas 2 — source manquante

```
gps_available = false
ou inertial_available = false
ou radio_available = false
↓
```

unknowns

```
↓
```

RECALC_TRAJECTORY

```
↓
```

HOLD

Cas 3 — source conflict fort

source_conflict_score élevé

```
↓
```

contradictions / SOURCE_CONFLICT

```
↓
```

ABORT_TRAJECTORY

↓

BLOCK

Cas 4 — brownout

brownout_score élevé

↓

risk_flags / power uncertainty

↓

DEGRADED_NAVIGATION

↓

HOLD

Cas 5 — time skew

time_skew_score élevé

↓

temporal alignment uncertain

↓

RECALC_TRAJECTORY

↓

HOLD

Message visuel

Une trajectoire critique ne passe pas parce qu’elle semble plausible.

Elle doit survivre aux sources,

à la cohérence temporelle,

à l’énergie,

à la preuve,

et au rollback.

Erreurs à éviter

Ne pas faire :

```
GPS OK → ALLOW
```

Faire :

sources + trajectoire + conflit + time skew + brownout + attestation + rollback

```
→ verdict candidat
→ X108
```

Ne pas cacher truth_score, sigma_score, mismatch_gap.

Ces trois métriques sont importantes parce qu’elles montrent la profondeur de la branche GPS.

SCHÉMA 15 — E-COMMERCE : ANNEXE DOMAINE

Rôle du schéma

Montrer que le kernel peut aussi recevoir un domaine e-commerce, mais sans le mettre au centre du support.

Question à laquelle il répond

Comment un domaine business non critique entre dans la même architecture ?

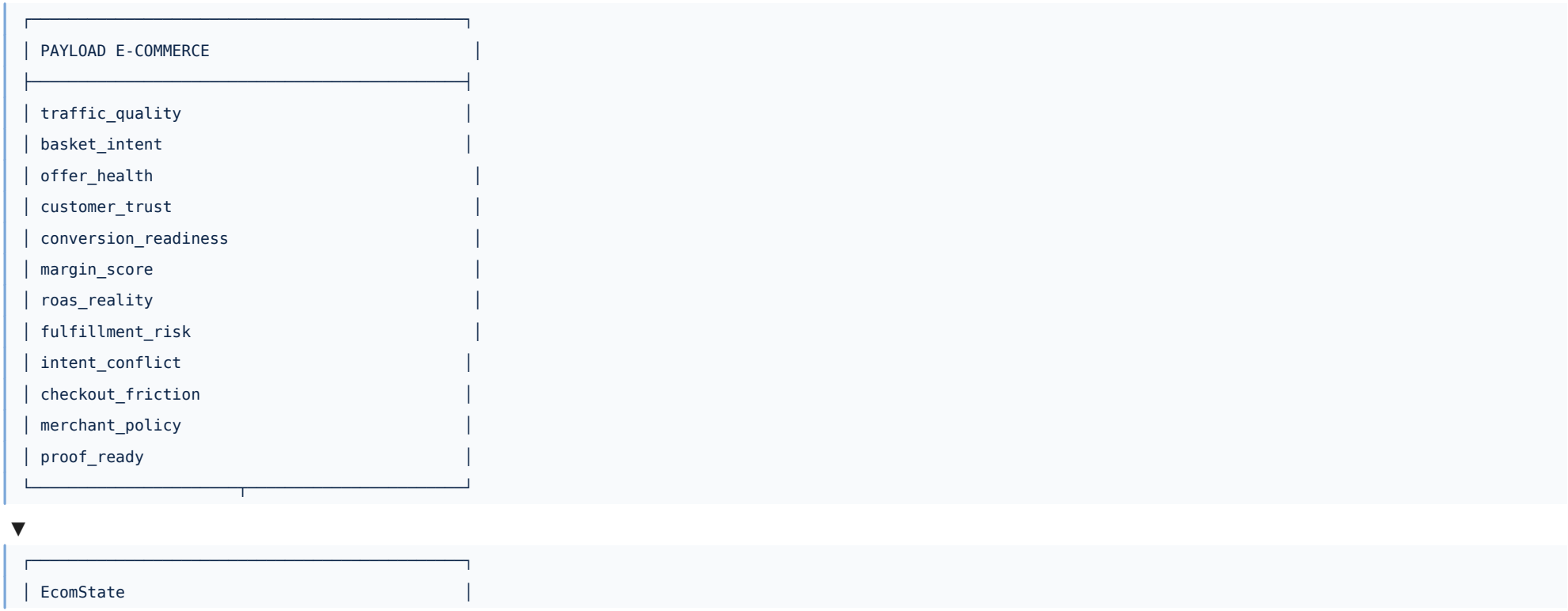
Statut du schéma

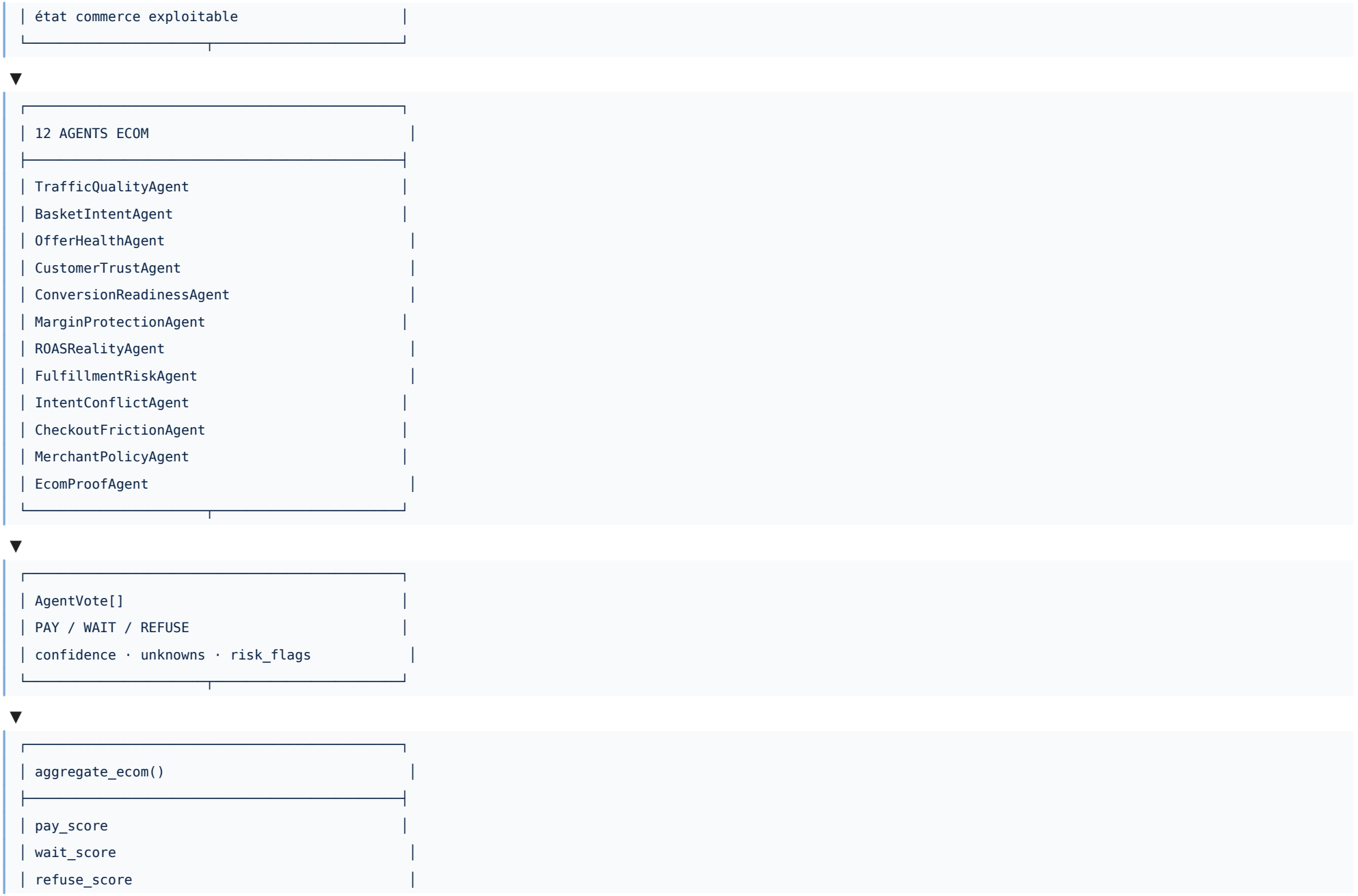
Annexe.

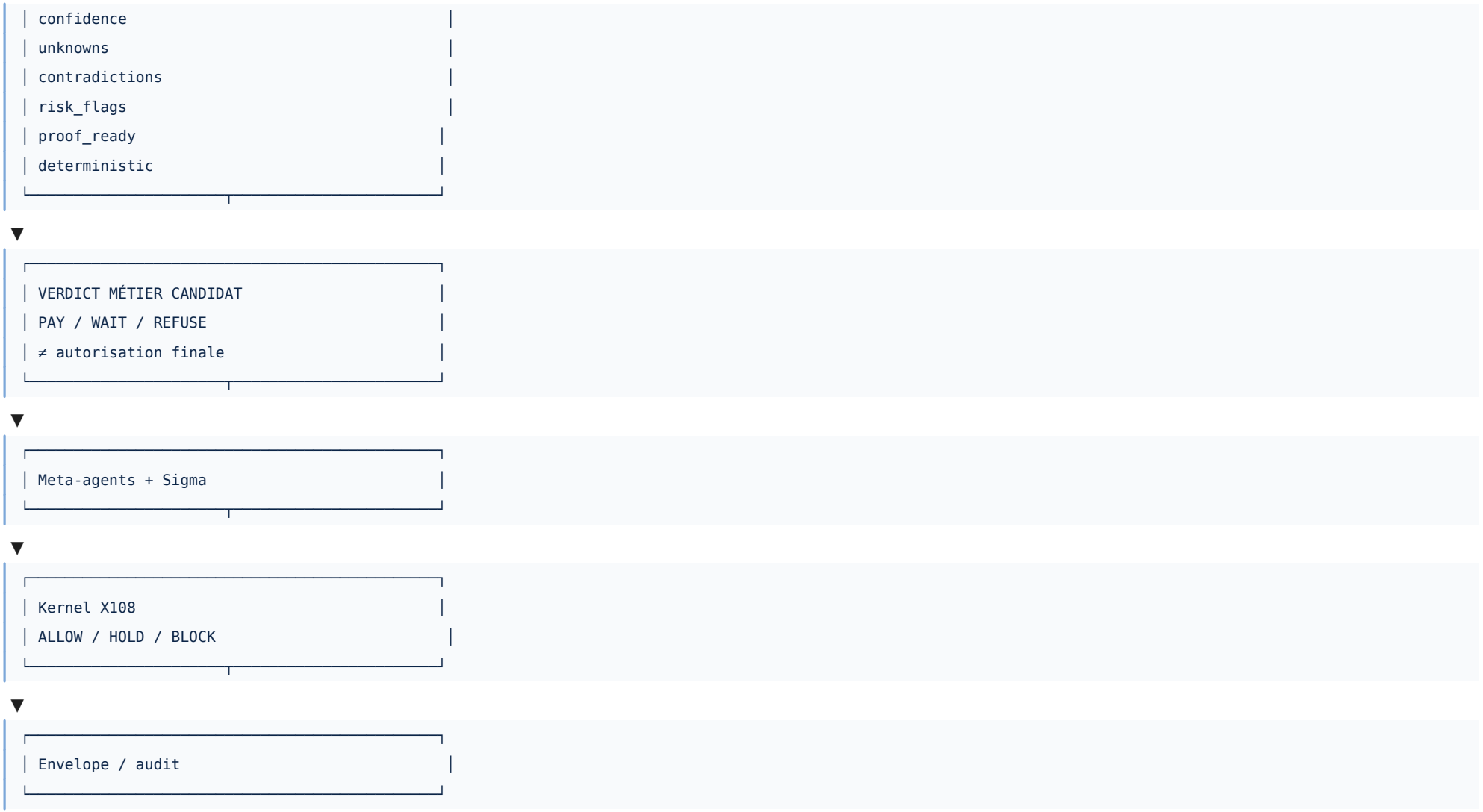
Pas central.

À garder après Banque / Trading / GPS.

Structure visuelle complète




état commerce exploitable
12 AGENTS ECOM
TrafficQualityAgent
BasketIntentAgent
OfferHealthAgent
CustomerTrustAgent
ConversionReadinessAgent
MarginProtectionAgent
ROASRealityAgent
FulfillmentRiskAgent
IntentConflictAgent
CheckoutFrictionAgent
MerchantPolicyAgent
EcomProofAgent
AgentVote[]
PAY / WAIT / REFUSE
confidence · unknowns · risk_flags
aggregate_ecom()
pay_score
wait_score
refuse_score



Métriques Ecom obligatoires

pay_score

wait_score

refuse_score

confidence

unknowns

contradictions

risk_flags

proof_ready

deterministic

Message visuel

E-commerce montre l’extensibilité du kernel.

Même un domaine business passe par :

state,

agents,

votes,

agrégation,

X108,

enveloppe.

Erreurs à éviter

Ne pas présenter Ecom comme égal au GPS en criticité.

Le statut correct :

```
Banque / Trading / GPS = démonstrateurs principaux
Ecom = annexe d’extensibilité
```

SCHÉMA 16 — AGENTVOTE : UNITÉ ATOMIQUE COMMUNE

Rôle du schéma

Montrer que tous les domaines parlent au kernel via le même type d'unité : AgentVote.

Question à laquelle il répond

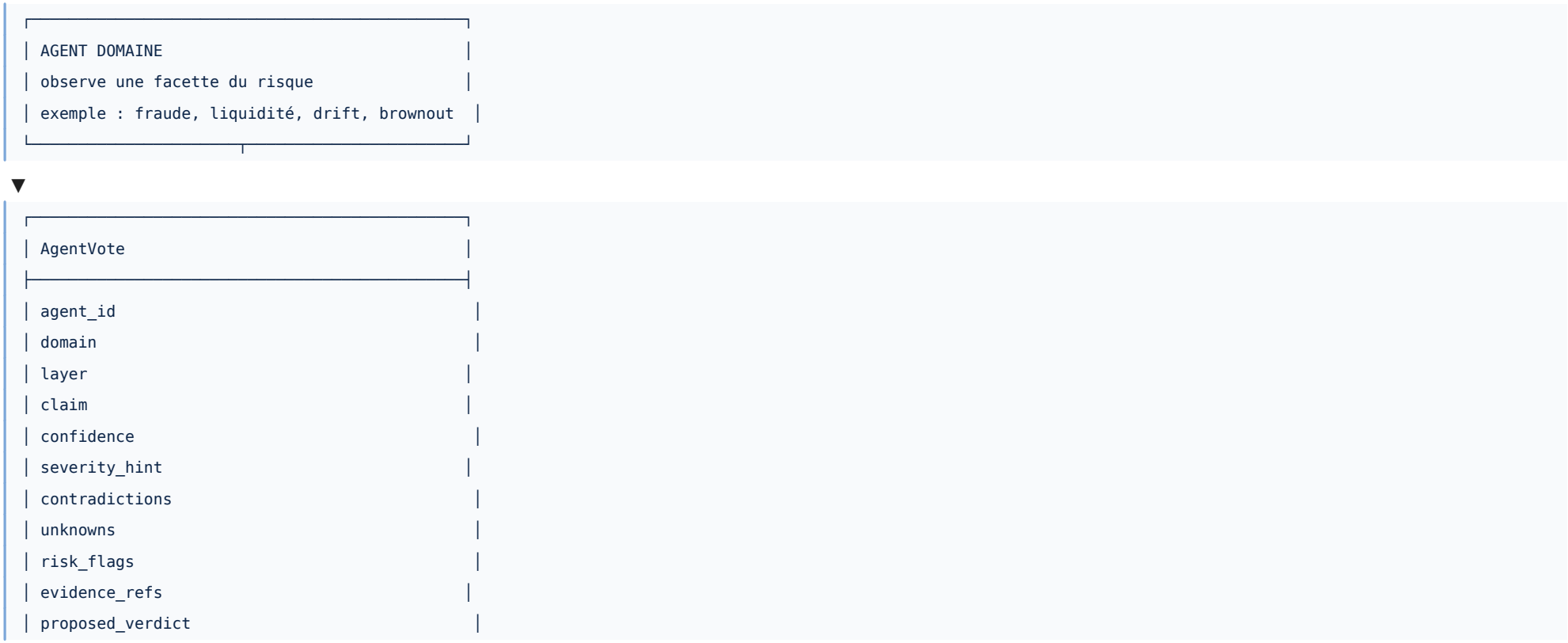
Qu'est-ce qu'un agent produit exactement ?

Pourquoi ce schéma est important

Sans ce schéma, les domaines ressemblent à des listes d'agents.

Avec ce schéma, on comprend que chaque agent transforme une observation métier en signal structuré comparable.

Structure visuelle recommandée



```
└──────────────────────┬──────────────────────┘
```

▼

```
┌──────────────────────────────────────────┐
│ SIGNAL STRUCTURÉ                         │
│ comparable entre domaines                │
│ exploitable par aggregate_*              │
│ non souverain                            │
└──────────────────────────────────────────┘
```

Champs obligatoires à expliquer

agent_id

identité de l’agent qui a produit le signal

domain

banque / trading / gps / ecom

layer

niveau de lecture :

observation,

interprétation,

contradiction,

preuve,

policy,

etc.

claim

ce que l’agent affirme ou observe

confidence

niveau de confiance du signal

severity_hint

gravité proposée par l’agent

contradictions

éléments incompatibles détectés

unknowns

éléments manquants / non prouvés / incertains

risk_flags

signaux de risque structurés

evidence_refs

références de preuve / hash / trace

proposed_verdict

verdict métier proposé par l’agent

pas décision finale

Exemples par domaine

Banque

FraudPatternAgent

```
→ proposed_verdict = BLOCK
→ risk_flags = FRAUD_PATTERN
→ confidence élevée
```

Trading

MomentumAgent

```
→ proposed_verdict = BUY
→ confidence moyenne/haute
```

GPS

SourceConflictAgent

```
→ proposed_verdict = ABORT_TRAJECTORY
→ contradictions = SOURCE_CONFLICT
```

Ecom

CheckoutFrictionAgent

```
→ proposed_verdict = WAIT
→ risk_flags = CHECKOUT_FRICTION
```

Message visuel

```
AgentVote = unité atomique du signal.
```

Un agent ne décide pas.

Il produit une pièce structurée que le système peut agréger.

À ne pas faire

Ne pas dessiner les agents comme des mini-décideurs.

Mauvais :

```
Agent → décision
```

Bon :

```
Agent → AgentVote → Aggregate → X108
```

MINI-CHECKLIST DE VALIDATION DU LOT 4

Schéma 13 validé si :

- payload GPS complet visible
- GpsDefenseAviationState visible
- les 6 agents GPS visibles
- aggregate_gps_defense_aviation visible
- truth_score / sigma_score / mismatch_gap visibles
- verdicts GPS séparés de ALLOW / HOLD / BLOCK

Schéma 14 validé si :

- nominal / source manquante / source conflict / brownout / time skew visibles
- chaque cas mène à un verdict métier puis X108
- les métriques GPS critiques sont visibles

Schéma 15 validé si :

- Ecom est marqué annexe
- Ecom montre l’extensibilité sans voler la place GPS
- PAY / WAIT / REFUSE est séparé de ALLOW / HOLD / BLOCK

Schéma 16 validé si :

- tous les champs AgentVote sont visibles
- on comprend qu’un agent ne décide pas
- proposed_verdict est séparé de x108_gate

ORDRE DE FABRICATION RECOMMANDÉ DANS LE LOT 4

1. Schéma 13 — GPS flux complet

2. Schéma 14 — GPS métriques et cas X108

3. Schéma 16 — AgentVote

4. Schéma 15 — Ecom annexe

Pourquoi :

GPS est le domaine critique du lot.

AgentVote est nécessaire pour relier GPS aux autres domaines.

Ecom doit rester annexe.

RÉSULTAT ATTENDU APRÈS LE LOT 4

Après ce lot, on doit pouvoir dire :

GPS / Défense / Aviation est un démonstrateur critique du kernel.

Le système ne vérifie pas seulement une position GPS.

Il vérifie des sources, une trajectoire, un conflit, un désalignement temporel, un brownout, une attestation et un rollback.

Chaque agent produit un AgentVote.

Ces votes deviennent un agrégat.

L’agrégat propose un verdict métier.

X108 tranche ALLOW / HOLD / BLOCK.

Ecom montre que la même architecture peut s’étendre à un domaine business, mais reste annexe.

Phrase verrou du lot :

Les agents observent.

Les votes structurent.

L’agrégat propose.

X108 tranche.

LOT 5 — MACHINERIE COMMUNE

LOT 5 =

17. DomainAggregate

18. Meta-agents

19. Sigma / proof signals

20. Triple confidence

Objectif du lot :

Montrer ce qui se passe entre les votes des agents et le kernel X108.

C’est la zone où beaucoup de gens se perdent :

- les agents ont parlé,

- mais le kernel n’a pas encore tranché,

- donc il faut montrer l’agrégation,

- les contrôles transverses,

- Sigma,

- puis les signaux de confiance proof.

> Le doc fourni confirme que le support doit montrer le noyau commun AgentVote → DomainAggregate → Meta-agents → Sigma / proof signals → GuardX108 → Envelope, et que ces couches doivent rester non souveraines : elles préparent, qualifient et enrichissent, mais ne décident pas à la place de X108.

SCHÉMA 17 — DOMAINAGGREGATE

Rôle du schéma

Montrer comment les votes d’agents deviennent un verdict métier candidat, sans devenir une décision kernel.

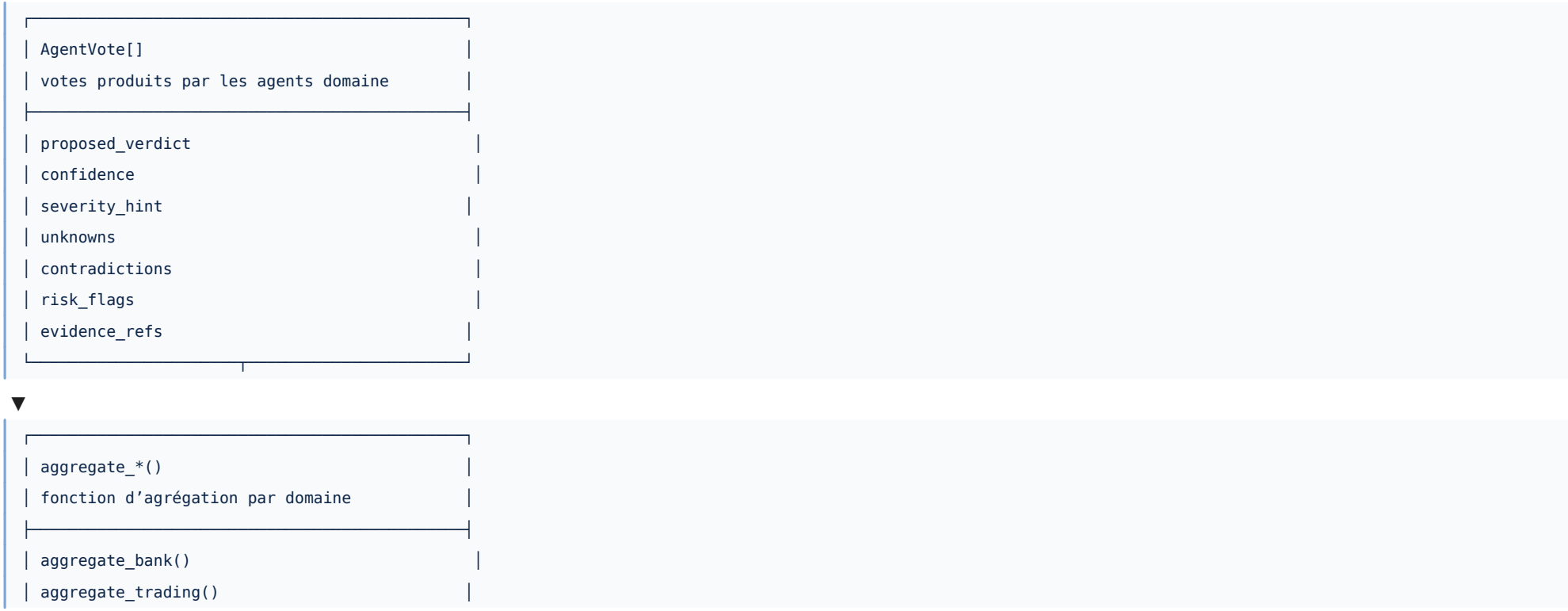
Question à laquelle il répond

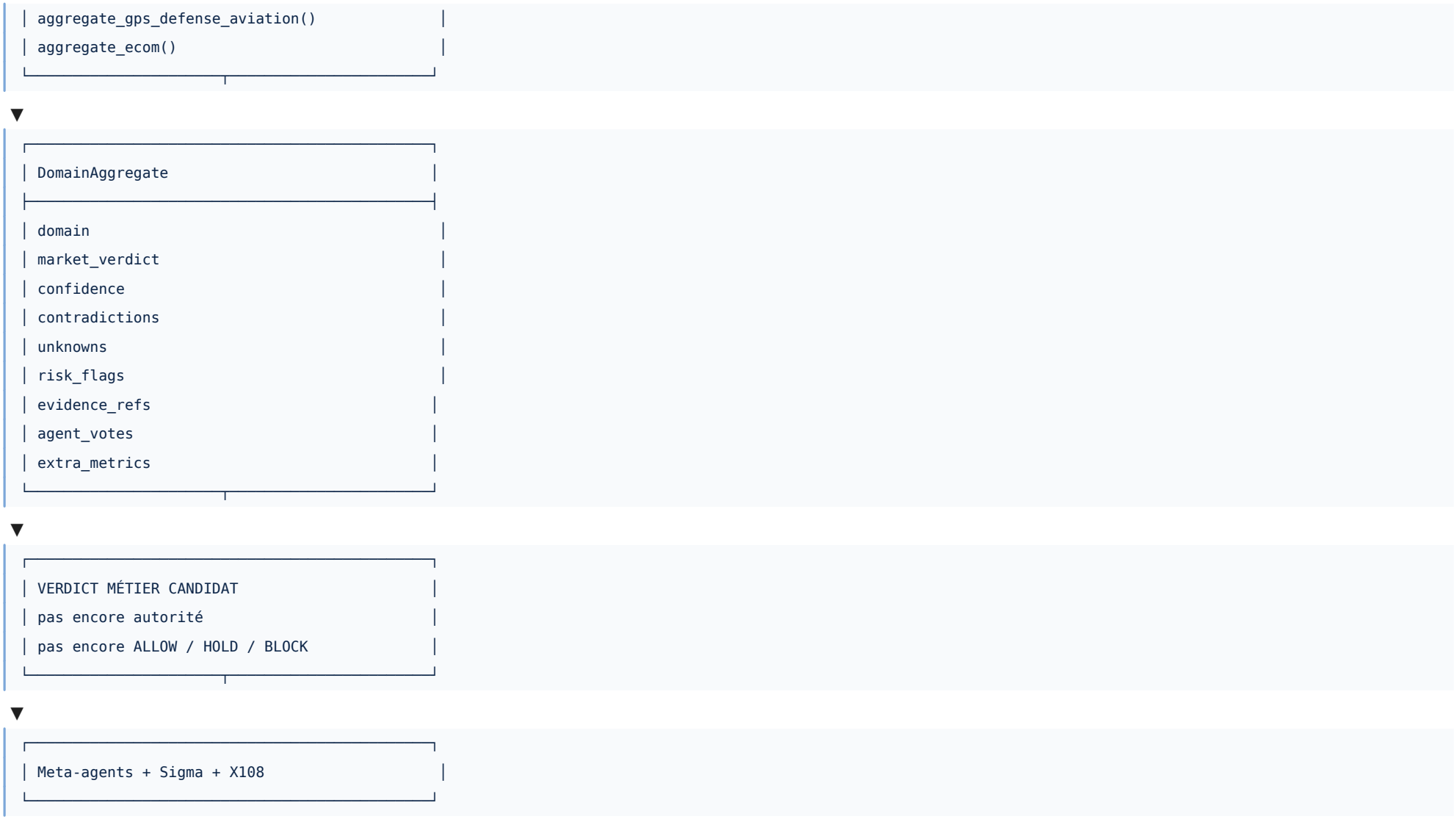
Comment plusieurs AgentVote deviennent une structure exploitable par X108 ?

Idée centrale

```
AgentVote = signal atomique.
DomainAggregate = synthèse métier.
X108 = autorité finale.
```

Structure visuelle complète





Champs obligatoires du DomainAggregate

domain

market_verdict

confidence

contradictions

unknowns

risk_flags

evidence_refs

agent_votes

extra_metrics

Ce que chaque champ signifie

domain

Domaine concerné :

bank

trading

gps_defense_aviation

ecom

market_verdict

Verdict métier candidat.

Il dit ce que le domaine propose.

Il ne dit pas si l’action est autorisée.

confidence

Confiance agrégée du domaine.

Elle alimente le kernel, mais ne décide pas seule.

contradictions

Liste des conflits détectés.

Peut mener à BLOCK si seuil critique atteint.

unknowns

Liste des inconnues / données manquantes.

Peut mener à HOLD.

risk_flags

Risques structurés.

Peut mener à HOLD ou BLOCK selon intensité.

evidence_refs

Références de preuve / hash / traces.

Préparent audit, replay, attestation.

agent_votes

Votes d’origine conservés.

Permet de rejouer / auditer la décision.

extra_metrics

Métriques propres au domaine.

C’est ici que la profondeur métier reste visible.

DomainAggregate par domaine

Banque

AgentVote[]

↓

aggregate_bank()

↓

DomainAggregate(bank)

Entrées métier

AUTHORIZE

ANALYZE

BLOCK

Métriques obligatoires

authorize_score

analyze_score

block_score

confidence

proof_ready

deterministic

Verdict candidat

```
market_verdict =
```

AUTHORIZE

ou ANALYZE

ou BLOCK

Lecture visuelle

AUTHORIZE / ANALYZE / BLOCK

```
= verdict bancaire candidat
```

ALLOW / HOLD / BLOCK

```
= décision kernel après X108
```

Trading / Bourse

AgentVote[]

```
↓
```

aggregate_trading()

```
↓
```

DomainAggregate(trading)

Entrées métier

BUY

SELL

HOLD

Métriques obligatoires

buy_score

sell_score

hold_score

confidence

proof_ready

deterministic

Verdict candidat

```
market_verdict =
```

EXECUTE_LONG

ou EXECUTE_SHORT

ou REVIEW

Lecture visuelle

BUY / SELL / HOLD

```
= votes agents
```

EXECUTE_LONG / EXECUTE_SHORT / REVIEW

```
= verdict trading candidat
```

ALLOW / HOLD / BLOCK

```
= décision kernel
```

GPS / Défense / Aviation

AgentVote[]

```
↓
```

aggregate_gps_defense_aviation()

```
↓
```

DomainAggregate(gps_defense_aviation)

Entrées métier

TRAJECTORY_VALID

RECALC_TRAJECTORY

DEGRADED_NAVIGATION

ABORT_TRAJECTORY

Métriques obligatoires

trajectory_valid_score

recalc_score

degraded_score

abort_score

truth_score

sigma_score

mismatch_gap

proof_ready

deterministic

Verdict candidat

```
market_verdict =
```

TRAJECTORY_VALID

ou RECALC_TRAJECTORY

ou DEGRADED_NAVIGATION

ou ABORT_TRAJECTORY

Lecture visuelle

Le GPS ne produit pas juste OK / pas OK.

Il produit une lecture trajectoire + vérité + Sigma + mismatch.

E-commerce

AgentVote[]

```
↓
```

aggregate_ecom()

```
↓
```

DomainAggregate(ecom)

Entrées métier

PAY

WAIT

REFUSE

Métriques obligatoires

pay_score

wait_score

refuse_score

confidence

proof_ready

deterministic

Verdict candidat

```
market_verdict =
```

PAY

ou WAIT

ou REFUSE

Statut

Annexe d’extensibilité.

Pas domaine central du support.

Message visuel du schéma

L’agrégation synthétise.

Elle ne gouverne pas.

Le verdict métier candidat doit encore survivre à :

meta-agents,

Sigma,

X108.

À ne pas faire

Ne pas dessiner :

```
AgentVote[] → DomainAggregate → décision finale
```

Dessiner :

AgentVote[]

```
→ DomainAggregate
→ Meta-agents
→ Sigma
→ X108
→ décision finale
```

SCHÉMA 18 — META-AGENTS

Rôle du schéma

Montrer la couche qui contrôle ce que les agents domaine ne couvrent pas directement :

preuve,

trace,

ticket,

policy,

attestation,

replay,

cohérence,

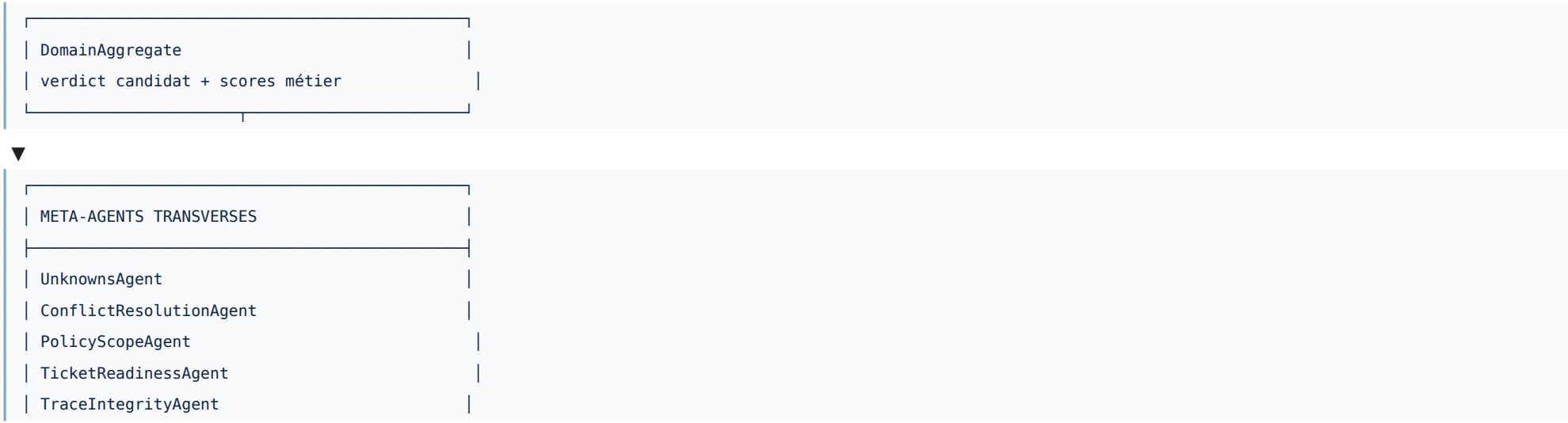
override humain,

sévérité.

Question à laquelle il répond

Qu’est-ce qui sécurise la décision entre le verdict métier et X108 ?

Structure visuelle complète





Liste détaillée des meta-agents

UnknownsAgent

Rôle

Détecte les données manquantes ou incertaines.

Produit

unknowns supplémentaires

signal HOLD potentiel

Exemple

source GPS manquante

contrepartie inconnue

payload incomplet

attestation absente

ConflictResolutionAgent

Rôle

Détecte les contradictions transverses.

Produit

contradictions supplémentaires

signal BLOCK potentiel

Exemple

agent A propose AUTHORIZE

agent B détecte FRAUD_PATTERN

ou

GPS annonce trajectoire valide

mais SourceConflictAgent détecte conflit fort

PolicyScopeAgent

Rôle

Vérifie si la décision reste dans le périmètre autorisé.

Produit

policy flags

risk_flags

boundary alerts

Exemple

action hors scope

domaine non autorisé

opération non conforme au périmètre

TicketReadinessAgent

Rôle

Vérifie si la décision nécessite un ticket ou une validation.

Produit

ticket_required

ticket_id si nécessaire

readiness signal

Exemple

action critique

montant élevé

risque élevé

action irréversible

TraceIntegrityAgent

Rôle

Vérifie que la décision peut être tracée.

Produit

trace_integrity

trace_id readiness

evidence_refs readiness

Exemple

pas de trace

trace incomplète

source non hashable

AttestationReadinessAgent

Rôle

Vérifie que la décision peut être attestée.

Produit

attestation_ref

attestation_ready

risk flag si attestation absente

Exemple

GPS attestation absente

preuve de décision incomplète

hash non prêt

HumanOverrideEligibilityAgent

Rôle

Détermine si une intervention humaine est admissible ou nécessaire.

Produit

human_override_eligible

human_review_required

Attention visuelle

> `Human override ≠ contournement X108.`

SeverityClassifierAgent

Rôle

Classe la gravité finale.

Produit

severity

S0 / S1 / S2 / S3 / S4

Exemple

```
S0 = nominal
S2 = risque modéré
S4 = critique / stabilité cassée
```

ReplayConsistencyAgent

Rôle

Vérifie que la décision pourra être rejouée.

Produit

replay_consistency

audit readiness

Exemple

input manquant

evidence_refs insuffisants

trace incomplète

ProofConsistencyAgent

Rôle

Vérifie la cohérence de preuve.

Produit

proof_consistency

risk flag si preuve incohérente

Exemple

verdict métier sans preuve

attestation absente

incohérence entre risk_flags et reason_code

Message visuel du schéma

Les meta-agents sécurisent le cadre.

Ils ne remplacent pas le domaine.

Ils ne remplacent pas X108.

À ne pas faire

Ne pas représenter les meta-agents comme :

mini-kernel

Les représenter comme :

contrôle transverse

avant X108

SCHÉMA 19 — SIGMA / PROOF SIGNALS

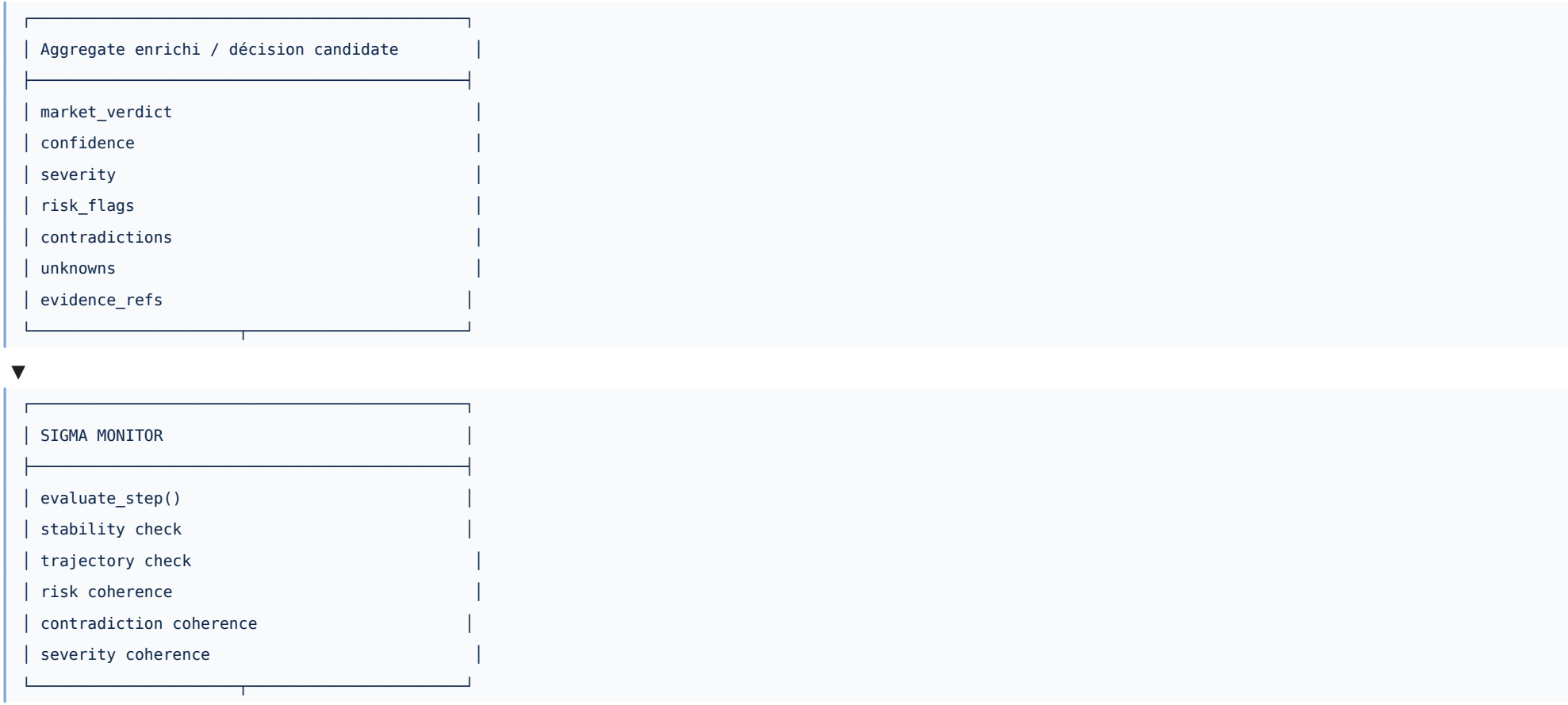
Rôle du schéma

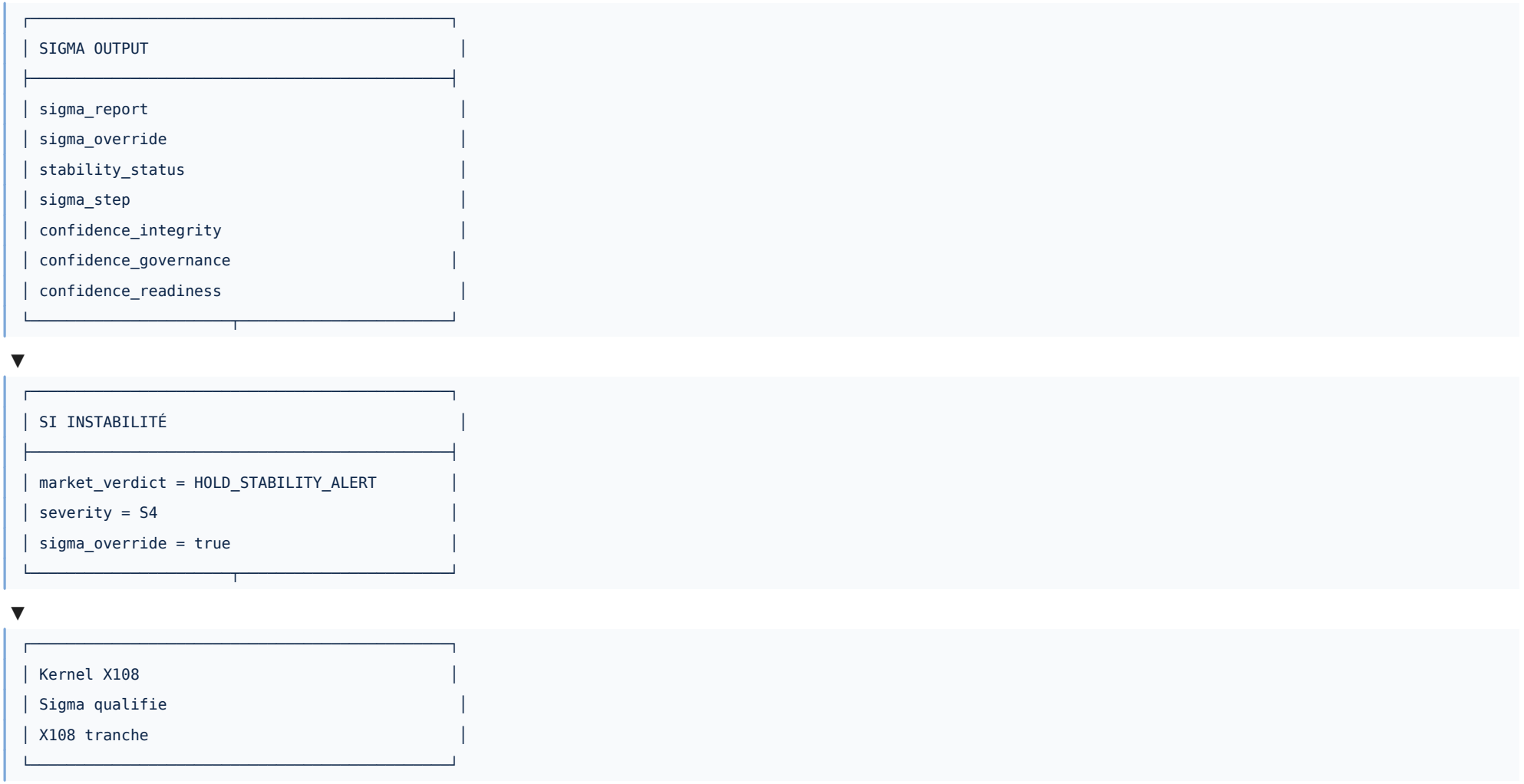
Montrer que Sigma est une couche de qualification de stabilité, pas une autorité finale.

Question à laquelle il répond

Où intervient la stabilité dynamique ?

Structure visuelle complète





Entrées Sigma à afficher

severity

risk_flags

contradictions

unknowns

confidence

market_verdict

evidence_refs

Sorties Sigma à afficher

sigma_report

sigma_override

sigma_step

stability_status

confidence_integrity

confidence_governance

confidence_readiness

Cas Sigma à montrer

Cas stable

risk_flags faibles

contradictions absentes

severity basse

confidence suffisante

```
↓
sigma_report = stable
sigma_override = false
```

Cas instable

risk_flags critiques

contradictions fortes

severity élevée

confidence incohérente

```
↓
sigma_report = instability detected
sigma_override = true
```

HOLD_STABILITY_ALERT

```
severity = S4
```

Message visuel

Sigma est un gyroscope.

X108 est le verrou.

Autre formulation

Sigma qualifie la stabilité.

X108 décide l’autorité.

À ne pas faire

Ne pas dessiner :

```
Sigma → ALLOW
```

Dessiner :

```
Sigma → proof signals → X108 → ALLOW / HOLD / BLOCK
```

SCHÉMA 20 — TRIPLE CONFIDENCE

Rôle du schéma

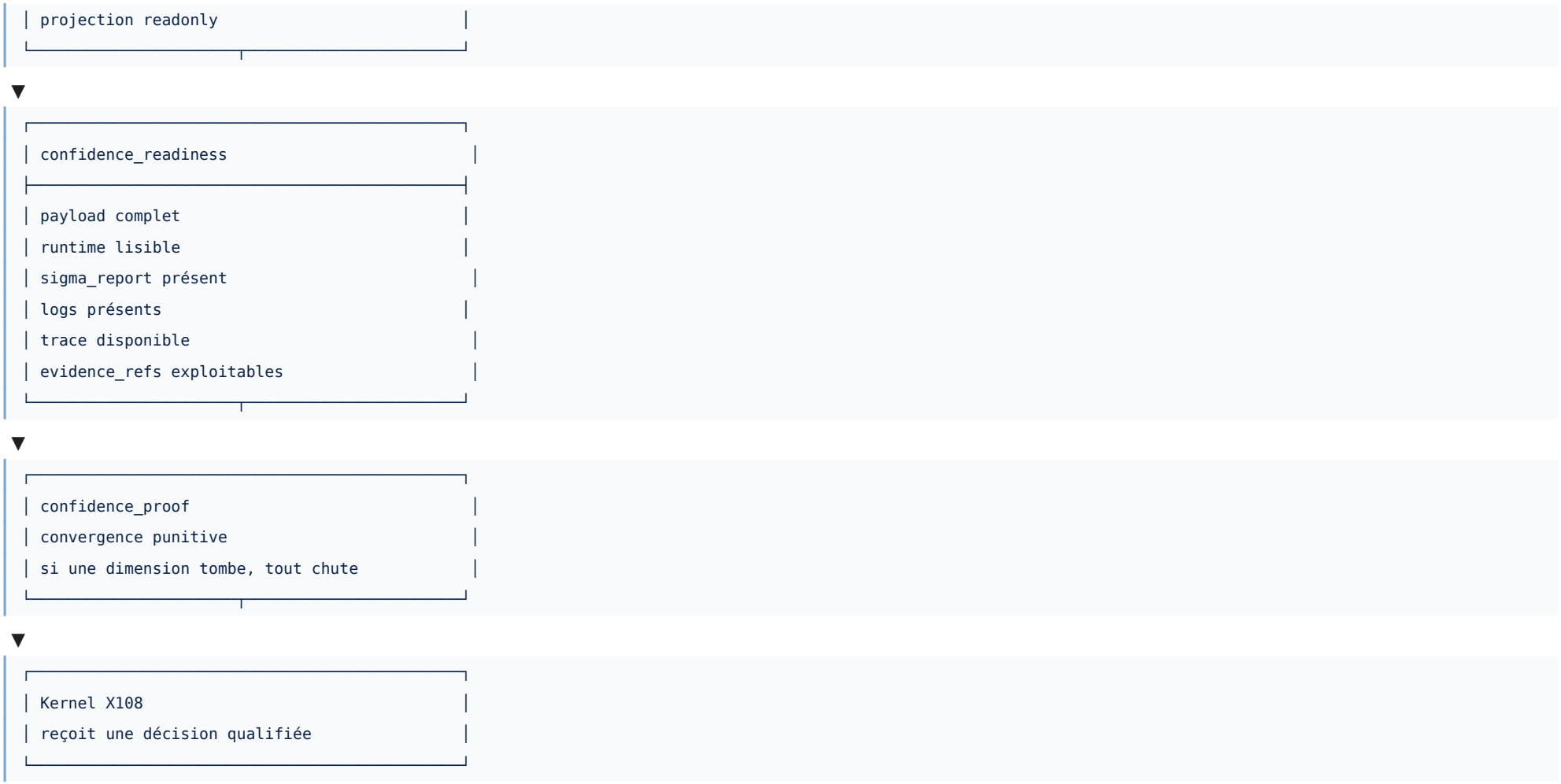
Montrer pourquoi la confiance unique ne suffit pas.

Question à laquelle il répond

Pourquoi un bon score local ne suffit pas ?

Structure visuelle complète

```
┌──────────────────────────────────────────────┐
│ SIGNAL CANDIDAT                              │
│ verdict métier + aggregate + sigma signals   │
└──────────────────────┬───────────────────────┘
                       ▼
┌──────────────────────────────────────────────┐
│ confidence_integrity                         │
├──────────────────────────────────────────────┤
│ cohérence interne                            │
│ contradictions                               │
│ risk_flags                                   │
│ stabilité du verdict                         │
│ cohérence des champs                         │
└──────────────────────┬───────────────────────┘
                       ▼
┌──────────────────────────────────────────────┐
│ confidence_governance                        │
├──────────────────────────────────────────────┤
│ decision_authority = KX108_ONLY              │
│ no-bypass                                    │
│ no-write                                     │
│ GuardX108 respecté                           │
│ Sigma non souverain                          │
```



Les 3 confiances

1. confidence_integrity

Mesure la cohérence interne de la décision.

À afficher :

structural coherence

contradiction cleanliness

risk consistency

verdict stability

field consistency

Déclencheurs de baisse :

contradictions fortes

risk_flags incohérents

verdict instable

champs contradictoires

2. confidence_governance

Mesure le respect de la gouvernance X108.

À afficher :

```
decision_authority = KX108_ONLY
```

boundary_safety

gate_consistency

no-bypass

no-write

Sigma non souverain

projection readonly

Déclencheurs de baisse :

contournement possible

autorité ambiguë

action hors X108

mémoire/write non autorisé

projection qui prétend décider

3. confidence_readiness

Mesure la maturité d’exposition / routage / audit.

À afficher :

input_completeness

runtime_readiness

audit_traceability

sigma_report presence

log availability

trace availability

evidence_refs readiness

Déclencheurs de baisse :

payload incomplet

trace absente

sigma_report absent

evidence_refs insuffisants

runtime non lisible

confidence_proof

À afficher comme :

lecture punitive globale

Formule visuelle possible :

```
confidence_proof =
```

harmonic / punitive convergence

of

integrity + governance + readiness

Ne pas afficher comme :

moyenne arithmétique simple

Le message important :

Si une dimension critique chute,

la confiance proof chute.

Message visuel

Une décision ne doit pas seulement être confiante.

Elle doit être intègre, gouvernée et prête à être auditée.

Formulation courte

Intégrité.

Gouvernance.

Readiness.

Sinon HOLD.

À ne pas faire

Ne pas dessiner :

```
confidence élevée → ALLOW
```

Dessiner :

confidence métier

+ integrity

+ governance

+ readiness

```
→ X108
```

MINI-CHECKLIST DE VALIDATION DU LOT 5

Schéma 17 validé si :

- DomainAggregate est séparé de AgentVote.
- Les metrics par domaine sont visibles.
- market_verdict est bien marqué “candidat”.
- extra_metrics est présent.
- evidence_refs est présent.

Schéma 18 validé si :

- Les 10 meta-agents sont visibles.
- Leur rôle transverse est clair.
- Ils enrichissent avant X108.
- Ils ne décident pas.

Schéma 19 validé si :

- Sigma reçoit severity / risk_flags / contradictions.
- sigma_report est visible.
- sigma_override est visible.
- HOLD_STABILITY_ALERT est visible.
- Sigma ne produit pas ALLOW / HOLD / BLOCK directement.

Schéma 20 validé si :

- confidence_integrity est visible.
- confidence_governance est visible.
- confidence_readiness est visible.
- confidence_proof est punitive, pas moyenne douce.
- La position est avant X108, pas après.

ORDRE DE FABRICATION RECOMMANDÉ DANS LE LOT 5

1. Schéma 17 — DomainAggregate

2. Schéma 18 — Meta-agents

3. Schéma 19 — Sigma / proof signals

4. Schéma 20 — Triple confidence

Pourquoi :

DomainAggregate consolide les votes.

Meta-agents enrichissent le cadre.

Sigma qualifie la stabilité.

Triple confidence qualifie la preuve.

Puis X108 tranche.

RÉSULTAT ATTENDU APRÈS LE LOT 5

Après ce lot, on doit pouvoir dire :

Les agents ne parlent pas directement au kernel comme des voix isolées.

Ils produisent des votes.

Ces votes sont agrégés en DomainAggregate.

Les meta-agents ajoutent trace, ticket, policy, attestation, replay et proof consistency.

Sigma qualifie la stabilité.

Les trois confiances qualifient intégrité, gouvernance et readiness.

Seulement ensuite X108 tranche.

Phrase verrou du lot :

Les agents signalent.

L’agrégation structure.

Les meta-agents sécurisent.

Sigma qualifie.

X108 tranche.

LOT 6 — SORTIE / PREUVE / EXPOSITION READONLY

LOT 6 =

21. CanonicalDecisionEnvelope

22. Audit / hash / replay

23. Exposition humaine readonly

24. OS / runtime / bus

> Correction verrouillée : on ne dit plus “projection française” comme couche canonique. On dit “exposition humaine readonly”. Le français peut être une forme d’exposition, mais le kernel sort d’abord une enveloppe JSON vérifiable. Le doc source place bien la sortie après X108 sous forme CanonicalDecisionEnvelope → Audit / Hash / Replay → Projection readonly, avec JSON · API · terminal · français contrôlé, donc le français doit rester une modalité d’exposition, pas le cœur du bloc.

SCHÉMA 21 — CANONICAL DECISION ENVELOPE

Rôle du schéma

Montrer que le kernel ne sort pas une phrase, une opinion, une réponse libre ou une explication.

Il sort un objet de décision canonique.

Question traitée

Sous quelle forme sort une décision X108 ?

Schéma complet



```
| confidence_readiness                          |
└──────────────────────┬────────────────────────┘
↓
┌──────────────────────────────────────────────┐
| OBJET VÉRIFIABLE                             |
| pas phrase                                   |
| pas opinion                                  |
| pas intuition                                |
| pas génération libre                         |
| pas autorité linguistique                    |
└──────────────────────────────────────────────┘
```

Champs à rendre visibles

Identité décisionnelle

domain

decision_id

trace_id

source

Rôle :

identifier la décision,

le domaine,

le chemin d’exécution,

et permettre le replay.

Verdict métier candidat

market_verdict

confidence

Exemples :

Banque :

AUTHORIZE / ANALYZE / BLOCK

Trading :

EXECUTE_LONG / EXECUTE_SHORT / REVIEW

GPS :

TRAJECTORY_VALID / RECALC_TRAJECTORY / DEGRADED_NAVIGATION / ABORT_TRAJECTORY

Ecom :

PAY / WAIT / REFUSE

Point critique :

```
market_verdict ≠ x108_gate
```

Décision kernel

x108_gate

Valeurs :

ALLOW

HOLD

BLOCK

C’est le champ souverain.

Signaux de risque

contradictions

unknowns

risk_flags

severity

reason_code

Rôle :

expliquer pourquoi X108 a bloqué, suspendu ou autorisé.

Preuve / traçabilité

evidence_refs

attestation_ref

ticket_required

ticket_id

metrics

raw_engine

Rôle :

préparer audit, hash, replay, ticket, preuve.

Qualification Sigma / Proof

sigma_report

confidence_integrity

confidence_governance

confidence_readiness

Rôle :

conserver la lecture de stabilité,

d’intégrité,

de gouvernance,

et de readiness.

Version compacte du schéma

X108

```
↓
```

CanonicalDecisionEnvelope

```
├─ identité : domain / decision_id / trace_id
```

```
├─ métier : market_verdict / confidence
├─ risques : unknowns / contradictions / risk_flags
├─ décision : x108_gate / reason_code / severity
├─ preuve : evidence_refs / attestation_ref / ticket
├─ métriques : metrics / raw_engine
└─ Sigma : sigma_report / triple confidence
```

Message visuel

Le kernel ne sort pas une opinion.

Il sort une enveloppe canonique.

Erreurs à éviter

Ne pas dessiner :

```
X108 → réponse française
```

Dessiner :

```
X108 → CanonicalDecisionEnvelope → exposition readonly
```

SCHÉMA 22 — AUDIT / HASH / REPLAY

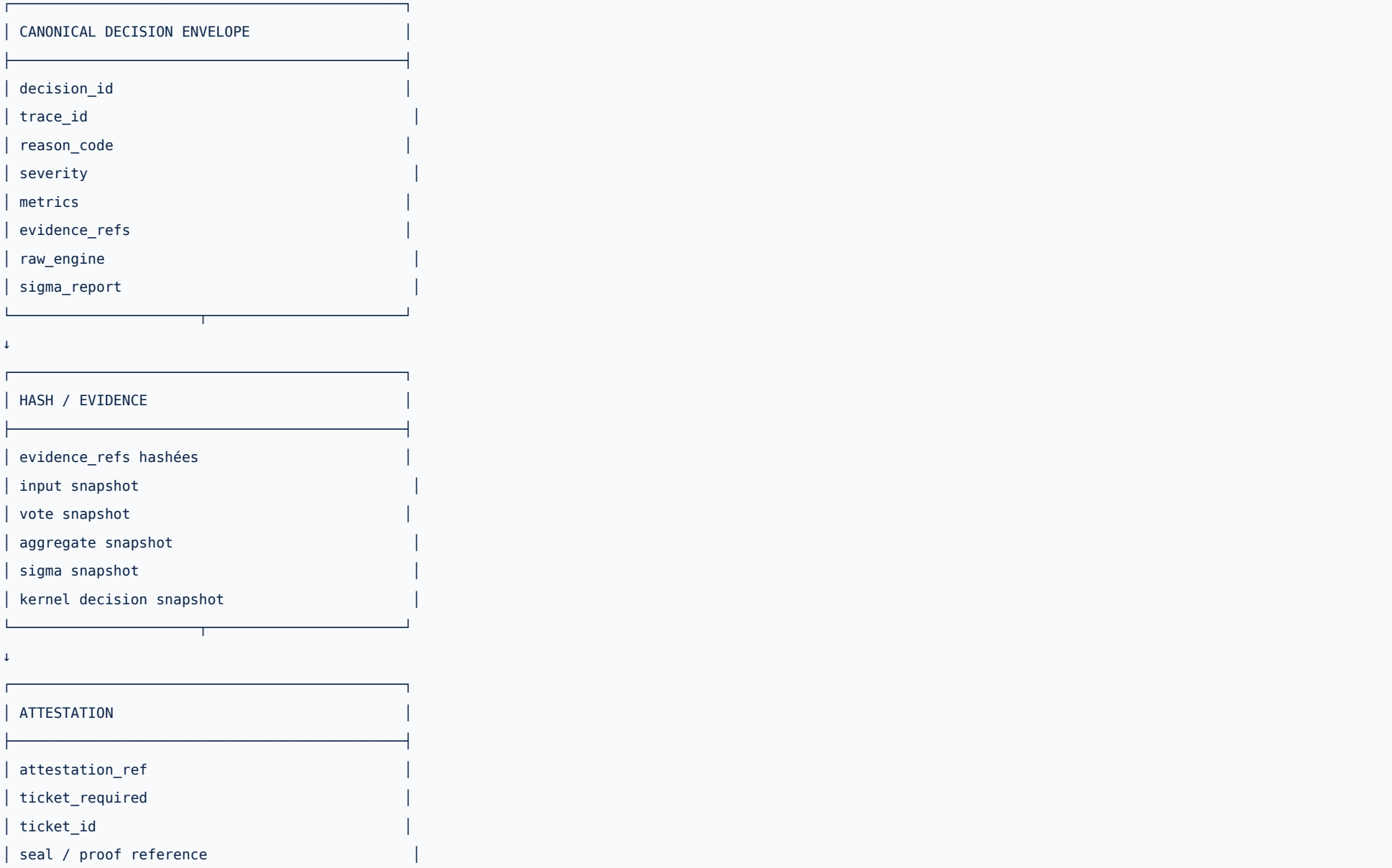
Rôle du schéma

Montrer comment une décision devient vérifiable, rejouable et opposable.

Question traitée

Comment prouver qu’une décision X108 n’a pas été inventée ou contournée ?

Schéma complet



```
└──────────────────────┬──────────────────────┘
                       ↓
┌──────────────────────────────────────────────┐
│ AUDIT STORE                                  │
├──────────────────────────────────────────────┤
│ audit.log                                    │
│ audit.chain                                  │
│ trace store                                  │
│ decision archive                             │
└──────────────────────┬──────────────────────┘
                       ↓
┌──────────────────────────────────────────────┐
│ REPLAY                                       │
├──────────────────────────────────────────────┤
│ trace_id                                     │
│ reload input                                 │
│ reload AgentVote[]                           │
│ reload DomainAggregate                       │
│ reload sigma_report                          │
│ reload CanonicalDecisionEnvelope             │
│ verify same decision                         │
└──────────────────────┬──────────────────────┘
                       ↓
┌──────────────────────────────────────────────┐
│ PREUVE                                       │
│ décision rejouable                           │
│ décision auditée                             │
│ décision non-contournable                    │
└──────────────────────────────────────────────┘
```

Étapes obligatoires

1. Capture de l’enveloppe

CanonicalDecisionEnvelope

```
→ decision snapshot
```

Contient :

x108_gate

reason_code

severity

trace_id

decision_id

metrics

sigma_report

2. Références de preuve

evidence_refs

```
→ hash / références courtes
→ attestation_ref
```

Rôle :

relier la décision aux signaux qui l’ont produite.

3. Journalisation

audit.log

audit.chain

decision archive

trace store

Rôle :

garder une chaîne consultable et vérifiable.

4. Replay

trace_id

```
→ récupérer input
```

```
→ récupérer votes
→ récupérer aggregate
→ récupérer sigma
→ récupérer envelope
→ vérifier cohérence
```

Rôle :

confirmer que la même structure mène au même résultat.

Version courte

Envelope

```
→ evidence_refs
→ hash
→ attestation_ref
→ audit.log
→ audit.chain
→ replay(trace_id)
→ verification
```

Message visuel

Le replay ferme la preuve.

Erreur à éviter

Ne pas représenter l’audit comme une couche qui redécide.

Mauvais :

```
X108 → audit → décision finale
```

Bon :

```
X108 → décision finale → audit / replay
```

L’audit vérifie après. Il ne gouverne pas.

SCHÉMA 23 — EXPOSITION HUMAINE READONLY

Rôle du schéma

Expliquer comment la décision devient lisible par un humain, sans que cette couche devienne une autorité.

Question traitée

Comment le système expose une décision sans la modifier ?

Correction centrale

Ancien intitulé à éviter :

Projection française readonly

Nouveau titre :

Exposition humaine readonly

Le français devient une option d’exposition, pas le nom de la couche.

Schéma complet



```
| gravité                                    |
| trace visible                              |
| preuve / audit si utile                    |
└──────────────────────┬─────────────────────┘
↓
┌────────────────────────────────────────────┐
| BOUNDARIES                                 |
├────────────────────────────────────────────┤
| readonly = true                            |
| emits_act = false                          |
| memory_write = false                       |
| decision_authority = KX108_ONLY            |
| cannot_authorize = true                    |
| cannot_override_kernel = true              |
└────────────────────────────────────────────┘
```

Ce que l’exposition peut faire

afficher

résumer

rendre lisible

retourner du JSON

répondre via API

imprimer dans le terminal

produire des logs opérateur

projeter en français si la route le prévoit

montrer reason_code

montrer severity

montrer trace_id

Ce qu’elle ne peut pas faire

autoriser

bloquer

modifier la décision

réécrire l’enveloppe

émettre ACT

écrire en mémoire

contourner X108

remplacer le kernel

Exemple de mapping readonly

```
x108_gate = HOLD
reason_code = UNKNOWNS_OR_CONFIDENCE_LOW
severity = S2
trace_id = abc123
```

→ exposition humaine :

“Décision suspendue. Le contexte est incomplet ou la confiance est insuffisante. Aucune action réelle n’est autorisée. Trace : abc123.”

```
x108_gate = BLOCK
reason_code = CONTRADICTION_THRESHOLD_REACHED
severity = S4
```

→ exposition humaine :

“Décision bloquée. Le seuil de contradiction ou un risque critique a été atteint. Action interdite.”

```
x108_gate = ALLOW
reason_code = GUARD_ALLOW
severity = S0
```

→ exposition humaine :

“Décision autorisée. Aucun signal critique n’a bloqué ou suspendu l’action. La décision reste tracée.”

Message visuel

La sortie lisible expose.

Elle ne gouverne pas.

Phrase canonique corrigée

Le français peut exposer la décision.

Il ne la produit pas.

Il ne la modifie pas.

Il ne l’autorise pas.

À exclure

Ne pas mentionner dans ce schéma :

Gemini

BankWorld

UI privée

TRPC explainDecision

Brody mémoire

Graphiti

pseudo-LLM

SCHÉMA 24 — OS / RUNTIME / BUS

Rôle du schéma

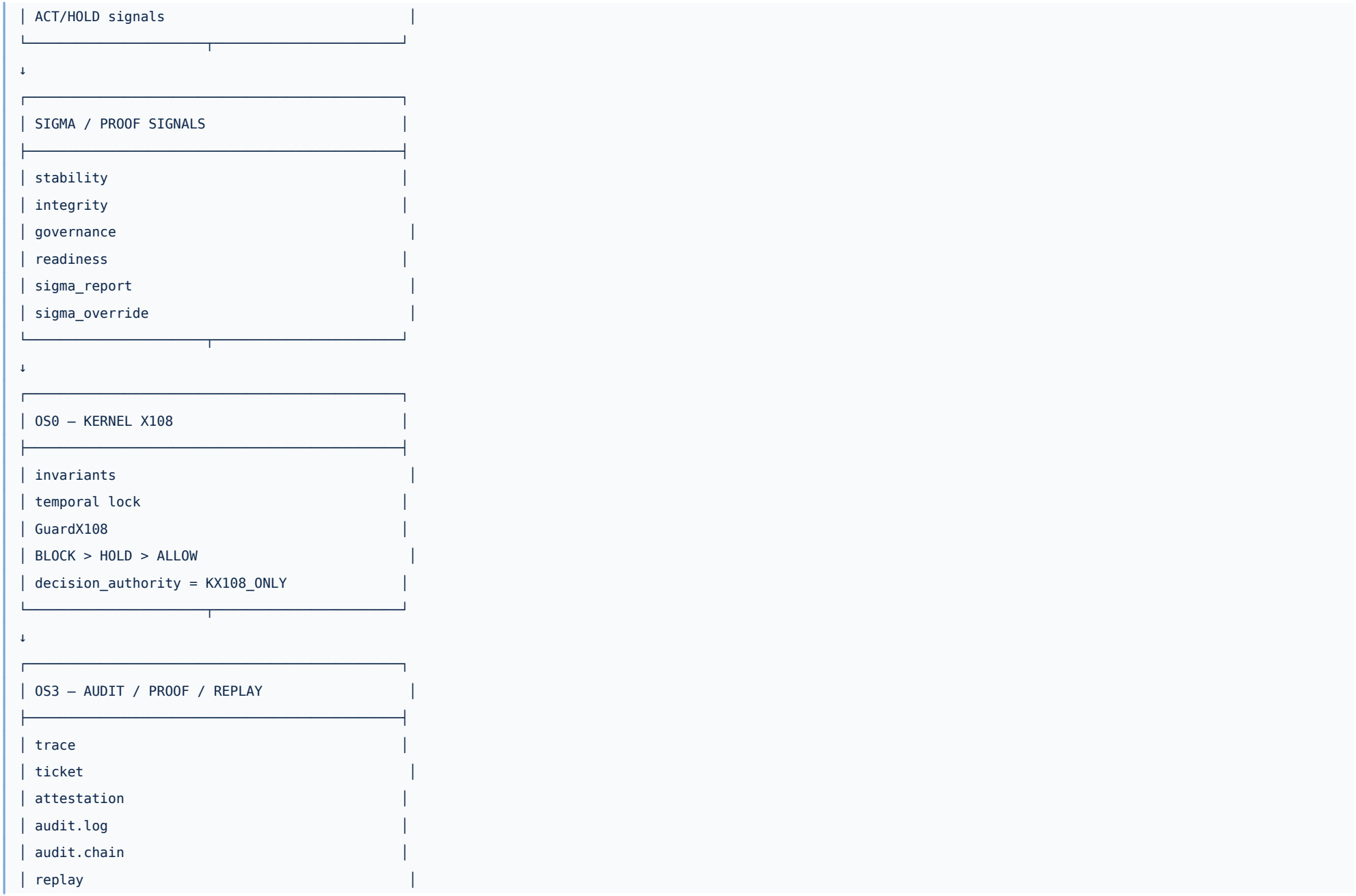
Montrer les couches qui portent le kernel, sans mélanger avec l’UI privée ou l’historique des repos.

Question traitée

Quelles couches portent la machination ?

Schéma complet

```
┌──────────────────────────────────────────────┐
│ OS4 — INTERFACE / EXPOSITION READONLY        │
├──────────────────────────────────────────────┤
│ payload                                      │
│ API                                          │
│ terminal                                     │
│ connecteur terrain                           │
│ JSON                                         │
│ logs opérateur                               │
│ projection FR si route prévue                │
└──────────────────────┬───────────────────────┘
                       ↓
┌──────────────────────────────────────────────┐
│ OS1 — STRUCTURE / PARSE / VALIDATION         │
├──────────────────────────────────────────────┤
│ parse                                        │
│ validate                                     │
│ DomainState                                  │
│ action candidate                             │
│ contract checks                              │
└──────────────────────┬───────────────────────┘
                       ↓
┌──────────────────────────────────────────────┐
│ OS2 — MÉTRIQUES / SCORES / SEUILS            │
├──────────────────────────────────────────────┤
│ confidence                                   │
│ scores métier                                │
│ thresholds                                   │
│ risk_flags                                   │
│ unknowns                                     │
│ contradictions                               │
```


ACT/HOLD signals
SIGMA / PROOF SIGNALS
stability
integrity
governance
readiness
sigma_report
sigma_override
OS0 — KERNEL X108
invariants
temporal lock
GuardX108
BLOCK > HOLD > ALLOW
decision_authority = KX108_ONLY
OS3 — AUDIT / PROOF / REPLAY
trace
ticket
attestation
audit.log
audit.chain
replay

```
| evidence_refs                                |
└──────────────────────┬──────────────────────┘
↓
┌──────────────────────────────────────────────┐
| RUNTIME / BUS                                |
├──────────────────────────────────────────────┤
| router                                       |
| registry                                     |
| messages                                     |
| CLI                                          |
| API                                          |
| localhost                                    |
| replay endpoint                              |
└──────────────────────────────────────────────┘
```

Version publique courte

OS4 reçoit / expose.

OS1 structure.

OS2 mesure.

Sigma qualifie.

OS0 tranche.

OS3 prouve.

Runtime route.

Version terrain

connecteur / terminal / API

```
→ OS4
→ OS1
→ OS2
→ Sigma
→ OS0 / X108
→ OS3
```

```
→ runtime / replay
```

Rôles à expliquer

OS4

Interface / exposition readonly.

Reçoit payloads, connecteurs, terminal, API.

Expose JSON, logs, API, terminal, projection FR si route prévue.

OS1

Structure.

Parse, valide, transforme en DomainState / action candidate.

OS2

Mesure.

Scores, seuils, risk_flags, unknowns, contradictions.

Sigma

Qualifie.

Stabilité, integrity, governance, readiness.

OS0 / Kernel X108

Tranche.

Invariants, temporal lock, GuardX108, BLOCK > HOLD > ALLOW.

OS3

Prouve.

Trace, ticket, attestation, audit, replay.

Runtime / Bus

Route.

Messages, API, CLI, registry, localhost, replay endpoint.

Message visuel

La machination n’est pas un chatbot.

C’est une chaîne OS :

recevoir,

structurer,

mesurer,

qualifier,

trancher,

prouver,

exposer.

À ne pas faire

Ne pas surcharger avec :

noms internes sensibles

alphabet complet

formules propriétaires

anciennes UI

Graphiti

Brody mémoire

Gemini

BankWorld

MINI-CHECKLIST DE VALIDATION DU LOT 6

Schéma 21 validé si

CanonicalDecisionEnvelope est clairement après X108.

x108_gate est distinct de market_verdict.

decision_id, trace_id, reason_code, severity sont visibles.

evidence_refs, attestation_ref, metrics sont visibles.

sigma_report et les confidence proof sont visibles si place disponible.

Schéma 22 validé si

Audit vient après la décision.

Hash / evidence_refs / attestation sont visibles.

Replay par trace_id est visible.

L’audit ne redécide pas.

Schéma 23 validé si

Le titre est “Exposition humaine readonly”.

Le français est optionnel / modalité d’exposition.

JSON, API, terminal, logs sont visibles.

```
readonly / emits_act=false / memory_write=false sont visibles.
decision_authority = KX108_ONLY est visible.
```

Aucune mention d’UI privée / Gemini / Brody mémoire.

Schéma 24 validé si

OS4 reçoit / expose.

OS1 structure.

OS2 mesure.

Sigma qualifie.

OS0 tranche.

OS3 prouve.

Runtime route.

Le schéma ne donne pas l’impression que l’exposition humaine redécide.

ORDRE DE FABRICATION RECOMMANDÉ

1. Schéma 21 — CanonicalDecisionEnvelope

2. Schéma 22 — Audit / hash / replay

3. Schéma 23 — Exposition humaine readonly

4. Schéma 24 — OS / runtime / bus

Pourquoi :

on part de la sortie kernel,

on montre la preuve,

on montre l’exposition humaine,

puis on replace tout dans les couches OS/runtime.

RÉSULTAT ATTENDU APRÈS LE LOT 6

Après ce lot, on doit pouvoir dire :

X108 ne produit pas une phrase.

Il produit une enveloppe canonique.

Cette enveloppe contient :

la décision,

la raison,

la gravité,

la trace,

les preuves,

les métriques.

Ensuite seulement :

l’audit archive,

le hash atteste,

le replay vérifie,

l’exposition readonly rend lisible.

Aucune de ces couches ne redécide.

Phrase verrou corrigée :

X108 tranche.

L’enveloppe structure.

L’audit prouve.

L’exposition readonly rend lisible.

LOT 7 — SOCLE FORMEL / LEAN / SYNTHÈSE FINALE

LOT 7 =

25. Lean 57 théorèmes

26. Lean → Runtime kernel

27. Synthèse finale — preuve visuelle du kernel

Objectif du lot :

Fermer le support.

Après avoir montré :

- le kernel,
- les domaines,
- les agents,
- l’agrégation,
- Sigma,
- l’enveloppe,
- l’audit,
- l’exposition readonly,

on montre maintenant :

1. ce qui prouve les lois,

2. comment ces preuves se relient au runtime,

3. comment tout se condense en une preuve visuelle finale.

Le doc de référence place bien le bloc Lean 57 comme socle formel final : Basic, TemporalKernel, TemporalBridge, Refinement, Consensus, ConsensusScratch, Merkle, CryptoAssumptions, Seal, Sensitivity, SystemModel, pour un total de 57.

SCHÉMA 25 — LEAN 57 THÉORÈMES

Rôle du schéma

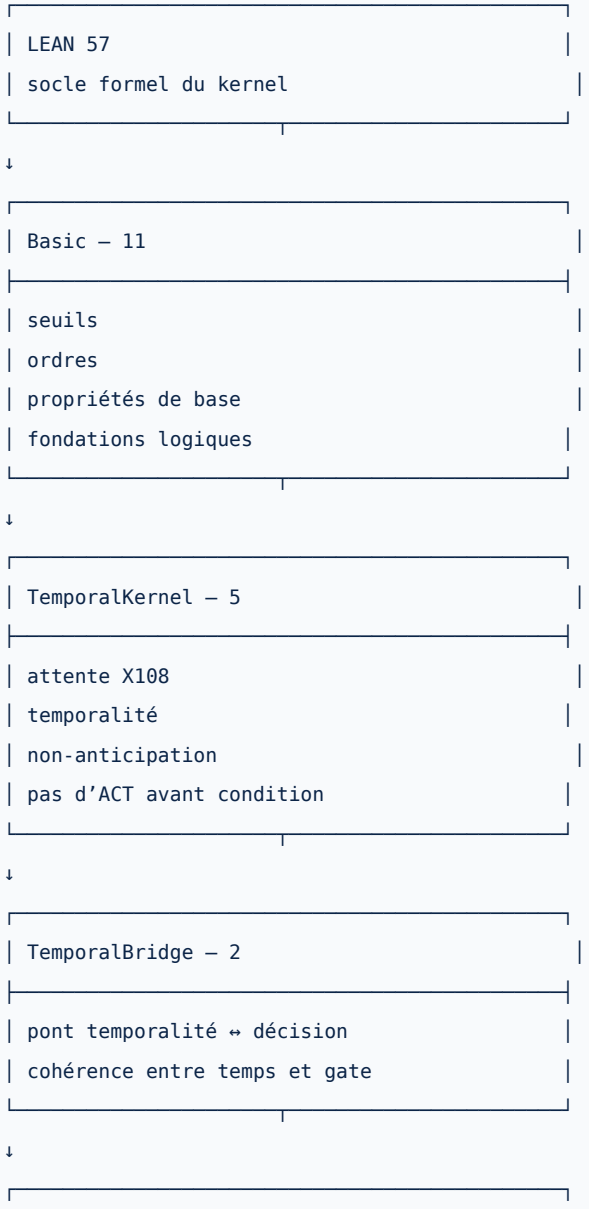
Montrer que le kernel n’est pas seulement un pipeline logiciel.

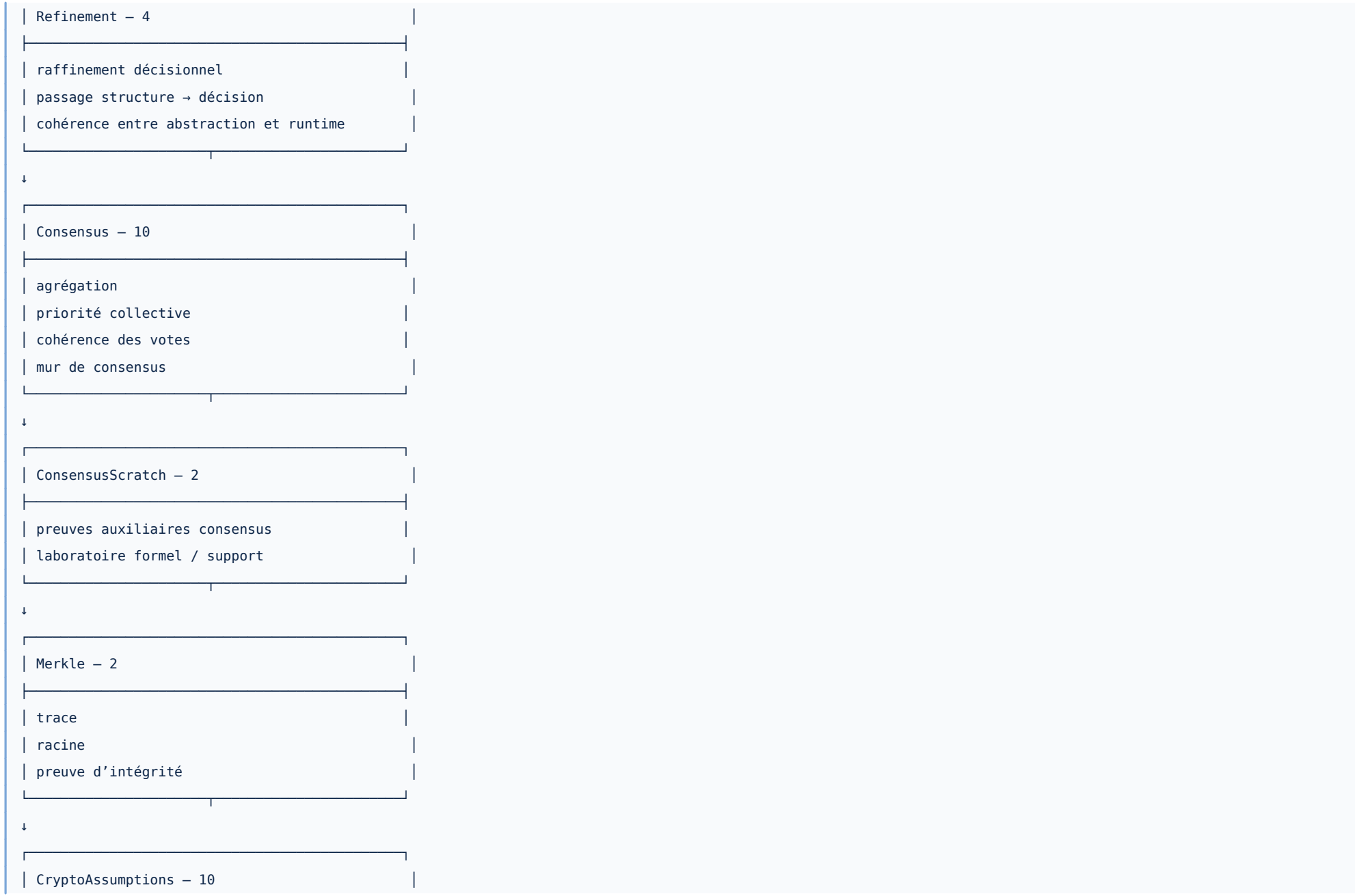
Il est adossé à un socle formel.

Question traitée

Qu’est-ce qui prouve les lois du kernel ?

Schéma complet




Refinement — 4
raffinement décisionnel
passage structure → décision
cohérence entre abstraction et runtime
Consensus — 10
agrégation
priorité collective
cohérence des votes
mur de consensus
ConsensusScratch — 2
preuves auxiliaires consensus
laboratoire formel / support
Merkle — 2
trace
racine
preuve d’intégrité
CryptoAssumptions — 10


hypothèses crypto
signature
anti-replay
intégrité externe
Seal — 1
scellement
fermeture de décision
Sensitivity — 3
sensibilité
seuils
robustesse
variation contrôlée
SystemModel — 7
modèle système
transition
auditabilité
cohérence globale

```
| TOTAL = 57                                  |
| lois formelles du kernel                    |
└─────────────────────────────────────────────┘
```

Version compacte

LEAN 57

```
├─ Basic — 11
├─ TemporalKernel — 5
├─ TemporalBridge — 2
├─ Refinement — 4
├─ Consensus — 10
├─ ConsensusScratch — 2
├─ Merkle — 2
├─ CryptoAssumptions — 10
├─ Seal — 1
├─ Sensitivity — 3
└─ SystemModel — 7
```

Lecture par rôle

Basic — 11

Rôle :

fondations minimales.

Ce que ça porte :

- seuils

- ordres

- propriétés primitives

- règles de base

TemporalKernel — 5

Rôle :

cœur temporel X108.

Ce que ça porte :

- attente
- non-anticipation
- pas d’action avant condition
- cohérence temporelle du verrou

TemporalBridge — 2

Rôle :

pont entre temporalité formelle et décision runtime.

Ce que ça porte :

- lien temps ↔ gate

```
- passage condition temporelle → décision
```

Refinement — 4

Rôle :

raffinement de la structure vers la décision.

Ce que ça porte :

- cohérence abstraction/runtime
- conservation des propriétés lors du passage vers l’exécution

Consensus — 10

Rôle :

mur de consensus.

Ce que ça porte :

- cohérence collective
- agrégation
- priorité

- stabilité des votes

ConsensusScratch — 2

Rôle :

preuves auxiliaires consensus.

Statut visuel :

support formel annexe.

Merkle — 2

Rôle :

intégrité de trace.

Ce que ça porte :

- racine
- preuve de chaînage
- vérifiabilité

CryptoAssumptions — 10

Rôle :

hypothèses crypto nécessaires à l’audit fort.

Ce que ça porte :

- signature
- anti-replay
- confiance dans les primitives externes

Seal — 1

Rôle :

scellement.

Ce que ça porte :

- fermeture d’une décision
- référence de preuve

Sensitivity — 3

Rôle :

robustesse aux variations.

Ce que ça porte :

- seuils
- sensibilité
- stabilité sous variation

SystemModel — 7

Rôle :

modèle global.

Ce que ça porte :

- transition
- auditabilité
- cohérence système

Message visuel

Lean prouve les lois.

X108 applique.

Audit vérifie.

À ne pas faire

Ne pas présenter Lean comme :

un bonus académique

Le présenter comme :

le socle formel qui ferme la crédibilité du kernel

SCHÉMA 26 — LEAN → RUNTIME KERNEL

Rôle du schéma

Relier les 57 théorèmes au fonctionnement concret du kernel.

Sans ce schéma, Lean reste une annexe isolée.

Question traitée

Comment les preuves Lean se relient au runtime X108 ?

Schéma complet



```
├──────────────────────────────────────────┤
│ CanonicalDecisionEnvelope                │
│ x108_gate                                │
│ reason_code                              │
│ severity                                 │
│ decision_id                              │
│ trace_id                                 │
└──────────────────────┬───────────────────┘
↓
┌──────────────────────────────────────────┐
│ PREUVE RUNTIME                           │
├──────────────────────────────────────────┤
│ evidence_refs                            │
│ attestation_ref                          │
│ audit.log                                │
│ audit.chain                              │
│ replay                                   │
└──────────────────────────────────────────┘
```

```
Mapping Lean → Runtime
```

Basic

```
→ seuils / ordres / règles primitives
```

TemporalKernel

```
→ temporal lock / non-anticipation / HOLD avant ACT
```

TemporalBridge

```
→ passage temps → gate
```

Refinement

```
→ structure abstraite → décision runtime
```

Consensus

```
→ AgentVote / DomainAggregate / mur collectif
```

Merkle

```
→ evidence_refs / hash / audit.chain
```

CryptoAssumptions

```
→ attestation / signature / anti-replay
```

Seal

```
→ scellement de décision
```

Sensitivity

```
→ robustesse des seuils
```

SystemModel

```
→ transition / auditabilité / cohérence globale
```

Version visuelle en colonnes

```
┌──────────────────────┬──────────────────────────────┬──────────────────────────────┐
│ MODULE LEAN          │ LOI PROUVÉE                  │ RUNTIME                      │
├──────────────────────┼──────────────────────────────┼──────────────────────────────┤
│ Basic                │ seuils / ordres              │ règles primitives X108       │
│ TemporalKernel       │ attente / non-anticipation   │ HOLD avant ACT               │
│ TemporalBridge       │ temps ↔ décision             │ temporal gate                │
│ Refinement           │ abstraction → runtime        │ pipeline décisionnel         │
│ Consensus            │ cohérence agrégation         │ AgentVote / DomainAggregate  │
│ Merkle               │ intégrité trace              │ evidence_refs / audit.chain  │
│ CryptoAssumptions    │ signature / anti-replay      │ attestation / replay         │
│ Seal                 │ scellement                   │ decision seal                │
│ Sensitivity          │ robustesse seuils            │ stabilité des gates          │
│ SystemModel          │ modèle global                │ audit / transition / replay  │
└──────────────────────┴──────────────────────────────┴──────────────────────────────┘
```

Message visuel

La preuve formelle ne remplace pas le runtime.

Elle encadre les lois que le runtime applique.

Phrase support

Lean prouve les propriétés.

Le kernel applique les règles.

L’enveloppe trace la décision.

Le replay vérifie l’exécution.

À ne pas faire

Ne pas dessiner :

```
Lean → décision directe
```

Dessiner :

```
Lean → lois → runtime X108 → envelope → audit/replay
```

Lean n’est pas le moteur d’exécution. Lean est le socle de preuve.

SCHÉMA 27 — SYNTHÈSE FINALE : PREUVE VISUELLE DU KERNEL

Rôle du schéma

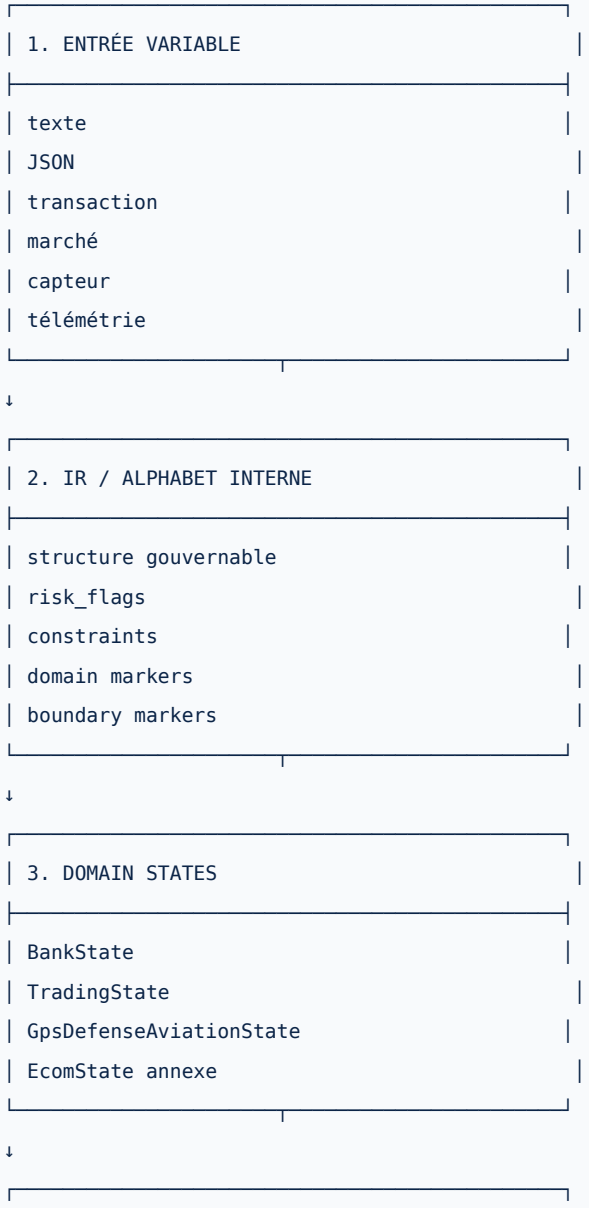
Créer le dernier schéma du support : une vue complète, lisible, finale.

Il doit permettre d’expliquer tout le système en une seule image.

Question traitée

Quelle est la preuve visuelle complète du kernel ?

Schéma complet




4. AGENTS DOMAINE
Banque : 12 agents
Trading : 17 agents
GPS : 6 agents
Ecom : annexe
5. AGENT VOTES
confidence
severity
unknowns
contradictions
risk_flags
proposed_verdict
evidence_refs
6. DOMAIN AGGREGATE
scores métier
market_verdict candidat
confidence
evidence_refs
extra_metrics
7. META-AGENTS


policy
ticket
trace
attestation
replay
proof consistency
severity
8. SIGMA / PROOF SIGNALS
stability
integrity
governance
readiness
sigma_report
confidence_integrity
confidence_governance
confidence_readiness
9. KERNEL X108 — DERNIER MUR
contradiction check
unknown check
confidence check
risk_flags check
temporal lock
BLOCK > HOLD > ALLOW


10. CANONICAL DECISION ENVELOPE
x108_gate
reason_code
severity
decision_id
trace_id
metrics
attestation_ref
11. AUDIT / HASH / REPLAY
evidence_refs
audit.log
audit.chain
replay
verification
12. EXPOSITION READONLY
JSON
API
terminal
logs
français contrôlé si route prévue
no ACT
no override

Version ultra-compacte

Entrée variable

```
→ IR / alphabet interne
→ DomainState
→ Agents
→ AgentVote
→ DomainAggregate
→ Meta-agents
→ Sigma
→ Kernel X108
→ Envelope
→ Audit / replay
→ Exposition readonly
```

Version phrase

Le domaine propose.

Les agents signalent.

L’agrégation synthétise.

Les meta-agents sécurisent.

Sigma qualifie.

X108 tranche.

L’enveloppe structure.

L’audit prouve.

L’exposition readonly rend lisible.

Message visuel final

On ne présente pas une vision générale.

On construit la preuve visuelle du kernel.

Phrase finale du support

Entrée variable.

Structure stable.

Lois formelles.

Kernel final.

Preuve rejouable.

Exposition readonly.

MINI-CHECKLIST DE VALIDATION DU LOT 7

Schéma 25 validé si

Les 57 théorèmes sont regroupés par module.

```
Le total = 57 est visible.
```

Chaque module a un rôle.

Lean est présenté comme socle formel, pas comme exécuteur runtime.

Schéma 26 validé si

Chaque grand bloc Lean est relié à une fonction runtime.

TemporalKernel est relié au temporal lock.

Consensus est relié à AgentVote / DomainAggregate.

Merkle / Crypto / Seal sont reliés à audit / hash / replay.

SystemModel est relié à cohérence globale / transition.

Schéma 27 validé si

Toute la chaîne est visible.

Le kernel est le dernier mur.

L’audit est après la décision.

L’exposition readonly ne redécide pas.

Les domaines sont présents sans voler le centre.

ORDRE DE FABRICATION RECOMMANDÉ

1. Schéma 25 — Lean 57 théorèmes

2. Schéma 26 — Lean → Runtime kernel

3. Schéma 27 — Synthèse finale

Pourquoi :

on montre d’abord le socle formel,

puis son lien avec le runtime,

puis la vue finale complète.

RÉSULTAT ATTENDU APRÈS LE LOT 7

Après ce lot, on doit pouvoir dire :

Le support est fermé.

On a montré :

- comment l’entrée devient structure,
- comment les domaines produisent des signaux,
- comment les votes sont agrégés,
- comment Sigma qualifie,
- comment X108 tranche,
- comment l’enveloppe sort,

- comment l’audit vérifie,
- comment l’exposition readonly rend lisible,
- et comment Lean relie tout à un socle formel.

Phrase verrou finale :

Lean prouve les lois.

X108 applique le dernier mur.

L’audit vérifie.

L’exposition readonly rend lisible.

PARTIE II - SRL / Session Registry Layer

Reprise complète et mise en forme visuelle de audit memoire ect(2).docx

SOURCE 2 - audit memoire ect(2).docx

Oui. C’est très intéressant — et surtout ça colle exactement à une logique Obsidia : ne pas tout mettre dans le contexte actif, mais ne rien perdre structurellement.

Tu viens de décrire une mémoire à étages avec cimetière vivant / mémoire froide réactivable.

Bloc conceptuel

1. Session brute

Tout ce qui se passe dans une session est capturé :

conversation brute

idées

frictions

bugs

pépites

décisions

noms

tests

liens

commandes

erreurs

validations

Rien n’est détruit.

2. Agent de tri contextuel

Un petit agent passe derrière et classe chaque fragment :

ACTIF

```
→ utile souvent
→ récupérable directement
```

SEMI-ACTIF

```
→ utile selon contexte
→ rappelé si sujet proche
```

FROID

```
→ pas utile maintenant
→ gardé hors contexte courant
```

FANTÔME

```
→ mis de côté
→ jamais chargé par défaut
→ consultable uniquement si recherche profonde / échec / demande explicite
```

Donc ce n’est pas “supprimé”.

C’est :

désindexé du flux principal

mais conservé dans une mémoire latérale

3. Tableau de mise de côté

Tu peux avoir un fichier/table :

memory_discard_table.jsonl

ou

context_cold_index.csv

Avec :

id: ctx_20260604_00127

status: GHOST

reason: bruit / doublon / faible utilité immédiate

source_session: session_x

symbolic_hash: "X108::friction::context_decay::memory_sidecar"

keywords:

- contexte

- mémoire
- suppression froide
- tri agent

reactivation_policy:

- explicit_user_request
- no_result_in_active_memory
- semantic_collision
- audit_needed

4. Pseudo-hachage symbolique

Très bonne pépite.

Même les éléments mis de côté gardent une empreinte compressée :

```
HASH SYMBOLIQUE ≠ hash cryptographique pur
HASH SYMBOLIQUE = signature de sens
```

Exemple :

CTXHASH:

SESSION:2026-06-04

DOMAIN:MEMORY

INTENT:CONTEXT_TRIAGE

TENSION:KEEP_WITHOUT_LOADING

GLYPH:COLD_SIDE_TABLE

BOUNDARY:X108_NO_DELETE

Ça permet de retrouver un vieux bloc même si les mots exacts changent.

Architecture simple

SESSION

```
↓
```

RAW LOG

```
↓
```

TRIAGE AGENT

```
↓

┌──────────────────────┬──────────────────────┬──────────────────────┐
│ ACTIVE MEMORY        │ COLD MEMORY          │ GHOST TABLE          │
│ chargée souvent      │ chargée si pertinent  │ jamais chargée défaut │
└──────────────────────┴──────────────────────┴──────────────────────┘

↓
```

RETRIEVAL POLICY

```
↓
```

CONTEXT PACK FINAL

Règle importante

Le moteur ne doit pas “oublier”.

Il doit réduire la charge contextuelle.

Donc la vraie loi serait :

SUPPRESSION INTERDITE

MISE EN FROID AUTORISÉE

RÉACTIVATION TRAÇABLE

Version Obsidia :

```
NO_DELETE → COLD_SHIFT → SYMBOLIC_TRACE → CONDITIONAL_RECALL
```

Nom possible du module

Très bons noms possibles :

Context Cold Sidecar

Mémoire Fantôme

Ghost Context Index

Side Memory Ledger

Obsidia Context Sieve

Tamis Contextuel

Mémoire Froide Réactivable

Le plus propre pour Obsidia :

OCS — Obsidia Context Sieve

ou

GCI — Ghost Context Index

Ce que ça donne comme utilité réelle

Ça règle un problème central :

```
trop de mémoire = bruit
pas assez de mémoire = perte de pépites
```

Donc tu crées une troisième voie :

mémoire disponible

mais non intrusive

C’est exactement compatible avec :

```
Graphiti V20 = mémoire navigable
X-108 = pas d’action sans verrou
OS3 = audit / cohérence
OS4 = interface de rappel / workbench
```

Freeze court

IDÉE VALIDABLE :

Créer une mémoire latérale froide où les fragments non prioritaires ne sont pas supprimés mais déplacés hors contexte actif, indexés par empreinte symbolique, et réactivables uniquement par demande explicite, échec de recherche active ou collision sémantique.

LOI :

Ne jamais supprimer une pépite potentielle.

Réduire sa présence, pas son existence.

C’est une vraie brique Obsidia. Pas un détail.

Oui. Là, c’est cohérent.

Tu ne proposes pas juste un “résumé de session”. Tu poses une couche mémoire opérationnelle :

SESSION UTILISATEUR

```
→ capturée
→ classée
→ découpée
→ indexée
→ refroidie si besoin
→ réactivable sans relecture totale
```

Ce que tu viens de verrouiller

Principe

Chaque session devient une fiche structurée, pas un bloc perdu dans l’historique.

session_2026_06_04_obsidia_memory_context.md

Elle contient :

- point de départ
- point d’arrêt
- décisions prises
- idées apparues
- pépites
- bugs/frictions
- fichiers évoqués
- modules touchés
- concepts créés
- éléments à reprendre
- statut : ouvert / figé / à trier / à brancher

La vraie force

Au lieu de faire :

relire toute la conversation

le système fait :

retrouver la bonne fiche

charger seulement le bon bloc

reprendre au bon point

Donc tu gagnes :

moins de bruit

moins de tokens

moins d’oubli

plus de continuité

plus de preuve

Architecture premier jet

SESSION RAW

```
↓
```

SESSION FICHE

```
↓
```

TRIAGE MÉMOIRE

```
↓
┌──────────────┬──────────────┬──────────────┬──────────────┐
│ ACTIF        │ SEMI-ACTIF   │ FROID        │ FANTÔME      │
│ à reprendre  │ utile contexte│ archive utile │ mis de côté  │
└──────────────┴──────────────┴──────────────┴──────────────┘
↓
```

INDEX JOUR / PROJET / MODULE / COUCHE / CONCEPT

```
↓
```

REPRISE CONTEXTUELLE

Nom propre possible

Pour Obsidia, je poserais ça comme :

OSM — Obsidia Session Memory

ou plus fort :

SCR — Session Continuity Registry

Le nom le plus clair :

OBSIDIA_SESSION_LEDGER

Parce que ce n’est pas juste une mémoire : c’est un registre de continuité.

Loi associée

Une session ne doit jamais mourir dans l’historique.

Elle doit être transformée en fiche de reprise.

Version moteur :

```
RAW_SESSION → SESSION_LEDGER → CONTEXT_INDEX → SELECTIVE_RECALL
```

Freeze conceptuel court

OBSIDIA_SESSION_MEMORY_V0

À la fin de chaque session, le système génère une fiche structurée contenant les décisions, pépites, points d’arrêt, modules concernés, éléments actifs, éléments froids et fragments fantômes.

À la reprise, le système ne relit pas tout l’historique. Il consulte l’index, identifie la fiche pertinente, recharge uniquement les blocs nécessaires et reprend depuis le dernier point stable.

Règle :

Ne pas tout garder dans le contexte actif.

Ne rien perdre.

Tout classer.

Tout rendre réactivable.

Oui : pour un premier jet, c’est largement assez solide. La suite logique, ce n’est pas d’améliorer l’idée. C’est de lui donner un format de fiche canonique.

Oui. Correction nette : ce n’est pas une mémoire Obsidia interne.

Obsidia a déjà ses principes mémoire, couches, lois, rangement, multi-dimensions. Là, on parle d’une couche sessionnelle pour les utilisateurs / opérateurs / projets, branchée autour du système, sans modifier le noyau.

Nom plus juste

Le bon axe n’est pas “Obsidia Session Memory” mais :

SESSION MEMORY LAYER

ou en version française :

MSU — Mémoire de Session Utilisateur

Mais comme tu dis “tous les types de session”, le plus propre serait plus large :

SRL — Session Registry Layer

Traduction structurelle :

Couche Registre de Sessions

C’est mieux, parce que ça peut gérer :

- session utilisateur
- session dev
- session audit
- session recherche
- session business
- session pitch
- session debug
- session documentation
- session mémoire

- session freeze

Donc le nom canonique possible :

SRL — SESSION REGISTRY LAYER

Position dans l’architecture

À ne pas mettre comme nouvelle couche moteur Obsidia.

Pas :

```
OS0 → OS1 → OS2 → OS3 → OS4 → OS5
```

Mais plutôt :

OBSIDIA CORE

↕

SESSION REGISTRY LAYER

↕

UTILISATEURS / SESSIONS / WORKBENCH / AGENTS

Donc :

```
SRL = couche périphérique de continuité sessionnelle
```

Elle ne décide pas. Elle ne modifie pas le noyau. Elle ne touche pas aux lois moteur. Elle range, indexe, classe, retrouve.

Définition propre

SRL — Session Registry Layer

Couche périphérique destinée à transformer chaque session utilisateur ou opérateur en fiche structurée, indexée, classée et réactivable, afin que le système puisse reprendre un contexte sans relire toute l’historique brut.

Rôle exact

RAW SESSION

```
→ SESSION CARD
→ SESSION INDEX
```

```
→ CONTINUITY POINTER
→ ACTIVE / SEMI-ACTIVE / COLD / GHOST STORAGE
→ CONTEXT RELOAD PACK
```

Ce que ça règle

Problème :

les sessions longues deviennent illisibles, coûteuses, bruitées, difficiles à reprendre.

Solution :

chaque session devient un objet mémoire classé, avec un point d’arrêt, des liens, des décisions, des pépites et des fragments mis de côté mais récupérables.

Types de fiches

session_user_2026_06_04.md

session_dev_2026_06_04.md

session_audit_2026_06_04.md

session_research_2026_06_04.md

session_business_2026_06_04.md

session_debug_2026_06_04.md

Contenu minimal d’une fiche SRL

session_id:

date:

session_type:

project:

entry_context:

exit_point:

active_blocks:

decisions:

validated_items:

open_threads:

frictions:

peptides_or_pepites:

files_links:

commands:

next_recall_targets:

cold_fragments:

ghost_fragments:

reactivation_rules:

status:

Loi associée

Une session ne doit pas être relue.

Elle doit être retrouvée.

Version moteur :

```
READ_ALL_CONTEXT = interdit par défaut
LOAD_RELEVANT_SESSION_CARD = autorisé
DEEP_RECALL = seulement si manque, demande explicite ou audit
```

Freeze V0

SRL — SESSION REGISTRY LAYER V0

Couche périphérique de mémoire sessionnelle destinée aux utilisateurs, opérateurs et projets.

Elle ne modifie pas Obsidia Core.

Elle ne décide pas.

Elle ne remplace pas la mémoire interne Obsidia.

Elle transforme chaque session en fiche structurée, indexée par type, projet, date, point d’arrêt, décisions, pépites, fragments actifs, froids et fantômes.

À la reprise, le système recharge la fiche pertinente au lieu de relire toute la conversation brute.

But :

continuité sans surcharge.

mémoire sans bruit.

rappel sans relecture totale.

Donc oui : là, le bon cadrage est SRL — Session Registry Layer. Pas une nouvelle loi centrale Obsidia. Une couche utilisateur/session périphérique, compatible avec le futur moteur, mais indépendante pour commencer.SRL — SESSION REGISTRY LAYER V0

Couche périphérique de mémoire sessionnelle utilisateur

STATUT :

Périphérique.

Non moteur.

Non décisionnel.

Non connecté à la mémoire obsidienne pour l'instant.

Branchable plus tard.

1. Positionnement

La SRL n'est pas une nouvelle couche interne d'Obsidia.

Elle sert à gérer les sessions utilisateur / opérateur / projet, pour éviter que chaque reprise oblige à relire tout l'historique.

OBSIDIA CORE

```
|
|  plus tard seulement
↓
```

SRL — SESSION REGISTRY LAYER

```
↓
```

UTILISATEURS / SESSIONS / CONVERSATIONS / WORKBENCH

Pour l'instant :

```
SRL = mémoire périphérique de continuité
```

Pas :

```
SRL = mémoire centrale Obsidia
```

2. Fonction principale

Chaque session devient une fiche structurée.

SESSION BRUTE

```
→ FICHE SESSION
→ INDEX
→ CLASSEMENT
→ REPRISE CONTEXTUELLE
```

But :

ne pas relire toute la conversation

retrouver la bonne fiche

reprendre au bon point

charger seulement les blocs utiles

3. Types de sessions gérées

La couche doit être générique.

session utilisateur

session dev

session audit

session recherche

session business

session pitch

session debug

session documentation

session freeze

session stratégie

session mémoire

session test

Donc le système ne classe pas seulement “une conversation”. Il classe des situations de travail.

4. Flux complet

1. Session en cours

↓

2. Capture brute

↓

3. Génération fiche session

↓

4. Extraction des éléments utiles

↓

5. Classement par statut mémoire

↓

6. Indexation par projet / date / type / module / sujet

↓

7. Point d’arrêt enregistré

↓

8. Fiche prête pour reprise future

5. Les 4 zones mémoire SRL

A. ACTIF

Ce qui doit être repris directement.

décision validée

point d’arrêt

module en cours

bloc à continuer

action prochaine

B. SEMI-ACTIF

Ce qui peut redevenir utile selon le contexte.

idée secondaire

hypothèse

piste non prioritaire

fragment lié à un projet

C. FROID

Ce qui est conservé mais non chargé par défaut.

détail utile plus tard

ancienne piste

contexte secondaire

discussion non urgente

D. FANTÔME

Ce qui est mis de côté sans être supprimé.

bruit possible

doublon

fragment faible

idée incertaine

élément non prioritaire

Règle :

```
FANTÔME ≠ supprimé
FANTÔME = hors contexte actif, mais retrouvable
```

6. Tableau latéral des éléments mis de côté

La SRL garde un registre séparé :

ghost_registry.jsonl

cold_index.csv

session_discard_table.json

Contenu possible :

fragment_id:

session_id:

date:

status: GHOST

reason:

- doublon
- bruit
- faible priorité
- hors sujet temporaire

symbolic_hash:

keywords:

reactivation_policy:

- demande explicite
- absence de résultat en mémoire active
- audit
- collision sémantique

Donc même ce qui est “écarté” reste traçable.

7. Pseudo-hachage symbolique

Même les fragments froids ou fantômes gardent une empreinte.

Pas forcément un hash cryptographique strict.

Plutôt une signature de sens :

SESSION:2026-06-04

TYPE:MEMORY_LAYER

DOMAIN:USER_SESSION

INTENT:CONTEXT_CONTINUITY

STATUS:COLD

GLYPH:NO_DELETE_SIDE_MEMORY

Utilité :

retrouver sans avoir les mots exacts

reconnecter une idée ancienne

réactiver une pépite oubliée

auditer ce qui a été mis de côté

8. Format canonique d’une fiche session

session_id:

date:

session_type:

project:

main_topic:

entry_context:

previous_session_ref:

starting_point:

user_intent:

exit_point:

last_stable_point:

next_action:

unresolved_threads:

active_blocks:

- id:

title:

content:

reason:

decisions:

- decision:

status:

impact:

validated_items:

- item:

validation_level:

open_threads:

- thread:

priority:

linked_project:

frictions:

- friction:

meaning:

status:

pepites:

- pepite:

destination:

priority:

files_links:

- name:

path_or_url:

role:

commands_or_actions:

- action:

status:

cold_fragments:

- fragment_id:

reason:

symbolic_hash:

ghost_fragments:

- fragment_id:

reason:

symbolic_hash:

reactivation_rules:

- explicit_user_request
- no_result_in_active_memory
- semantic_collision
- audit_needed

status:

value: OPEN / FROZEN / TO_SORT / TO_BRANCH / ARCHIVED

9. Indexation

Chaque fiche doit pouvoir être retrouvée par plusieurs entrées.

par date

par projet

par type de session

par module

par concept

par statut

par point d’arrêt

par pépite

par décision

par fichier

Exemples :

2026-06-04 / mémoire session / SRL / périphérie / V0

2026-06-04 / Obsidia / utilisateur / contexte / reprise

2026-06-04 / ghost memory / cold storage / no delete

10. Reprise de session

Quand une nouvelle session démarre :

1. Identifier le sujet actuel

2. Chercher les fiches liées

3. Charger seulement les fiches pertinentes

4. Reprendre depuis exit_point

5. Récupérer active_blocks

6. Ignorer cold/ghost sauf besoin

Règle stricte :

```
READ_ALL_HISTORY = interdit par défaut
LOAD_SESSION_CARD = autorisé
DEEP_RECALL = conditionnel
```

11. Règles de réactivation

Un fragment froid ou fantôme peut revenir seulement si :

l’utilisateur le demande

la mémoire active ne suffit pas

un audit exige de vérifier

une collision sémantique apparaît

un projet lié revient

une pépite ancienne redevient utile

Donc le système ne fouille pas partout sans raison.

12. Rôle du petit agent de tri

Nom fonctionnel :

Session Triage Agent

Rôle :

classer

ranger

marquer

indexer

mettre en froid

mettre en fantôme

préparer la fiche de reprise

Il ne décide pas. Il ne supprime pas. Il ne modifie pas le noyau. Il prépare la continuité.

13. Limites V0

Pour le premier jet, on ne branche pas encore à :

mémoire obsidienne centrale

Graphiti profond

X-108 décisionnel

kernel

runtime moteur

lois internes complètes

On reste sur :

fiche session

index

classement

reprise

mémoire froide

mémoire fantôme

14. Branchage futur

Plus tard seulement :

SRL

> ↓

Mémoire Obsidienne

> ↓

Graphiti / Atlas / Workbench

> ↓

OS3 audit

> ↓

OS4 interface

Mais le branchage futur devra respecter :

SRL ne décide pas

SRL ne modifie pas le noyau

SRL fournit du contexte

SRL reste traçable

SRL peut être auditée

15. Lois V0

LOI 1 — Une session ne doit pas mourir dans l’historique.

Elle doit devenir une fiche de reprise.

LOI 2 — Ne pas tout charger.

Charger seulement ce qui sert au contexte présent.

LOI 3 — Ne pas supprimer.

Déplacer en froid ou en fantôme.

LOI 4 — Tout fragment écarté doit garder une trace.

LOI 5 — La reprise doit partir du dernier point stable.

LOI 6 — La mémoire sessionnelle est périphérique tant qu’elle n’est pas branchée au noyau.

LOI 7 — Le rappel profond est conditionnel, jamais automatique par défaut.

16. Architecture V0 figée

SRL — SESSION REGISTRY LAYER V0

[SESSION RAW]

↓

[SESSION CARD BUILDER]

↓

[TRIAGE AGENT]

↓

[ACTIVE / SEMI-ACTIVE / COLD / GHOST]

↓

[SESSION INDEX]

↓

[CONTINUITY POINTER]

↓

[CONTEXT RELOAD PACK]

17. Freeze propre

SRL — SESSION REGISTRY LAYER V0

Couche périphérique de mémoire sessionnelle destinée à transformer chaque session utilisateur, opérateur ou projet en fiche structurée, indexée, classée et réactivable.

Elle permet au système de reprendre un travail sans relire tout l’historique brut.

Elle classe les fragments en actif, semi-actif, froid et fantôme.

Elle ne supprime pas réellement les fragments faibles : elle les met de côté dans un registre latéral, avec empreinte symbolique et règles de réactivation.

Elle ne modifie pas Obsidia Core.

Elle ne décide pas.

Elle ne touche pas au noyau.

Elle pourra être branchée plus tard à la mémoire obsidienne, mais reste pour l’instant une couche périphérique autonome.

Objectif V0 :

continuité sans surcharge,

mémoire sans bruit,

reprise sans relecture totale,

classement sans perte.

Oui. Après inspection GitHub : le bon branchement SRL n’est pas dans le noyau X108/proof-core. Il faut le brancher en périphérie côté ObsidiaShell, puis seulement plus tard exposer des “context packets” vers Obsidia Core.

GitHub montre bien trois zones :

Eaubin08/obsidiashell existe comme repo séparé.

Eaubin08/obsidia-engine-proof-core existe comme noyau de preuve public.

Eaubin08/obsidia-x108-proofs existe séparément aussi, donc à protéger.

Verdict de branchement

La SRL doit être :

ObsidiaShell / périphérie

PAS Obsidia Core

PAS X108 proofs

PAS kernel

Raison : ObsidiaShell est déjà défini comme shell unifié de mémoire, d’indexation, de cartographie et de recherche pour le corpus Obsidia. Il sert à ingérer, classer, indexer, cartographier, interroger et exporter le matériau de travail.

```
Donc SRL = nouvelle brique ObsidiaShell.
```

Architecture de branchement SRL

UTILISATEUR / SESSION

```
↓
```

SRL_CAPTURE

```
↓
```

SESSION_CARD_BUILDER

```
↓
```

SESSION_TRIAGE_AGENT

```
↓
```

ACTIVE / SEMI_ACTIVE / COLD / GHOST

```
↓
```

SESSION_INDEX

```
↓
```

CONTEXT_RELOAD_PACK

```
↓
```

ObsidiaShell Workbench / Search / Export

```
↓
```

plus tard seulement :

Graphiti / mémoire obsidienne / OS3 audit

Où brancher dans GitHub

Repo cible V0

Eaubin08/obsidiashell

Parce que le README dit explicitement qu’ObsidiaShell n’est pas le noyau canonique, mais une application locale opératoire pour centraliser, ranger, rendre requêtable, explorer et exporter le corpus.

Repo à ne pas modifier maintenant

Eaubin08/obsidia-engine-proof-core

Eaubin08/obsidia-x108-proofs

Le proof-core est présenté comme moteur décisionnel déterministe, audité cryptographiquement, avec tests, preuves et artefacts vérifiables. Donc on ne met pas une couche session utilisateur dedans en V0.

Structure fichiers proposée

Dans obsidiashell :

srl/

__init__.py

models.py

capture.py

card_builder.py

triage.py

indexer.py

recall.py

metrics.py

storage.py

schemas/

session_card.schema.json

ghost_fragment.schema.json

tests/

test_session_card_builder.py

test_triage_policy.py

test_recall_policy.py

test_metrics.py

data/

srl/

sessions/

indexes/

cold/

ghost/

reports/

docs/

```
SRL_SESSION_REGISTRY_LAYER_V0.md
```

Puis routes dans le gateway :

POST /srl/session/capture

POST /srl/session/finalize

GET /srl/session/{session_id}

```
GET  /srl/search?q=
GET  /srl/recall?q=
```

GET /srl/metrics

GET /srl/report/{session_id}

Le agent_bridge.py GitHub actuel est encore minimal : il expose surtout /health et vérifie Graphiti/FastGPT/Danswer. Donc SRL peut être ajouté proprement comme bloc de routes sans casser l'existant.

Modèle de fiche SRL

session_id:

date:

session_type:

project:

main_topic:

entry_context:

previous_session_ref:

starting_point:

user_intent:

exit_point:

last_stable_point:

next_action:

unresolved_threads:

active_blocks:

- id:

title:

content:

reason:

priority:

semi_active_blocks:

- id:

title:

content:

trigger_condition:

cold_fragments:

- fragment_id:

reason:

symbolic_hash:

recall_policy:

ghost_fragments:

- fragment_id:

reason:

symbolic_hash:

recall_policy:

decisions:

- decision:

status:

impact:

evidence_ref:

pepites:

- title:

content:

destination:

priority:

frictions:

- content:

meaning:

status:

files_links:

- name:

path_or_url:

role:

metrics:

capture:

triage:

recall:

continuity:

noise:

integrity:

status:

value: OPEN / FROZEN / TO_SORT / TO_BRANCH / ARCHIVED

Métriques nécessaires V0

1. Métriques de capture

raw_token_count

captured_fragment_count

session_duration_estimate

topic_count

project_count

decision_count

pepite_count

friction_count

file_link_count

But : savoir ce que la session contient réellement.

2. Métriques de tri

active_count

semi_active_count

cold_count

ghost_count

active_ratio

cold_ratio

ghost_ratio

triage_confidence

classification_entropy

But : vérifier si l'agent range proprement ou s'il met tout en actif/froid n'importe comment.

3. Métriques de continuité

entry_point_found: true/false

```
exit_point_quality: 0.0 → 1.0
next_action_clarity: 0.0 → 1.0
resume_readiness: 0.0 → 1.0
```

open_threads_count

unresolved_threads_count

But : savoir si la session est vraiment reprenable demain sans relire tout.

4. Métriques de rappel

recall_precision

recall_relevance

loaded_card_count

loaded_fragment_count

deep_recall_used: true/false

ghost_reactivation_count

cold_reactivation_count

But : vérifier que SRL recharge les bons blocs au bon moment.

5. Métriques de bruit

noise_ratio

duplicate_ratio

discard_candidate_count

ghost_false_positive_rate

active_overload_score

context_compression_ratio

But : éviter que la mémoire active devienne une poubelle.

6. Métriques d’intégrité

À aligner avec ton style proof-core.

Le core utilise déjà des enveloppes avec decision_id, trace_id, ticket_id, attestation_ref, evidence_refs et metrics.

Pour SRL, équivalent périphérique :

session_hash

card_hash

fragment_hash

symbolic_hash

source_session_id

created_at

updated_at

evidence_refs

trace_id

But : chaque fiche doit être traçable, rejouable, vérifiable.

Métriques SRL spécifiques

Je poserais ces métriques canon :

SRL-CQ — Context Quality

Qualité du contexte repris.

SRL-RR — Recall Relevance

Pertinence du rappel.

SRL-NR — Noise Reduction

Réduction du bruit par rapport à la session brute.

SRL-CR — Compression Ratio

```
Compression utile : brut → fiche.
```

SRL-GR — Ghost Ratio

Part mise de côté sans suppression.

SRL-ER — Exit Readiness

Qualité du point d’arrêt.

SRL-DR — Decision Retention

Taux de décisions conservées correctement.

SRL-PR — Pepite Retention

Taux de pépites conservées.

SRL-FR — Friction Retention

Taux de frictions conservées.

SRL-AR — Auditability Rate

Part des fragments traçables avec hash/ref/source.

Règles de sécurité SRL

SRL ne décide pas.

SRL ne modifie pas le kernel.

SRL ne pousse rien vers X108 sans export volontaire.

SRL ne supprime pas : elle déplace en cold/ghost.

SRL ne charge pas ghost par défaut.

SRL produit des context packets, pas des décisions.

C’est aligné avec ObsidiaShell : shell documentaire, ingestion, indexation, recherche, export — pas noyau souverain.

Tests nécessaires

test_session_card_created

```
→ une session brute produit une fiche non vide
```

test_exit_point_required

```
→ impossible de finaliser sans point d’arrêt
```

test_no_delete_policy

```
→ aucun fragment n’est supprimé, seulement classé
```

test_ghost_not_loaded_by_default

```
→ les fragments ghost ne reviennent pas au recall normal
```

test_deep_recall_can_load_ghost

```
→ ghost revient uniquement si demande explicite / échec / audit
```

test_symbolic_hash_stable

```
→ même fragment logique = même signature symbolique
```

test_context_reload_minimal

```
→ recall ne recharge pas toute l'historique
```

test_metrics_complete

```
→ chaque fiche contient ses métriques SRL
```

Plan de branchement V0

Palier 1 — Documentation freeze

Créer :

docs/SRL_SESSION_REGISTRY_LAYER_V0.md

Contenu : définition, rôle, limites, métriques, règles, schéma.

Palier 2 — Modèles

Créer :

srl/models.py

srl/schemas/session_card.schema.json

Palier 3 — Builder

Créer :

srl/card_builder.py

Entrée :

raw_session_text

metadata

Sortie :

session_card.json

Palier 4 — Triage

Créer :

srl/triage.py

Classement :

ACTIVE

SEMI_ACTIVE

COLD

GHOST

Palier 5 — Index

Créer :

srl/indexer.py

Index par :

date

projet

type

module

concept

point d’arrêt

décision

pépite

Palier 6 — Recall

Créer :

srl/recall.py

Sortie :

context_reload_pack.json

Palier 7 — API ObsidiaShell

Ajouter routes dans agent_bridge.py :

/srl/session/finalize

/srl/search

/srl/recall

/srl/metrics

Palier 8 — Workbench

Ajouter onglet :

Session Registry

Avec :

Sessions

Active

Cold

Ghost

Recall Pack

Metrics

Architecture finale V0

Eaubin08/obsidiashell

```
│
├── srl/
│   ├── models.py
│   ├── capture.py
│   ├── card_builder.py
│   ├── triage.py
│   ├── indexer.py
│   ├── recall.py
│   ├── metrics.py
│   └── storage.py
│
├── data/srl/
│   ├── sessions/
│   ├── indexes/
```

```
│  ├── cold/
│  ├── ghost/
│  └── reports/
│
├── docs/
│  └── SRL_SESSION_REGISTRY_LAYER_V0.md
│
└── obsidia_core/
└── agent_bridge.py
```

Freeze direction

```
SRL_V0 = périphérie ObsidiaShell
SRL_V0 ≠ mémoire obsidienne centrale
SRL_V0 ≠ X108
SRL_V0 ≠ décision
SRL_V0 = continuité sessionnelle, tri, rappel, audit léger
```

Le bon mouvement : nouvelle branche dédiée dans obsidiashell, pas dans proof-core.

Nom de branche :

feature/srl-session-registry-layer-v0

Commit freeze cible :

SRL_SESSION_REGISTRY_LAYER_V0_READONLY_PERIPHERAL

Oui. Rectification nette :

```
AUDIT CIBLE = Eaubin08/obsidia-x108-proofs UNIQUEMENT
```

PAS obsidiashell

PAS obsidia-engine-proof-core

PAS autre repo

Verdict GitHub

Dans obsidia-x108-proofs, la SRL n’est pas à créer de zéro. Elle existe déjà en germe dans la zone :

periphery/brody_memory_readonly/

Le repo indique que brody_memory_readonly contient déjà des memory readonly packets, du Graphiti bridge material, un session ledger, des artefacts de boucle opérateur, et que la décision reste KX108_ONLY.

Donc ton concept SRL doit devenir :

```
SRL = formalisation propre / audit / extension V0
```

de la couche session memory déjà présente

dans obsidia-x108-proofs.

Pas un nouveau système externe.

Branchement correct dans obsidia-x108-proofs

Zone cible

periphery/brody_memory_readonly/

Sous-zone existante déjà proche de SRL

periphery/brody_memory_readonly/session_presave_buffer_readonly/

periphery/brody_memory_readonly/project_intake_capture_buffer_readonly/

apps/obsidia_api/brody_session_memory_adapter.py

apps/obsidia_api/brody_session_memory_runtime.py

docs/freeze/BRODY_FOUNDATION_B_SESSION_MEMORY_FOLLOWUP_FREEZE.md

Le module session_presave_buffer_readonly contient déjà une liste de pointeurs comme :

CURRENT_BRODY_SESSION_MEMORY_LEDGER_READONLY_V2_VALIDATE.txt

CURRENT_BRODY_AUTO_TRIAGE_MEMORY_INTAKE_READONLY_V1_VALIDATE.txt

CURRENT_BRODY_CANDIDATE_EXPORT_FOR_GRAPHITI_READONLY_V1_VALIDATE.txt

CURRENT_BRODY_MEMORY_REPLAY_QUERY_REGRESSION_READONLY_V1_VALIDATE.txt

```
Donc la chaîne existe déjà partiellement : ledger → triage → export candidat → replay.
```

Ce qu’on doit auditer maintenant

1. Est-ce que la mémoire session actuelle couvre SRL ?

Actuellement, le runtime session memory fait déjà :

remember_session_turn()

resolve_session_followup()

build_session_memory_snapshot()

> Mais il reste limité : remember_session_turn() prépare seulement une candidate de tour, sans écriture réelle, avec memory_write=false, graphiti_write=false, write_blocked=true.

Donc SRL V0 doit auditer :

> session raw → session turn candidate → presave buffer → ledger → follow-up resolver

Puis vérifier ce qui manque :

ACTIVE / SEMI_ACTIVE / COLD / GHOST

session card complète

symbolic hash

exit point

context reload pack

deep recall conditionnel

2. Ce qui est déjà validé

Le freeze Foundation B dit que la mémoire session / follow-up est FOUNDATION_B_READY, que le ledger V2 existe, que le resolver fonctionne sur mots-clés explicites, mais que c'est encore partiel et sans vector similarity.

Composants déjà présents :

session_ledger: true

presave_buffer: variable

followup_resolver: PARTIAL

last_turn_context: extrait

conversation_topic: extrait

Donc audit à faire :

> FOUNDATION_B_READY ≠ SRL complète
>
> FOUNDATION_B_READY = base existante pour SRL V0

Métriques nécessaires pour l’audit SRL dans X108 Proof

A. Métriques de ledger

SRL_LEDGER_FOUND

SRL_LEDGER_RECORD_COUNT

SRL_JSONL_TRACE_VALID

SRL_SESSION_ID_PRESENT

SRL_PREVIOUS_EVENT_HASH_PRESENT

SRL_HASH_CHAIN_VALID

SRL_LAST_TURN_CONTEXT_FOUND

Ces métriques correspondent déjà à ce que l’adapter vérifie : session_ledger_found, jsonl_trace, hash_chain, session_id, previous_event_hash, record_count, last_turn_context.

B. Métriques de follow-up

SRL_FOLLOWUP_SUPPORTED

SRL_FOLLOWUP_REQUESTED

SRL_FOLLOWUP_RESOLVED

SRL_TOPIC_EXTRACTED

SRL_LAST_CONTEXT_EXTRACTED

SRL_USED_BY_API_NOW

Le repo suit déjà followup_supported, followup_requested, followup_resolved, conversation_topic, used_by_api_now.

C. Métriques de presave buffer

SRL_PRESAVE_POINTER_COUNT

SRL_POINTER_HASH_VALID

SRL_REFERENCED_PATH_COUNT

SRL_EVENT_HASH_CHAIN_VALID

SRL_MANUAL_VALIDATION_REQUIRED

Le presave buffer génère déjà pointer_file_count, latest_event_hash, records JSON/JSONL, et impose validation manuelle.

D. Métriques de tri SRL à ajouter

Celles-là ne sont pas encore assez formalisées :

SRL_ACTIVE_COUNT

SRL_SEMI_ACTIVE_COUNT

SRL_COLD_COUNT

SRL_GHOST_COUNT

SRL_ACTIVE_RATIO

SRL_COLD_RATIO

SRL_GHOST_RATIO

SRL_TRIAGE_CONFIDENCE

SRL_CONTEXT_OVERLOAD_SCORE

But : prouver que le système ne remet pas tout dans le contexte actif.

E. Métriques de reprise

SRL_EXIT_POINT_FOUND

SRL_EXIT_POINT_QUALITY

SRL_RELOAD_PACK_CREATED

SRL_RELOAD_PACK_FRAGMENT_COUNT

SRL_RELOAD_PACK_MINIMALITY

SRL_DEEP_RECALL_USED

SRL_GHOST_REACTIVATION_COUNT

But : prouver que la session peut reprendre sans relire tout l’historique.

F. Métriques de non-souveraineté

Obligatoires dans X108 Proof :

```
readonly = true
memory_write = false
graphiti_write = false
neo4j_write = false
memory_decision = false
allowed_to_decide = false
emits_act = false
emits_verdict = false
kernel_mutation = false
x108_runtime_binding = false
x108_merge = false
decision_authority = KX108_ONLY
```

Ces contraintes sont déjà présentes dans les modules session/presave et dans le freeze.

Plan d’audit SRL dans obsidia-x108-proofs

Phase 1 — Audit existant

Créer un audit doc :

docs/runtime/OBSIDIA_SRL_SESSION_REGISTRY_LAYER_AUDIT_V0.md

Il doit vérifier :

- composants session existants
- fichiers ledger existants
- presave buffer
- follow-up resolver
- hash chain

- boundary KX108_ONLY
- absence d’écriture mémoire
- absence Graphiti write
- absence X108 merge

Phase 2 — Matrice SRL

Créer :

docs/runtime/OBSIDIA_SRL_COMPONENT_MATRIX_V0.json

Avec :

{

"session_presave_buffer": "present",

"session_ledger": "present",

"followup_resolver": "partial",

"active_semi_cold_ghost": "missing_or_partial",

"symbolic_hash": "missing_or_partial",

"context_reload_pack": "missing_or_partial",

"deep_recall_policy": "missing_or_partial",

"decision_authority": "KX108_ONLY"

}

Phase 3 — Script d’audit readonly

Créer :

scripts/f_srl_session_registry_layer_audit_v0.py

Le script ne doit rien modifier. Il doit seulement lire et produire :

docs/runtime/OBSIDIA_SRL_SESSION_REGISTRY_LAYER_AUDIT_V0.json

docs/runtime/OBSIDIA_SRL_SESSION_REGISTRY_LAYER_AUDIT_V0.md

Phase 4 — Extension seulement si audit OK

Après audit, ajouter éventuellement :

periphery/brody_memory_readonly/srl_session_registry_readonly/

Avec :

srl_session_card_builder_readonly_v0.py

srl_triage_metrics_readonly_v0.py

srl_context_reload_pack_readonly_v0.py

Mais seulement en readonly.

Ce qu’il ne faut pas faire

NE PAS toucher sigma/

NE PAS toucher proofs/lean/

NE PAS toucher formal/tla/

NE PAS toucher merkle_seal.json

NE PAS brancher Graphiti en écriture

NE PAS faire x108_merge

```
NE PAS faire memory_intake=true
NE PAS faire allowed_to_decide=true
```

Le repo a déjà une politique de chemins protégés et de quarantaine contre MERGE, CREATE, DELETE, execute_write, git commit, git push, etc.

Conclusion nette

SRL ne se branche pas ailleurs.

SRL s’audite dans obsidia-x108-proofs.

SRL doit s’appuyer sur Foundation B / Brody Session Memory existant.

```
SRL V0 = audit + formalisation + métriques + gaps.
SRL V1 = extension readonly si l’audit confirme les manques.
```

Nom freeze propre :

OBSIDIA_SRL_SESSION_REGISTRY_LAYER_READONLY_AUDIT_V0

Statut attendu :

READONLY

PERIPHERY

SESSION_MEMORY

NO_WRITE

NO_GRAPHITI_WRITE

NO_X108_MERGE

KX108_ONLY

Audit SRL profond — Eaubin08/obsidia-x108-proofs

Verdict court

SRL n’est pas à créer de zéro.

La base existe déjà dans obsidia-x108-proofs.

Mais elle n’est pas encore proprement figée comme :

SRL — SESSION REGISTRY LAYER.

Elle existe aujourd’hui sous plusieurs modules Brody Memory Readonly :

ledger, presave, triage, review gate, replay, post-human triage, Graphiti candidate export.

Statut réel :

BASE SRL PRÉSENTE

COUCHE SRL NON CANONISÉE

FRONTIÈRES READONLY À RÉALIGNER

MÉTRIQUES SRL À AJOUTER

Audit effectué en lecture source GitHub. Je n’ai pas exécuté les scripts localement.

1. Surface officielle du repo

Le repo obsidia-x108-proofs est bien le périmètre public P1 de preuve/vérification X108, avec Lean, TLA+, verifiers Python, Sigma minimal, QA RFC3161/TLC/Sigma et runner public. Il n’est pas le moteur propriétaire complet.

La frontière actuelle est claire :

no kernel mutation

no X108 merge

no Graphiti reduction

public documentation alignment only

Le repo précise aussi que main peut continuer après le freeze P1, mais que l’audit P1 canonique reste le tag p1-freeze-2026-04-22 / commit 99e966a.

Donc pour SRL :

```
SRL = périphérie X108 proofs
```

PAS P1 canonical core

PAS kernel

PAS preuve Lean/TLA

PAS x108_merge

2. Zone SRL réelle trouvée

La bonne zone est :

periphery/brody_memory_readonly/

Le PERIPHERY_PUBLIC_INDEX dit explicitement que cette zone contient :

memory readonly packets

Graphiti bridge material

session ledger

operator control loop artifacts

boundary receipts

et que sa frontière est :

```
readonly
```

memory_decision: false

```
decision_authority: KX108_ONLY
```

La frontière globale confirme :

Periphery does not decide.

Memory does not decide.

Graphiti does not decide.

Brody does not decide.

X108 remains the decision authority.

Donc le bon nom structurel :

```
SRL = formalisation canonique de la session memory readonly existante.
```

Pas un nouveau repo.

3. Chaîne mémoire existante

Chaîne actuelle trouvée

project_intake_capture_buffer

```
↓
```

session_presave_buffer

```
↓
```

session_memory_ledger_v2

```
↓
```

auto_triage_memory_intake

```
↓
```

candidate_export_for_graphiti

```
↓
```

graphiti_candidate_review_gate

```
↓
```

graphiti_import_apply_guarded_manual_only

```
↓
```

memory_replay_query_regression

↓

memory_pipeline_freeze_report

↓

session_close_human_validation_gate

↓

session_close_decision_apply

↓

post_human_review_memory_triage

Le memory_pipeline_freeze_report liste 14 stages dont :

BRODY_SESSION_MEMORY_LEDGER_READONLY

BRODY_AUTO_TRIAGE_MEMORY_INTAKE_READONLY

BRODY_CANDIDATE_EXPORT_FOR_GRAPHITI_READONLY

BRODY_GRAPHITI_CANDIDATE_REVIEW_GATE_READONLY

BRODY_GRAPHITI_IMPORT_APPLY_GUARDED_MANUAL_ONLY

BRODY_MEMORY_REPLAY_QUERY_REGRESSION_READONLY

Donc ton idée SRL rejoint bien une chaîne déjà présente, mais elle n’est pas nommée ni auditée comme SRL.

4. État par module

A. Project intake capture buffer

Présent.

Rôle : capture les fichiers/projets récents, les classe par type, produit records JSON/JSONL/rapport, garde hash chain. Il marque tout comme PENDING_HUMAN_REVIEW / PENDING_NOT_CANON.

Frontière correcte :

memory_decision: false

allowed_to_decide: false

```
emits_act: false
kernel_mutation: false
x108_merge: false
decision_authority: KX108_ONLY
```

Verdict :

OK pour SRL capture périphérique.

Pas encore session-card utilisateur.

B. Session presave buffer

Présent.

Il collecte les pointeurs CURRENT_BRODY_*, notamment :

CURRENT_BRODY_SESSION_MEMORY_LEDGER_READONLY_V2_VALIDATE.txt

CURRENT_BRODY_AUTO_TRIAGE_MEMORY_INTAKE_READONLY_V1_VALIDATE.txt

CURRENT_BRODY_CANDIDATE_EXPORT_FOR_GRAPHITI_READONLY_V1_VALIDATE.txt

CURRENT_BRODY_MEMORY_REPLAY_QUERY_REGRESSION_READONLY_V1_VALIDATE.txt

Il produit :

BRODY_SESSION_PRESAVE_BUFFER_RECORDS.jsonl

BRODY_SESSION_PRESAVE_BUFFER_RECORDS.json

BRODY_SESSION_PRESAVE_BUFFER_SUMMARY.json

BRODY_SESSION_PRESAVE_BUFFER_REPORT.md

Verdict :

OK pour SRL pre-save.

Il garde les pointeurs, pas encore la fiche session complète.

C. Session memory ledger V2

Présent et important.

Il crée :

record_json

record_md

SESSION_INDEX.json

SESSION_LEDGER.jsonl

avec :

session_id

sequence

user_input_sha256

response_md_sha256

response_json_sha256

previous_event_hash

event_hash

Il maintient aussi record_count, latest_event_hash, et une liste des records dans l’index.

Point clé :

triage_status: NOT_APPLIED_NEXT_PALIER

memory_intake_status: TRACE_ONLY_NOT_INDEXED

graphiti_index_write: False

auto_triage: False

Verdict :

Très bon socle SRL.

> `Mais ledger ≠ SRL complète.`

Il trace les tours, pas encore la fiche de fin de session structurée.

D. Session memory follow-up runtime

Présent.

Le runtime contient :

remember_session_turn()

resolve_session_followup()

build_session_memory_snapshot()

> Mais remember_session_turn() ne fait aucune écriture : il renvoie une candidate avec memory_write=false, graphiti_write=false, neo4j_write=false, write_blocked=true.

Le freeze Foundation B confirme que le follow-up est READY, mais partiel : détection par mots-clés explicites, pas encore vector similarity.

Verdict :

OK pour reprise simple.

Insuffisant pour rappel SRL intelligent multi-session.

E. Auto-triage memory intake

Présent, mais problème critique.

Il trie en zones :

CRISTAL

TRANSITION

NEANT

MANDATORY_HOLD

Il calcule :

heat

resonance_signature

resonance_match

axes

alerts

reflex_status

record_hash

Mais incohérence de frontière :

Dans le code :

emits_allow_hold_block: True

emits_verdict: True

Dans le manifeste :

emits_verdict: false

memory_decision: false

allowed_to_decide: false

```
emits_act: false
```

Verdict :

CRITIQUE.

Le module auto-triage est conceptuellement utile.

Mais sa frontière n’est pas stable.

Pour SRL, il faut remplacer HOLD/BLOCK/verdict par des statuts non décisionnels :

REFLEX_ALERT

BOUNDARY_ALERT

REVIEW_REQUIRED

F. Candidate export for Graphiti

Présent et plutôt propre.

Il exporte uniquement les zones CRISTAL et TRANSITION, produit des candidates readonly, et marque explicitement :

memory_decision: false

allowed_to_decide: false

```
emits_act: false
```

emits_verdict: false

graphiti_index_write: false

memory_intake: false

canon: false

needs_human_review: true

Il produit aussi rejected JSONL pour les non-exportables.

Verdict :

OK.

Mais pour SRL il faut renommer les zones :

```
CRISTAL → ACTIVE / KEEP_CANDIDATE
TRANSITION → SEMI_ACTIVE / REVIEW
NEANT → GHOST / SIDE_TABLE
REFLEX → ALERT_TRACE
```

G. Graphiti candidate review gate

Présent et propre.

Il vérifie les violations de frontière, rejette si violation, accepte CRISTAL, met TRANSITION en review, rejette NEANT, et ne permet jamais l'import Graphiti directement :

graphiti_import_allowed: False

graphiti_index_write: False

memory_intake: False

```
readonly: True
decision_authority: KX108_ONLY
```

Verdict :

OK pour SRL review gate.

Peut devenir SRL_REVIEW_GATE_READONLY.

H. Graphiti import apply guarded manual only

Présent, mais zone dangereuse pour SRL.

Le script a un mode --apply et un token manuel :

```
CONFIRM_TOKEN = BRODY_MANUAL_GRAPHITI_WRITE_READONLY_MEMORY_ONLY
```

S'il est appelé avec --apply, il écrit réellement dans Neo4j avec MERGE et SET.

Le résumé indique alors :

graphiti_index_write: bool(args.apply)

memory_intake: bool(args.apply)

manual_apply: bool(args.apply)

Verdict :

CRITIQUE MAIS PAS FORCÉMENT À SUPPRIMER.

Ce module n’est pas SRL readonly.

Il doit être séparé de SRL V0.

À reclasser :

GRAPHITI_IMPORT_APPLY_GUARDED_MANUAL_ONLY

```
= OUT_OF_SRL_READONLY
= MANUAL_WRITE_ZONE
= HUMAN_OPERATOR_ONLY
= QUARANTINE_FOR_SRL_AUDIT
```

Pour SRL V0, il doit rester hors chemin par défaut.

I. Replay query regression

Présent et propre.

Il lance des requêtes de régression, vérifie les frontières, refuse si readonly n’est pas vrai, si decision_authority n’est pas KX108_ONLY, ou si un flag interdit n’est pas False.

Il produit :

query_count

source_query_count

boundary_ok_count

records_jsonl

raw_records_json

report_md

Verdict :

OK pour SRL replay / recall regression.

Mais il teste des requêtes Brody générales, pas encore les scénarios SRL :

- reprendre dernière session
- chercher fiche par date
- charger cold seulement si active insuffisant
- ghost uniquement si demande explicite

J. Session close human validation gate

Présent et utile.

Il produit une queue de validation humaine avec :

KEEP

TRANSITION

NEANT

REFLEX

et précise :

Nothing below is canon until manually validated.

Nothing writes Graphiti from this gate.

Il garde :

human_decision: PENDING

pending_zone: PENDING_HUMAN_REVIEW

learning_state: PENDING_NOT_CANON

Verdict :

Très compatible SRL.

C’est la base du “ne pas supprimer / mettre de côté / validation humaine”.

K. Session close decision apply

Présent.

Il applique les décisions humaines :

```
KEEP → KEEP_MEMORY_CANDIDATE
TRANSITION → TRANSITION_REVIEW_LATER
REFLEX → REFLEX_ALERT_TRACE
NEANT → NEANT_REJECTED
```

Mais son mode V1 est limité : il impose --accept-suggested; il n'y a pas encore de saisie fine décision par décision.

Verdict :

Bon début.

Mais pas encore un vrai panneau humain SRL.

L. Post-human review memory triage

Présent.

Il sépare après décision humaine :

keep_candidates

transition_records

reflex_records

neant_records

Il produit :

BRODY_POST_HUMAN_MEMORY_CANDIDATES_READONLY.jsonl

BRODY_POST_HUMAN_TRANSITION_BACKLOG.jsonl

BRODY_POST_HUMAN_REFLEX_ALERTS.jsonl

BRODY_POST_HUMAN_NEANT_REJECTED.jsonl

Verdict :

Très proche de SRL cold / ghost / active.

Mais les noms ne sont pas encore alignés SRL.

5. Correspondance SRL ↔ existant

SRL_CAPTURE

→ project_intake_capture_buffer

→ session_presave_buffer

SRL_LEDGER

→ session_memory_ledger_readonly_v2

SRL_FOLLOWUP

→ brody_session_memory_runtime

→ brody_session_memory_adapter

SRL_TRIAGE

→ auto_triage_memory_intake_readonly

→ post_human_review_memory_triage_readonly

SRL_REVIEW

→ graphiti_candidate_review_gate_readonly

→ session_close_human_validation_gate_readonly

SRL_RECALL_TEST

→ memory_replay_query_regression_readonly

SRL_FREEZE_REPORT

→ memory_pipeline_freeze_report_readonly

6. Ce qui manque pour une SRL propre

Manque 1 — Fiche session canonique

Aujourd’hui, le ledger trace des tours. Il ne produit pas encore une fiche de fin de session complète avec :

entry_context

exit_point

active_blocks

semi_active_blocks

cold_fragments

ghost_fragments

decisions

pepites

frictions

next_recall_targets

Donc il faut créer :

SRL_SESSION_CARD_READONLY_V0

Manque 2 — Taxonomie SRL stable

Aujourd’hui :

CRISTAL

TRANSITION

NEANT

REFLEX

MANDATORY_HOLD

Pour SRL, il faut :

ACTIVE

SEMI_ACTIVE

COLD

GHOST

REFLEX_ALERT

BOUNDARY_ALERT

Mapping conseillé :

```
CRISTAL → ACTIVE / KEEP_CANDIDATE
TRANSITION → SEMI_ACTIVE / REVIEW_LATER
NEANT → GHOST / SIDE_TABLE
REFLEX → REFLEX_ALERT_TRACE
MANDATORY_HOLD → BOUNDARY_ALERT_NON_DECISIONAL
```

Manque 3 — Mémoire froide

TRANSITION_BACKLOG existe, mais il n’y a pas encore une vraie zone :

```
SRL_COLD_INDEX.jsonl
```

avec règles :

not_loaded_by_default: true

reactivation_policy:

- active_memory_insufficient
- explicit_user_request
- audit_needed
- semantic_collision

Manque 4 — Ghost table

NEANT_REJECTED existe. Mais pour ton concept, il faut éviter le mot “rejected” comme suppression. Il faut :

```
SRL_GHOST_TABLE.jsonl
```

avec :

not_deleted: true

excluded_from_default_context: true

symbolic_hash:

reactivation_policy:

Manque 5 — Symbolic hash

Il y a déjà beaucoup de SHA/event hashes. Mais il manque le hash symbolique de sens :

```
symbolic_hash = SESSION::TYPE::PROJECT::INTENT::ZONE::BOUNDARY
```

Exemple :

SRL::20260604::USER_SESSION::MEMORY_LAYER::GHOST::NO_DELETE

Manque 6 — Recall pack SRL

Le follow-up actuel reprend le dernier contexte. Il faut un vrai paquet :

```
SRL_CONTEXT_RELOAD_PACK.json
```

Avec :

loaded_session_cards

loaded_active_blocks

loaded_semi_active_blocks

cold_loaded: false par défaut

ghost_loaded: false par défaut

deep_recall_reason

Manque 7 — Tests SRL dédiés

Tests à ajouter :

test_srl_no_delete

test_srl_ghost_not_loaded_by_default

test_srl_cold_loaded_only_on_condition

test_srl_exit_point_required

test_srl_context_reload_pack_minimal

test_srl_symbolic_hash_present

test_srl_boundary_flags_consistent

test_srl_no_graphiti_apply_in_readonly_path

7. Métriques SRL à ajouter

Ledger

SRL_LEDGER_RECORD_COUNT

SRL_HASH_CHAIN_VALID

SRL_SESSION_INDEX_PRESENT

SRL_LAST_EVENT_HASH_PRESENT

SRL_RECORD_MD_JSON_PAIR_RATE

Session card

SRL_SESSION_CARD_CREATED

SRL_ENTRY_CONTEXT_PRESENT

SRL_EXIT_POINT_PRESENT

SRL_EXIT_POINT_QUALITY

SRL_NEXT_RECALL_TARGETS_COUNT

Triage

SRL_ACTIVE_COUNT

SRL_SEMI_ACTIVE_COUNT

SRL_COLD_COUNT

SRL_GHOST_COUNT

SRL_REFLEX_ALERT_COUNT

SRL_BOUNDARY_ALERT_COUNT

SRL_ACTIVE_RATIO

SRL_GHOST_RATIO

SRL_TRIAGE_CONFIDENCE

Recall

SRL_CONTEXT_RELOAD_PACK_CREATED

SRL_RELOAD_ACTIVE_COUNT

SRL_RELOAD_COLD_COUNT

SRL_RELOAD_GHOST_COUNT

SRL_DEEP_RECALL_USED

SRL_DEEP_RECALL_REASON

SRL_RECALL_MINIMALITY_SCORE

Boundary

SRL_BOUNDARY_CONSISTENCY_PASS

SRL_READONLY_TRUE

SRL_MEMORY_DECISION_FALSE

SRL_GRAPHITI_WRITE_FALSE

SRL_NEO4J_WRITE_FALSE

SRL_X108_MERGE_FALSE

SRL_KERNEL_MUTATION_FALSE

SRL_DECISION_AUTHORITY_KX108_ONLY

8. Risques détectés

Risque 1 — Incohérence auto-triage

Le code auto-triage active :

```
emits_allow_hold_block = True
emits_verdict = True
```

alors que le manifeste dit false.

Correction :

Remplacer les champs décisionnels par :

emits_boundary_alert: true

emits_reflex_alert: true

emits_verdict: false

emits_allow_hold_block: false

Risque 2 — Module readonly qui peut écrire

graphiti_import_apply_guarded_manual_only peut écrire dans Neo4j avec MERGE si --apply + token sont fournis.

Correction :

Sortir ce module du chemin SRL readonly.

Le classer MANUAL_WRITE_ZONE.

Le garder hors recall / triage / session memory par défaut.

Risque 3 — “NEANT_REJECTED” ressemble à suppression

Ton principe est :

ne pas supprimer

mettre de côté

retrouvable

Le code produit NEANT_REJECTED.

Correction :

```
NEANT_REJECTED → GHOST_SIDE_TABLE
DO_NOT_KEEP → DO_NOT_LOAD_BY_DEFAULT
```

Risque 4 — SRL pas encore nommée

Tout est dispersé sous Brody Memory Readonly. Il manque un document canonique :

docs/runtime/OBSIDIA_SRL_SESSION_REGISTRY_LAYER_READONLY_AUDIT_V0.md

et un module racine :

periphery/brody_memory_readonly/srl_session_registry_layer_readonly/

9. Verdict final

```
SRL_STATUS = PARTIAL_FOUNDATION_READY
```

Base existante :

```
✔ session ledger
✔ presave buffer
✔ follow-up adapter
✔ hash chain
✔ replay regression
✔ human validation gate
✔ post-human triage
✔ Graphiti candidate export
✔ KX108_ONLY globalement respecté
```

Non prêt :

```
✘ pas de fiche session canonique
✘ pas de taxonomie ACTIVE / SEMI_ACTIVE / COLD / GHOST
✘ pas de ghost table explicite
✘ pas de cold index explicite
✘ pas de symbolic hash SRL
✘ pas de context reload pack SRL
✘ incohérence auto-triage boundary
✘ module Graphiti apply à isoler du chemin readonly
```

Freeze d’audit proposé

OBSIDIA_SRL_SESSION_REGISTRY_LAYER_READONLY_AUDIT_V0

Verdict:

FOUNDATION_PRESENT_NOT_CANONICAL

Decision:

NO_X108_MERGE

NO_KERNEL_MUTATION

NO_GRAPHITI_WRITE_IN_SRL_PATH

CREATE_SRL_CANONICAL_READONLY_LAYER

Prochaine action exacte

Créer un palier audit sans toucher au noyau :

scripts/f_srl_session_registry_layer_readonly_audit_v0.py

docs/runtime/OBSIDIA_SRL_SESSION_REGISTRY_LAYER_READONLY_AUDIT_V0.md

docs/runtime/OBSIDIA_SRL_SESSION_REGISTRY_LAYER_READONLY_AUDIT_V0.json

Puis seulement après :

periphery/brody_memory_readonly/srl_session_registry_layer_readonly/

srl_session_card_builder_readonly_v0.py

srl_zone_mapper_readonly_v0.py

srl_cold_ghost_index_readonly_v0.py

srl_context_reload_pack_readonly_v0.py

srl_boundary_consistency_audit_v0.py

Conclusion nette :

Ton idée est déjà en train d’exister dans le repo.

Mais elle est dispersée.

L’audit montre qu’il faut la canoniser comme SRL,

corriger les frontières,

et séparer strictement readonly session memory de manual Graphiti apply.

Audit profond — après exécution locale SRL / Brody Memory

Je ne vois aucun fichier SRL canonique poussé côté GitHub (SRL_*, srl_session_registry_layer_*, audit V0, etc.). Donc si tu as généré les rapports localement, ils sont encore dans _local_audits ou non commit. Mon audit ci-dessous porte sur la structure visible du repo + les problèmes probables qui remontent à l’exécution locale.

Verdict

```
SRL_FOUNDATION = PRÉSENTE
SRL_CANONIQUE = ABSENTE
PIPELINE_MEMORY = PARTIELLEMENT STRUCTURÉ
BOUNDARY = GLOBALEMENT BONNE MAIS 2 FAILLES CRITIQUES
RISQUE PRINCIPAL = confusion READONLY / WRITE_MANUAL
```

La SRL existe déjà en morceaux dans :

periphery/brody_memory_readonly/

Le repo confirme que cette zone contient déjà memory readonly packets, Graphiti bridge material, session ledger, operator control loop artifacts, et que la décision reste KX108_ONLY.

Problèmes critiques qui remontent

1. Auto-triage : incohérence de frontière

Dans le code auto-triage, la frontière dit :

```
emits_allow_hold_block = True
emits_verdict = True
```

Mais le manifeste du même module dit :

```
emits_verdict = false
memory_decision = false
allowed_to_decide = false
emits_act = false
decision_authority = KX108_ONLY
```

Verdict

CRITIQUE

Pour SRL, auto_triage ne doit jamais émettre HOLD / BLOCK / ALLOW / verdict.

Correction conceptuelle :

MANDATORY_HOLD_IMMEDIATE_BLOCK

```
→ BOUNDARY_ALERT_NON_DECISIONAL
```

emits_allow_hold_block: false

emits_verdict: false

emits_boundary_alert: true

emits_reflex_alert: true

2. Le runner auto-triage est incomplet

Le fichier PowerShell run_brody_auto_triage_memory_intake_readonly_v1.ps1 ne lance pas réellement le script Python. Il contient seulement une optimisation CPU + commentaire “le code d’appel s’insérera ici au prochain bloc”.

Problème probable en local

Si tu as lancé la chaîne via les runners, le pipeline peut remonter :

MISSING_AUTO_TRIAGE_VALIDATE_POINTER

MISSING_TRIAGE_RECORDS_JSONL

BRODY_CANDIDATE_EXPORT_FOR_GRAPHITI_READONLY_FAILED

> Parce que l’export Graphiti attend un pointeur CURRENT_BRODY_AUTO_TRIAGE_MEMORY_INTAKE_READONLY_V1_VALIDATE.txt contenant TRIAGE_RECORDS_JSONL=....

Verdict

CRITIQUE PIPELINE

Le module Python existe, mais le runner n’est pas câblé.

3. Graphiti apply : module “readonly” avec écriture réelle possible

Le module graphiti_import_apply_guarded_manual_only peut écrire dans Neo4j avec MERGE / SET si --apply + token manuel sont fournis.

Son résumé active alors :

graphiti_index_write: bool(args.apply)

memory_intake: bool(args.apply)

manual_apply: bool(args.apply)

Verdict

CRITIQUE NOMMAGE / FRONTIÈRE

Ce module ne doit pas être dans SRL V0.

Classement correct :

OUT_OF_SRL_READONLY

MANUAL_GRAPHITI_WRITE_ZONE

HUMAN_OPERATOR_ONLY

QUARANTINE_FOR_SRL_PATH

4. Deuxième module manual apply aussi en écriture possible

Le module graphiti_guarded_manual_apply_from_review_decision_readonly_memory_only écrit aussi dans Neo4j avec MERGE (d:Document...) si --apply est utilisé.

Il indique explicitement :

Graphiti write: true only if --apply and valid token

Neo4j write executed: true only if --apply and valid token

Verdict

PAS SRL READONLY

À garder, mais hors chemin SRL par défaut.

Problèmes structurels SRL

5. Pas encore de fiche session canonique

Le ledger V2 trace des tours avec :

session_id

sequence

user_input_sha256

response_md_sha256

response_json_sha256

previous_event_hash

event_hash

SESSION_LEDGER.jsonl

SESSION_INDEX.json

Mais il ne produit pas encore :

> `SRL_SESSION_CARD.json`

exit_point

active_blocks

semi_active_blocks

cold_fragments

ghost_fragments

next_recall_targets

Verdict

```
SRL = ledger présent
SRL_SESSION_CARD = manquant
```

6. Triage actif mais pas aligné ACTIVE / SEMI_ACTIVE / COLD / GHOST

Aujourd’hui, la chaîne utilise :

CRISTAL

TRANSITION

NEANT

REFLEX

MANDATORY_HOLD

Pour SRL, il faut stabiliser :

ACTIVE

SEMI_ACTIVE

COLD

GHOST

REFLEX_ALERT

BOUNDARY_ALERT

Mapping propre :

```
CRISTAL      → ACTIVE
TRANSITION   → SEMI_ACTIVE / COLD selon contexte
NEANT        → GHOST_SIDE_TABLE
REFLEX       → REFLEX_ALERT_TRACE
MANDATORY_HOLD → BOUNDARY_ALERT_NON_DECISIONAL
```

7. NEANT_REJECTED contredit ton principe “pas supprimé”

Le post-human triage produit :

BRODY_POST_HUMAN_NEANT_REJECTED.jsonl

Mais ton principe SRL est :

ne pas supprimer

mettre de côté

retrouvable si besoin

Correction

NEANT_REJECTED

```
→ GHOST_SIDE_TABLE
```

DO_NOT_KEEP

```
→ DO_NOT_LOAD_BY_DEFAULT
```

8. Hardcode dangereux : len(decisions) != 51

Dans post_human_review_memory_triage_readonly_v1.py, le script échoue si le nombre de décisions n’est pas exactement 51.

Problème local probable

Si ton audit local génère 50, 52, ou autre :

```
BAD_DECISION_COUNT=<n>
```

Verdict

À CORRIGER

Il faut comparer contre le decision_count du summary source, pas contre 51.

Problèmes de preuve / CI / sécurité

9. _local_audits ignoré par le scanner forbidden content

Le scanner de contenu interdit exclut _local_audits, .local_audits, _freezes, _backups, etc.

Risque

Si tes sorties locales contiennent :

tokens

paths privés

secrets

données personnelles

clé API

elles ne seront pas détectées tant qu'elles restent dans _local_audits.

Verdict

OK pour local

DANGEREUX si recopié dans docs/runtime ou commit public

Avant commit : scan manuel des rapports SRL.

10. Follow-up encore partiel

Le freeze Foundation B dit clairement que le resolver fonctionne, mais reste PARTIAL : détection par mots-clés, pas de vector similarity.

Donc

reprendre / continue / développe

```
→ OK partiel
```

retrouver une fiche par sens profond

```
→ pas encore fiable
```

charger cold/ghost conditionnellement

```
→ pas encore implémenté
```

Ce qui est solide

Ledger V2

Solide : hash chain, index, JSONL, records MD/JSON, KX108_ONLY.

Candidate export

```
Solide : export en candidate readonly, canon=false, needs_human_review=true, graphiti_index_write=false, memory_intake=false.
```

Review gate

Solide : rejette si violation boundary, n’écrit pas Graphiti, ne fait pas memory intake, reste KX108_ONLY.

Replay regression

```
Solide : vérifie readonly=true, decision_authority=KX108_ONLY, et refuse les flags interdits s’ils ne sont pas False.
```

Human validation gate

Solide : produit une queue humaine KEEP / TRANSITION / NEANT / REFLEX, rien n’est canon avant validation, rien n’écrit Graphiti.

Audit final des problèmes à remonter

P0 — CRITIQUE

1. auto_triage boundary mismatch :

```
emits_allow_hold_block=True / emits_verdict=True dans le code
```

mais false dans le manifeste.

2. auto_triage runner incomplet :

le .ps1 ne lance pas le script Python.

3. manual Graphiti apply présent dans chemin memory :

écriture Neo4j possible avec --apply + token.

P1 — IMPORTANT

4. SRL canonique absente :

pas de SRL_SESSION_CARD.

5. Pas de taxonomie ACTIVE / SEMI_ACTIVE / COLD / GHOST.

6. NEANT_REJECTED doit devenir GHOST_SIDE_TABLE.

7. post_human_review hardcode decision_count = 51.

P2 — AMÉLIORATION

8. Follow-up resolver partiel, mots-clés seulement.

9. Pas de SRL_CONTEXT_RELOAD_PACK.

10. Pas de symbolic_hash SRL.

11. Pas de métriques SRL dédiées :

active_count, cold_count, ghost_count, reload_minimality, deep_recall_reason.

12. _local_audits exclu du forbidden scan.

Verdict opérationnel

NE PAS MERGE SRL COMME CANONIQUE MAINTENANT.

Faire d’abord :

1. corriger auto_triage boundary

2. câbler le runner auto_triage

3. sortir manual Graphiti apply du chemin SRL readonly

4. créer SRL_SESSION_CARD_READONLY_V0

5. renommer NEANT en GHOST_SIDE_TABLE

6. remplacer hardcode 51 par compteur dynamique

7. ajouter SRL_CONTEXT_RELOAD_PACK

8. ajouter audit boundary SRL dédié

Nom de freeze propre après correction :

OBSIDIA_SRL_SESSION_REGISTRY_LAYER_READONLY_V0_PRECANONICAL

Statut actuel :

FOUNDATION_PRESENT

NOT_CANONICAL

READONLY_BOUNDARY_PARTIAL

MANUAL_WRITE_MODULES_TO_QUARANTINE

SRL_METRICS_MISSING

Oui. J’ai audité plus large que SRL dans obsidia-x108-proofs. Je vois plusieurs incohérences code à remonter.

Verdict global

```
REPO_STATUS = SOLIDE MAIS PAS PROPREMENT UNIFIÉ
```

Le noyau KX108 reste protégé.

La majorité des enveloppes readonly sont cohérentes.

Mais la périphérie Brody / Graphiti / Memory contient des incohérences de contrat, de nommage et de runners.

P0 — Critique

1. Auto-triage : contrat cassé

Le module auto_triage_memory_intake_readonly déclare dans son code :

emits_allow_hold_block: True

emits_verdict: True

```
alors que le manifeste du même module dit emits_verdict=false, memory_decision=false, allowed_to_decide=false, emits_act=false.
```

Correction :

```
emits_allow_hold_block = false
emits_verdict = false
emits_boundary_alert = true
emits_reflex_alert = true
```

Et renommer :

MANDATORY_HOLD_IMMEDIATE_BLOCK

```
→ BOUNDARY_ALERT_NON_DECISIONAL
```

2. Runner auto-triage incomplet

Le .ps1 d'auto-triage ne lance pas le script Python. Il ajuste juste les priorités CPU puis indique que "le code d'appel du script d'auto-triage s'insérera ici au prochain bloc".

Conséquence probable :

MISSING_AUTO_TRIAGE_VALIDATE_POINTER

MISSING_TRIAGE_RECORDS_JSONL

BRODY_CANDIDATE_EXPORT_FOR_GRAPHITI_READONLY_FAILED

L'export Graphiti attend justement un pointeur CURRENT_BRODY_AUTO_TRIAGE_MEMORY_INTAKE_READONLY_V1_VALIDATE.txt contenant TRIAGE_RECORDS_JSONL.

3. Graphiti write dans des modules "readonly / memory"

Deux modules peuvent écrire réellement dans Neo4j avec --apply + token :

graphiti_import_apply_guarded_manual_only

graphiti_guarded_manual_apply_from_review_decision_readonly_memory_only

```
Le premier fait MERGE / SET sur BrodyMemoryDoc. Le second fait MERGE (d:Document...) et marque graphiti_index_write=true, memory_intake=true quand --apply est utilisé.
```

À sortir du chemin SRL / readonly par défaut :

MANUAL_GRAPHITI_WRITE_ZONE

HUMAN_OPERATOR_ONLY

OUT_OF_SRL_READONLY

4. Mot de passe Neo4j par défaut

neo4j_brody_guide_bridge_readonly_v1.py a un fallback :

```
password = os.environ.get("NEO4J_PASSWORD", "admin1234")
```

Même si c'est local, c'est mauvais pour un repo public/auditable.

Correction :

NEO4J_PASSWORD obligatoire

aucun fallback

fail fermé

P1 — Important

5. Full runtime : bug de statut Graphiti live

_probe_graphiti() renvoie :

GRAPHITI_NEO4J_LIVE_READONLY_PASS

quand Neo4j live est disponible.

Mais plus bas, le runtime teste :

```
graphiti_live = r["graphiti_probe"]["status"] == "GRAPHITI_LIVE_READONLY_PASS"
```

Donc graphiti_live peut rester faux même si Neo4j est live.

Correction :

GRAPHITI_LIVE_READONLY_PASS

```
→ GRAPHITI_NEO4J_LIVE_READONLY_PASS
```

ou créer une fonction :

is_graphiti_live(status)

6. Orchestrateur automation ↔ auto-triage : schéma incompatible

L’orchestrateur lit :

memory_candidate

review_candidate

reject_candidate

axes

à la racine du résultat triage.

Mais classify_record() renvoie plutôt :

zone

is_candidate

reasons

metrics.axes

metrics.alerts

metrics.reflex_status

Résultat : le pipeline peut tourner sans crash, mais interpréter le triage comme vide/faux.

Correction :

```
memory_candidate = item["is_candidate"]
review_candidate = zone == "TRANSITION"
reject_candidate = zone in {"NEANT", "GHOST"}
axes = item["metrics"]["axes"]
```

7. Hardcodes 29 et 51

```
post_human_review_memory_triage_readonly_v1.py échoue si len(decisions) != 51.
graphiti_review_decision_apply_readonly_v1.py échoue si review_record_count != 29 et si len(reviews) != 29.
```

Correction :

comparer au summary source

ne jamais figer un nombre de lignes

8. “NEANT_REJECTED” contredit la logique SRL

Le post-human triage produit :

BRODY_POST_HUMAN_NEANT_REJECTED.jsonl

Pour SRL, ce n’est pas un rejet. C’est une mise en côté.

Correction :

NEANT_REJECTED

```
→ GHOST_SIDE_TABLE
```

DO_NOT_KEEP

```
→ DO_NOT_LOAD_BY_DEFAULT
```

9. allow_manual_apply exposé dans l’API mais pas utilement câblé

La route /api/brody/chat expose :

```
allow_manual_apply: bool = False
```

Mais la route principale ne transmet pas ce champ à un chemin manual apply. Elle passe par run_brody_automation_layer, pas par le full runtime manual apply.

Verdict :

champ mort / ambigu

Soit le supprimer de cette route, soit le faire remonter explicitement comme :

```
manual_apply_not_available_from_api_chat = true
```

P2 — Cohérence / auditabilité

10. safe_backend_response ne nettoie pas final_answer

La route met :

response

final_answer

response_md

dans le payload.

Mais safe_backend_response() ne sanitize que :

response

response_text

controlled_response.text

Donc final_answer peut diverger de response si un token interdit est nettoyé dans response mais pas dans final_answer.

Correction :

for field in ("response", "response_text", "final_answer"):

sanitize

11. Terminal : texte “HOLD STRUCTUREL” malgré emits_allow_hold_block=false

Le terminal readonly met bien :

emits_allow_hold_block: False

emits_verdict: False

Mais en cas de risque d’action, il produit :

HOLD STRUCTUREL.

Action détectée.

Ce n’est pas forcément une décision moteur, mais pour un auditeur externe c’est ambigu.

Correction :

HOLD STRUCTUREL

```
→ REFUS CONTEXTUEL READONLY
→ ACTION_RISK_DETECTED_CONTEXT_ONLY
```

12. Context packet query dépend de Neo4j live

context_packet_query_readonly exige NEO4J_PASSWORD, ouvre un driver Neo4j, puis fait un MATCH live.

C’est readonly, mais ce n’est pas autonome.

Correction :

si Neo4j absent :

fallback frozen index / Graphiti V20 / JSON readonly

Sinon le runtime readonly dépend trop d’un service live.

Résumé des incohérences à corriger

P0

1. auto_triage emits_allow_hold_block=True / emits_verdict=True

2. auto_triage runner incomplet

3. modules Graphiti write dans famille readonly/memory

4. NEO4J_PASSWORD fallback admin1234

P1

5. statut Graphiti live mal comparé

6. schéma auto_triage incompatible avec orchestrateur

7. hardcodes 29 / 51

8. NEANT_REJECTED mauvais nom SRL

9. allow_manual_apply exposé mais non câblé

P2

10. final_answer non sanitizé par safe_backend_response

11. HOLD STRUCTUREL ambigu côté terminal readonly

12. context_packet_query dépend de Neo4j live sans fallback frozen

Verdict final

PAS DE FAILLE NOYAU KX108 OBSERVÉE.

PAS DE X108_MERGE DIRECT OBSERVÉ.

PAS DE KERNEL_MUTATION DIRECTE OBSERVÉE.

Mais la périphérie Brody/Graphiti/Memory doit être nettoyée avant canonisation SRL.

Priorité patch :

1. auto_triage boundary + runner

2. isoler manual Graphiti write

3. corriger graphiti_live status

4. corriger schéma orchestrator ↔ triage

5. retirer hardcodes 29/51

6. ajouter SRL boundary audit dédié

Audit remonté plus loin — au-delà SRL

Verdict net :

LE NOYAU X108 N’EST PAS LE PROBLÈME PRINCIPAL.

Les incohérences sont dans :

1. CI / audit coverage

2. manifest / hash coverage

3. runtime API Brody

4. hydratation mémoire

5. Graphiti / Neo4j bridge

6. contrats readonly périphériques

P0 — Très critique

1. Le manifeste récursif exclut apps/

Le script de manifeste exclut :

"apps"

"Demo-obsidia-x108-proof"

"_local_audits"

".claude"

Donc toute la surface API Brody dans apps/obsidia_api/ n’est pas couverte par le hash récursif.

Problème :

apps/ contient pourtant :

- routes API
- safe_response
- runtime loader
- Brody chat
- orchestration mémoire
- authority matrix

Donc aujourd’hui :

```
MANIFEST_SHA256_RECURSIVE ≠ audit complet runtime
```

Correction :

retirer "apps" des EXCLUDE_DIRS

ou créer MANIFEST_RUNTIME_API_SHA256.json séparé

2. CI ne se déclenche pas sur Markdown ni _local_audits

Le workflow ignore :

'**.md'

'_local_audits/**'

Donc si tu ajoutes/corriges uniquement des audits .md, des docs de freeze ou des rapports SRL locaux, la CI peut ne pas tourner.

Correction :

ne pas ignorer tous les .md

au minimum déclencher CI sur :

docs/runtime/**

docs/freeze/**

docs/status/**

periphery/**/README_BOUNDARY.md

3. Content hydration : validation désactivée volontairement

Dans brody_content_hydration_readonly_v1.py, la fonction validate_packet() fait directement :

return # LOBOTOMIE REUSSIE - PLUS AUCUN CHECK SÉMANTIQUE

Donc tout le bloc de vérification boundary après ce return est mort.

Problème :

Le module s’appelle readonly.

Mais il ne valide plus réellement :

- memory_decision
- allowed_to_decide
- emits_act
- kernel_binding
- x108_merge
- decision_authority

Correction :

retirer le return

restaurer validate_packet()

```
ajouter test qui échoue si allowed_to_decide=true
```

4. Content hydration écrit sur disque avec chemin absolu Windows

À la fin du script d’hydratation :

```
output_log_path = r"C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs\GAVAGE_REALISE_ACCORD.json"
with open(output_log_path, 'w', encoding='utf-8') as f:
```

...

```
print('[✓] === GAVAGE RÉUSSI - EXPORT DISQUE SÉCURISÉ ===')
```

C’est incompatible avec :

```
readonly
```

cross-platform

public repo

CI Linux

audit propre

zéro chemin personnel

Correction :

supprimer écriture absolue

remplacer par --out-json optionnel uniquement

interdire tout write si aucun out-json explicite

5. Content hydration scanne trop large si le packet est vide

Si context_packet.items est vide, le module scanne les roots et ajoute tous les .md / .json trouvés.

Or les roots incluent :

x108_root

obsidia-engine-candidate

_local_audits

Problème :

Un context packet vide peut devenir un gavage massif de fichiers locaux.

Risque :

- bruit énorme
- fuite de contenu local
- réponses polluées
- audit non déterministe

Correction :

```
si items vide → NO_ITEMS_TO_HYDRATE
```

ne jamais remplir automatiquement depuis tout le filesystem

P1 — Important

6. Runtime loader probablement obsolète / partiellement déconnecté

runtime_loader.py cherche des modules comme :

periphery.brody.brody_runtime_readonly

periphery.language.language_router

periphery.context.context_packet_builder_v2

periphery.memory.memory_candidate

periphery.graphiti.graphiti_readonly_bridge

Mais la vraie surface active semble maintenant être surtout :

periphery/brody_memory_readonly/**

apps/obsidia_api/**

Problème :

runtime_components peut afficher BACKEND_STUB alors que les vrais modules Brody récents existent ailleurs.

Correction :

mettre runtime_loader à jour sur les modules actuels

ou le marquer LEGACY_RUNTIME_LOADER

7. Matrice d’autorité : chevauchement CAPABILITY_SCOPE / CONTEXT_ANALYSIS

La matrice met des questions du type :

que peux-tu faire

tes capacités

quelles sont tes limites

dans plusieurs catégories.

CAPABILITY_SCOPE contient ces patterns. CONTEXT_ANALYSIS contient aussi :

quelles sont tes limit

tes capacités

que peux-tu faire

L’ordre protège une partie, mais les variantes orales peuvent tomber dans CONTEXT_ANALYSIS au lieu de CAPABILITY_SCOPE.

Correction :

sortir les patterns capacité/limite de CONTEXT_ANALYSIS

laisser CAPABILITY_SCOPE seul propriétaire

ajouter tests typographiques/oraux

8. safe_backend_response() ne sanitize pas final_answer

La route Brody renvoie :

response

final_answer

response_md

Mais safe_backend_response() ne nettoie que :

response

response_text

controlled_response.text

Donc response peut être protégé, mais final_answer garder une formulation interdite.

Correction :

for field in ("response", "response_text", "final_answer"):

...

9. Memory response chain dépend encore trop de services locaux

Le fallback Graphiti V20 HTTP appelle directement :

http://127.0.0.1:8011/graph/v20/frozen/context

http://127.0.0.1:8011/graph/v20/frozen/search

C’est readonly, mais ça reste une dépendance locale implicite.

Correction :

déclarer explicitement :

```
local_http_dependency=true
fallback_used=true
service_required=ObsidiaShell:8011
```

et ne jamais appeler ça “fully offline”.

10. context_packet_query_readonly dépend de Neo4j live

Le module exige NEO4J_PASSWORD, ouvre un driver Neo4j, puis fait un MATCH live.

Il est readonly, oui, mais pas autonome.

Correction :

ajouter fallback JSON / frozen index dans ce module aussi

ou le renommer LIVE_NEO4J_CONTEXT_QUERY_READONLY

P2 — Dette propre / auditabilité

11. Texte non professionnel dans code public

Exemples :

LOBOTOMIE REUSSIE

KX108 FORCE PASS

GAVAGE RÉUSSI

Pour un audit RSSI / institutionnel, ça doit disparaître.

Correction :

NO_BOUNDARY_VALIDATION_DISABLED_FOR_LEGACY_COMPAT

HYDRATION_EXPORT_WRITTEN

Mais mieux : réactiver la validation.

12. Hydration lit beaucoup de formats sensibles

READ_EXTS inclut :

.md, .txt, .json, .jsonl, .py, .ps1, .csv, .lean, .yaml, .yml, .toml, .html, .css, .js, .ts, .docx

Ce n’est pas mauvais en soi, mais combiné au scan automatique de _local_audits, ça peut aspirer trop large.

Correction :

hydration_scope_allowlist obligatoire

max_files

max_total_chars

exclude _local_audits par défaut

Synthèse des nouvelles incohérences trouvées

P0

1. MANIFEST exclut apps/

2. CI ignore .md et _local_audits

3. content_hydration validate_packet désactivé

4. content_hydration écrit sur chemin absolu Windows

5. context_packet vide → scan massif fichiers locaux

P1

6. runtime_loader pointe vers anciens modules / stubs probables

7. authority matrix chevauche CAPABILITY_SCOPE / CONTEXT_ANALYSIS

8. final_answer non sanitizé

9. memory_response_chain dépend de localhost:8011

10. context_packet_query dépend de Neo4j live sans fallback interne

P2

11. wording non professionnel dans code public

12. hydration scope trop large

Patch order recommandé

1. Restaurer validate_packet dans content_hydration.

2. Supprimer l’écriture absolue GAVAGE_REALISE_ACCORD.json.

3. Interdire scan automatique si context_packet vide.

4. Retirer apps/ du manifest exclude ou créer manifest API dédié.

5. Modifier CI : ne pas ignorer docs/runtime, docs/freeze, docs/status.

6. Nettoyer wording public.

7. Corriger safe_backend_response pour final_answer.

8. Renommer context_packet_query_readonly en live_neo4j_context_query_readonly ou ajouter fallback.

9. Mettre runtime_loader à jour ou le déclasser LEGACY.

10. Ajouter tests pour authority matrix oral/freestyle.

Verdict final remonté

Le plus gros risque n’est pas X108.

Le plus gros risque est la périphérie Brody Memory :

elle dit readonly,

mais certains modules désactivent les checks,

scannent trop large,

écrivent localement,

ou dépendent de services vivants sans le déclarer clairement.

Nom d’audit à créer :

OBSIDIA_PERIPHERY_DEEP_INCOHERENCE_AUDIT_V0

Statut :

CORE_X108_OK

PERIPHERY_BOUNDARY_PARTIAL

HYDRATION_CRITICAL_FIX_REQUIRED

MANIFEST_COVERAGE_INCOMPLETE

CI_DOC_AUDIT_GAP

Audit remonté — niveau plus ancien

Je confirme : plus on remonte, plus le problème se déplace vers runtime / CI / preuves d’audit / routes API, pas vers KX108 Core.

```
CORE_X108 = stable
PERIPHERY/API = incohérences historiques
AUDIT_COVERAGE = incomplet
READONLY = parfois déclaratif, pas toujours réel
```

P0 — Très critique

1. /api/x108/memory/candidates/append écrit réellement

La route annonce une boundary readonly :

```
memory_write: false
decision_authority: KX108_ONLY
```

mais l’endpoint appelle réellement :

append_memory_candidate(candidate)

Et append_memory_candidate() écrit dans :

_local_audits/memory_candidate_ledger.jsonl

avec ouverture en mode append :

```
with open(path, "a", encoding="utf-8") as f:
```

f.write(...)

Verdict

P0 — CONTRADICTION READONLY / WRITE LOCAL

Ce n’est pas une écriture Graphiti, ni une mutation kernel, mais ce n’est pas readonly strict.

Correction :

```
renommer memory_write=false → canonical_memory_write=false
ajouter local_audit_write=true
```

ou bloquer l’endpoint append en mode public

2. Content hydration : mauvaise version actuelle, backup plus propre

La version actuelle ajoute un export forcé vers :

C:\Users\User\Desktop\obsidia-engine-proof-core\obsidia-x108-proofs\GAVAGE_REALISE_ACCORD.json

Mais le backup bak_safe_scan_20260520_231137 n’a pas cet export forcé : il écrit seulement si --out-json ou --out-md sont fournis, puis print le résultat.

Verdict

P0 — RÉGRESSION INTRODUITE APRÈS BACKUP

Action :

restaurer la fin du backup propre

supprimer GAVAGE_REALISE_ACCORD.json

3. validate_packet() désactivé

Le module d’hydratation retourne immédiatement :

return # LOBOTOMIE REUSSIE - PLUS AUCUN CHECK SÉMANTIQUE

Donc les vérifications boundary après ce return ne s’exécutent jamais.

Verdict

P0 — BOUNDARY CHECK COURT-CIRCUITÉ

Action :

supprimer le return

réactiver les checks

ajouter test hostile :

```
allowed_to_decide=true → doit fail
```

4. CI ignore trop de surfaces d’audit

Le workflow ignore les .md et _local_audits/**.

Donc des modifications sur :

docs/runtime/**

docs/freeze/**

docs/status/**

_local_audits/**

peuvent ne pas déclencher la CI.

Verdict

P0 — AUDIT DOCS NON GARANTI PAR CI

Correction :

paths-ignore:

- '.claude/**'

- '_local_audits/**'

Mais ne plus ignorer tous les **.md.

5. Le manifeste exclut apps/

Le manifest récursif exclut :

"apps"

Mais apps/obsidia_api contient :

routes Brody

auth

safe_response

authority matrix

runtime orchestration

memory routes

x108 routes

Donc le hash récursif ne couvre pas une grosse partie du runtime réel.

Verdict

P0 — MANIFEST GLOBAL INCOMPLET

Correction :

retirer apps/ de EXCLUDE_DIRS

ou créer MANIFEST_API_RUNTIME_SHA256.json

P1 — Important

6. Auth API désactivée par défaut en dev/test

```
auth.py laisse passer sans API key si APP_ENV est dev, development, test, sauf si OBSIDIA_AUTH_ENABLED=true.
```

C’est acceptable localement, mais dangereux si une démo publique est lancée avec l’environnement par défaut.

Correction runbook

```
APP_ENV=prod
OBSIDIA_AUTH_ENABLED=true
OBSIDIA_API_KEY=<clé>
```

À imposer dans tout runbook public.

7. source_runtime_cache: “available” ne veut plus dire hydratable

Le cache dit que available signifie maintenant “available for selection”, pas “fully hydratable”.

Problème

Pour un audit externe, available_families peut être mal interprété comme :

packs réellement présents

alors que ça peut vouloir dire :

familles connues dans le registre

Correction :

```
available_families → selectable_families
hydration_available_families → familles avec pack local présent
```

8. Memory routes masquent les erreurs

Dans /api/memory/sources, /api/memory/candidates, /api/memory/candidate/from-message, /api/memory/promotion-policy, les exceptions sont avalées avec except Exception: pass.

Verdict

P1 — FAIL-SOFT TROP SILENCIEUX

Correction :

```
retourner module_error=true
```

error_type

error_message_short

```
source=MODULE_ERROR
```

Sans exposer stack trace complète.

9. Runtime loader probablement legacy

runtime_loader.py charge des anciens chemins :

periphery.brody.*

periphery.language.*

periphery.context.*

periphery.memory.*

periphery.graphiti.*

Mais la surface réelle actuelle est surtout :

periphery/brody_memory_readonly/**

apps/obsidia_api/**

runtime_wiring/**

Verdict

P1 — LOADER DÉCALÉ DE LA RÉALITÉ RUNTIME

Correction :

```
runtime_loader.py → LEGACY_RUNTIME_LOADER
```

ou le reconnecter aux modules actuels

10. Hydration lit trop large

Le module accepte beaucoup d’extensions :

.md, .txt, .json, .jsonl, .py, .ps1, .csv, .lean, .yaml, .yml, .toml, .html, .css, .js, .ts, .docx

Et les roots par défaut incluent _local_audits.

Verdict

P1 — RISQUE DE GAVAGE / FUITE LOCALE / BRUIT

Correction :

exclude _local_audits par défaut

max_files

max_total_chars

allowlist stricte

P2 — Cohérence ancienne

11. Source pack context : “metadata-only” peut devenir contexte

Le bridge accepte des résultats avec hydration_status en :

OK

METADATA_ONLY

Ce n’est pas forcément faux, mais il faut signaler quand le contexte est seulement metadata.

Correction :

```
context_quality = FULL_LOCAL / METADATA_ONLY / EMPTY
```

12. Source pack bridge route par X108 mais uniquement context-only

```
Le bridge force critical_action_requested=False et action_candidate_type="EMIT_CONTEXT".
```

C’est cohérent, mais il ne faut pas vendre ça comme une vraie décision X108 d’action. C’est seulement :

ALLOW_CONTEXT_ONLY

Correction wording :

x108_context_gate_decision

```
≠ x108_action_decision
```

Nouvelle synthèse remontée

P0

1. /api/x108/memory/candidates/append écrit dans _local_audits malgré memory_write=false.

2. content_hydration current a régressé par rapport au backup propre.

3. validate_packet est court-circuité.

4. CI ignore les .md et _local_audits.

5. manifest récursif exclut apps/.

P1

6. API auth désactivée par défaut en dev/test.

7. available_families ≠ hydratable_families.

8. routes memory avalent les erreurs.

9. runtime_loader legacy/décalé.

10. hydration scope trop large.

P2

11. METADATA_ONLY peut être injecté comme contexte.

12. X108 context gate peut être confondu avec X108 action decision.

Priorité patch remontée

1. Réparer content_hydration depuis le backup propre.

2. Réactiver validate_packet.

3. Supprimer l’écriture GAVAGE_REALISE_ACCORD.

4. Interdire scan massif si context_packet vide.

5. Corriger /api/x108/memory/candidates/append :

```
local_audit_write=true ou endpoint désactivé en public.
```

6. Inclure apps/ dans un manifest API.

7. Modifier CI pour auditer docs/runtime + docs/freeze.

8. Renommer available_families.

9. Ajouter erreurs explicites dans routes memory.

10. Ajouter mode prod/auth obligatoire dans runbooks publics.

Verdict :

```
KX108_CORE = OK
BRODY_RUNTIME = PARTIELLEMENT DURCI
```

```
MEMORY/HYDRATION = À CORRIGER AVANT SRL CANONIQUE
AUDIT_COVERAGE = INCOMPLET
PUBLIC_READY = NON, tant que P0 non corrigés
```

Audit continué — couche API / routes / enveloppes / journaux

Verdict nouveau :

NOUVEAUX P0 TROUVÉS

1. Plusieurs routes écrivent localement malgré boundary readonly.

2. Certaines routes sensibles ne semblent pas protégées par require_api_key.

3. build_output_envelope peut laisser un module écraser les flags souverains.

4. periphery_ops a un boundary explicitement tronqué.

5. event bus et interface log écrivent en audit local tout en se déclarant readonly.

P0 — Critique

1. build_output_envelope() laisse les données écraser les flags souverains

Dans output_envelope.py, l’enveloppe pose d’abord :

```
readonly=True
memory_write=False
graphiti_write=False
neo4j_write=False
emits_act=False
emits_verdict=False
```

puis fait :

payload.update(data)

à la fin.

Donc si data contient :

{

"memory_write": true,

"emits_act": true

}

il peut écraser la boundary.

Contraste : safe_backend_response() fait l'inverse proprement : il merge d'abord, puis réapplique les flags souverains.

Correction :

payload.update(data)

payload.update(_CORE_BOUNDARY)

Même logique que safe_backend_response.

2. /api/periphery/pipeline/run utilise l'enveloppe faible

periphery_pipeline_run() utilise build_output_envelope() au lieu de safe_backend_response().

Donc cette route hérite du problème précédent.

Correction :

soit remplacer par safe_backend_response

soit patcher build_output_envelope

Priorité : patcher build_output_envelope, car il peut être utilisé ailleurs.

3. Routes API sensibles probablement non protégées par API key

main.py inclut tous les routers globalement : Brody, memory, graphiti, x108, blockchain, audit, periphery_ops, bus, etc.

Mais la protection Depends(require_api_key) apparaît explicitement sur /api/brody/chat.

> `Les routes x108, blockchain, memory, periphery visibles ne montrent pas cette dépendance dans leurs endpoints principaux. Exemple : x108 définit directement ses routes sous APIRouter(prefix="/api/x108"). blockchain fait pareil sous /api/blockchain. periphery_ops fait pareil sous /api/periphery.`

Correction :

mettre require_api_key au niveau router

ou middleware global pour toutes les routes sauf /api/health, /api/status, /api/readiness

4. /api/x108/memory/candidates/append écrit dans _local_audits

La route appelle :

append_memory_candidate(candidate)

La fonction écrit réellement en append dans :

_local_audits/memory_candidate_ledger.jsonl

Mais la route renvoie une boundary avec :

```
memory_write: false
readonly: true
```

Correction :

```
canonical_memory_write=false
local_audit_write=true
filesystem_write=true
```

ou désactiver cet endpoint en public.

5. /api/blockchain/world/gateway écrit un sovereign ticket

La route crée un ticket puis appelle :

store_ticket(ticket)

store_ticket() écrit en append dans :

audit/sovereign_tickets.jsonl

Même problème : c’est dry-run côté action réelle, mais pas readonly côté filesystem.

Correction :

```
dry_run_only=true
real_egress=false
local_audit_write=true
filesystem_write=true
```

6. /api/blockchain/world/bus-dispatch écrit dans le bus local

La route appelle :

publish_event(...)

publish_event() écrit en append dans :

audit/world_action_bus.jsonl

Correction identique :

```
world_action=false
real_egress=false
local_audit_write=true
filesystem_write=true
```

P1 — Important

7. periphery_ops annonce READ_ONLY mais contient une route de log

Le fichier annonce :

Mode: READ_ONLY

Mais /api/periphery/interface/log-event appelle :

log_interface_event(...)

Et log_interface_event() écrit en append dans _local_audits/interface_event_log.jsonl.

Correction :

```
renommer READ_ONLY → NO_KERNEL_MUTATION / NO_GRAPHITI_WRITE
ajouter filesystem_write=true
```

8. Bug de type dans /api/periphery/interface/log-event

InterfaceEventPayload.log_path est un str.

Mais log_interface_event() attend plutôt Path | None, puis fait :

path.parent.mkdir(...)

Avec un str, ça peut provoquer :

AttributeError: 'str' object has no attribute 'parent'

La route catch l’erreur, donc ça peut échouer silencieusement côté fonctionnel.

Correction :

log_interface_event(..., Path(body.log_path))

ou supprimer log_path utilisateur.

9. log_path utilisateur = risque d’écriture arbitraire locale

Même si le bug de type est corrigé, exposer log_path dans l’API permettrait potentiellement d’écrire ailleurs que _local_audits.

Correction :

ne jamais accepter log_path depuis l’API

forcer _local_audits/interface_event_log.jsonl

10. periphery_ops reconnaît lui-même une boundary tronquée

Le fichier contient :

LEGACY_BOUNDARY_TRUNCATED

4 of 15 canonical flags

Expand to 15 flags or deprecate legacy routes

Donc c’est déjà une dette documentée.

Correction :

remplacer _BOUNDARY par la boundary 15 flags canonique

11. Event bus : déclaré readonly mais persiste des logs

event_bus.py met :

```
memory_write=False
readonly=True
```

Mais AuditEventWriter écrit dans audit_logs/event_bus_<date>.jsonl.

Correction :

```
memory_write=false
canonical_memory_write=false
audit_log_write=true
filesystem_write=true
```

12. Event bus drop silencieux

Le commentaire dit :

Bus: drops events silently when full

Et le code incrémente _dropped, mais ne force pas d'alerte.

Pour un système audit-ready, "silently" est dangereux.

Correction :

drop_count exposé

```
drop_alert=true si dropped > 0
```

jamais "silent" dans le wording audit

P2 — Cohérence

13. memory_write=false est trop ambigu

Le repo utilise memory_write=false pour dire :

pas d'écriture mémoire canonique

pas d'écriture Graphiti

pas de mutation kernel

Mais il y a quand même :

_local_audits/*.jsonl

audit/*.jsonl

audit_logs/*.jsonl

Donc il faut séparer :

```
canonical_memory_write=false
```

```
graphiti_write=false
kernel_mutation=false
local_audit_write=true/false
filesystem_write=true/false
```

Nouvelle synthèse cumulée

NOUVEAUX P0

1. build_output_envelope permet override des flags souverains.

2. /api/periphery/pipeline/run utilise cette enveloppe faible.

3. Plusieurs routes sensibles semblent sans require_api_key.

4. /api/x108/memory/candidates/append écrit localement.

5. /api/blockchain/world/gateway écrit sovereign_tickets.jsonl.

6. /api/blockchain/world/bus-dispatch écrit world_action_bus.jsonl.

NOUVEAUX P1

7. periphery_ops annonce READ_ONLY mais log-event écrit.

8. log-event a bug str vs Path.

9. log_path utilisateur = risque d’écriture arbitraire.

10. boundary periphery_ops explicitement tronquée.

11. event_bus écrit audit_logs malgré readonly.

12. event_bus drop silencieux.

NOUVEAU P2

13. memory_write=false doit être séparé de filesystem_write/local_audit_write.

Patch order mis à jour

1. Patch build_output_envelope :

data merge d’abord, boundary après.

2. Ajouter auth globale :

require_api_key sur x108/blockchain/memory/periphery/audit/bus.

3. Renommer les flags :

```
memory_write=false → canonical_memory_write=false
```

ajouter local_audit_write / filesystem_write.

4. Corriger /api/x108/memory/candidates/append :

endpoint disabled public ou local_audit_write explicite.

5. Corriger blockchain world gateway/bus-dispatch :

audit write explicite, pas readonly strict.

6. Corriger periphery interface log :

pas de log_path utilisateur,

Path forcé,

filesystem_write explicite.

7. Étendre boundary periphery_ops à 15 flags.

8. Event bus :

```
audit_log_write=true,
```

drop_alert si dropped > 0.

Verdict actualisé :

```
KX108_CORE = OK
API_RUNTIME = NON PUBLIC_READY
READONLY_TERME = À REFACTORER
FILESYSTEM_WRITES = SOUS-DÉCLARÉS
AUTH_COVERAGE = INCOMPLÈTE
```

```
BOUNDARY_ENVELOPE = À PATCHER EN PRIORITÉ
```

Audit continué — nouvelles incohérences remontées

NIVEAU AUDIT :

routes API

bus layer

Graphiti proxy

source runtime

runtime freeze dashboard

auth coverage

envelope boundary

P0 — Critique

1. Auth appliquée seulement à Brody

Recherche repo : require_api_key n’apparaît que dans apps/obsidia_api/routes/brody.py.

Mais main.py inclut beaucoup de routers sensibles :

memory

graphiti

x108

blockchain

audit

periphery_ops

bus

runtime_freeze

source_runtime

os_map

sans auth globale visible.

Verdict

P0 — AUTH COVERAGE INCOMPLETE

Correction :

auth middleware global

ou Depends(require_api_key) au niveau de chaque router sensible

Exception possible uniquement :

/api/health

/api/readiness

/api/status minimal

/

2. Routes /bus/* hors namespace /api

bus.py expose :

GET /bus/stats

GET /bus/bridge

POST /bus/signal

et pas /api/bus/*.

Elles utilisent APIRouter() sans prefix.

Problème

surface publique hors convention API

auth non visible

rate-limit seulement global si activé

route plus facile à oublier dans les audits

Correction :

```
prefix="/api/bus"
```

ou documenter explicitement :

BUS_ROUTES_PUBLIC_READONLY_UNAUTHENTICATED

3. /bus/* utilise build_output_envelope() faible

Les trois routes bus renvoient avec :

build_output_envelope(...)

Or build_output_envelope() applique la boundary d’abord puis fait payload.update(data), donc les données peuvent écraser readonly, memory_write, emits_act, etc.

Verdict

P0 — BUS ROUTES HÉRITENT DU BOUNDARY OVERRIDE BUG

Patch prioritaire :

payload.update(data)

payload.update(_CORE_BOUNDARY)

4. graphiti_v20_readonly_client protège mal les flags entrants

Le client Graphiti fait :

data.setdefault("readonly", True)

data.setdefault("graphiti_write", False)

data.setdefault("neo4j_write", False)

Problème

setdefault ne corrige pas un payload distant qui renverrait déjà :

{

"graphiti_write": true,

"neo4j_write": true,

"readonly": false

}

Il faut forcer, pas seulement poser par défaut.

Correction :

data.update({

"readonly": True,

"graphiti_write": False,

"neo4j_write": False,

"decision_authority": "KX108_ONLY",

...

})

P1 — Important

5. Bus state : readiness codée en dur

build_bus_stats_state() retourne directement :

f54_minimal_ready: true

f56_signal_ingress_ready: true

f57_bus_layer_audited: true

bus_stats_route: true

bus_bridge_route: true

bus_signal_route: true

Problème

Ce ne sont pas des mesures runtime. Ce sont des assertions statiques.

Correction :

*_declared: true

*_measured: true/false

*_evidence_ref: path/hash/test

6. Bus bridge : is_attached=true codé en dur

build_bus_bridge_state() renvoie :

is_attached: true

```
stats: emitted=0, dropped=0
```

graphiti_status: not_collected

brody_context_status: not_collected

Problème

> is_attached=true alors que les vraies connexions sont not_collected.

Correction :

is_attached_declared: true

is_attached_measured: false/unknown

7. /api/runtime-wiring/source-runtime/status confond “source runtime available” et “hydration available”

Le status expose :

real_readonly_hydration_available: _SOURCE_RUNTIME_AVAILABLE

Mais l’ancienne logique source_runtime_cache dit que les familles peuvent être “available for selection” même si les packs locaux ne sont pas hydratables.

Correction

source_runtime_available

selectable_families

local_pack_hydration_available

real_readonly_hydration_available

séparés.

8. Source runtime preview peut exposer trop de contexte

/api/runtime-wiring/source-runtime/preview renvoie :

hydrated_entries

context_summary_for_brody

source_file_refs

selected_functions

selected_classes

selected_tests

selected_docs

Sans auth visible, c’est une surface sensible.

Correction :

auth obligatoire

compact par défaut

```
debug=false par défaut strict
```

no full hydrated_entries without auth/debug

9. Graphiti proxy avale les erreurs

Dans graphiti.py, fallback query_graphiti_readonly fait :

except Exception:

pass

Problème

Ça masque :

module cassé

schema cassé

Neo4j erreur

Graphiti bridge erreur

Correction :

```
graphiti_backend_error=true
```

error_type

```
source=MODULE_ERROR
```

P2 — Cohérence / wording

10. /api/runtime/freeze-dashboard exécute git via subprocess

Le dashboard runtime appelle :

```
subprocess.run(["git", *args], cwd=ROOT, ...)
```

pour log, status, diff.

C’est readonly, mais ce n’est pas “no execution” au sens strict : c’est une commande locale.

Correction :

```
local_subprocess_read=true
filesystem_write=false
```

et auth obligatoire si exposé.

11. Runtime freeze readiness dépend de rapports/tags, pas de tests exécutés

Le dashboard calcule READY avec :

late_ok

```
covered_phases >= 18
```

report_count

tag_count

Problème

C’est une readiness documentaire, pas une readiness runtime.

Correction :

documentation_readiness

runtime_test_readiness

ci_readiness

séparés.

Synthèse nouvelle couche

NOUVEAUX P0

1. Auth seulement visible sur Brody.

2. Routes /bus hors /api.

3. /bus utilise build_output_envelope faible.

4. graphiti_v20_readonly_client utilise setdefault au lieu de forcer les flags.

NOUVEAUX P1

5. Bus readiness codée en dur.

6. Bus bridge is_attached=true non mesuré.

7. Source runtime confond available/selectable/hydratable.

8. Source runtime preview expose hydrated_entries sans auth visible.

9. Graphiti proxy avale les erreurs.

NOUVEAUX P2

10. Runtime dashboard exécute git subprocess malgré wording readonly.

11. Freeze dashboard = readiness documentaire, pas runtime readiness.

Patch order ajouté

1. Auth globale sur tous les routers sauf health/readiness minimal.

2. Déplacer /bus/* vers /api/bus/*.

3. Patch build_output_envelope.

4. Remplacer setdefault par update forcé dans graphiti_v20_readonly_client.

5. Séparer declared_ready / measured_ready dans bus state.

6. Séparer selectable / hydratable dans source runtime.

7. Masquer hydrated_entries hors debug/auth.

8. Remonter erreurs Graphiti au lieu de pass.

9. Marquer git subprocess comme local_read_command=true.

10. Renommer runtime freeze dashboard :

DOCUMENTATION_FREEZE_DASHBOARD

pas RUNTIME_READY.

Verdict actualisé :

```
CORE_X108 = OK
BRODY/SRL/MEMORY = À DURCIR
API_AUTH = INCOMPLÈTE
READONLY = TROP LARGE / MAL DÉFINI
BUS_LAYER = SURFACE PUBLIQUE À RECADRER
RUNTIME_FREEZE = DOCUMENTAIRE, PAS PREUVE LIVE
```

Audit continué — routes restantes

Nouveaux problèmes remontés :

P0

1. Routes restantes sans auth visible.

2. /api/translation/trace renvoie user_input brut.

3. /api/context/from-message avale l’erreur et peut masquer un backend cassé.

4. Gencoin lit un ledger réel mais masque les erreurs.

5. Monitoring Brody scanne tout brody_memory_readonly via os.walk sans cache ni limite.

P1

6. OS Map expose beaucoup d’inventaire runtime sans auth visible.

7. OS Map / source runtime réutilisent la confusion selectable/hydratable.

8. Runtime wiring preview cache un snapshot permanent sans invalidation.

9. Sigma monitoring est propre côté boundary, mais exposé sans auth visible.

10. Translation route est surtout stub mais annonce PARSED.

P2

11. Plusieurs endpoints utilisent `READY`, `REAL_BACKEND`, `LIVE_*` alors que la preuve est partielle ou silencieuse.

P0 — Critique

1. Routes restantes sans auth visible

Les routes suivantes sont incluses globalement dans main.py :

gencoin

context

translation

os_map

brody_monitoring

sigma_monitoring

runtime_wiring_preview

source_runtime_status

Et require_api_key n’apparaît que dans routes/brody.py.

Donc même les surfaces d’inventaire, de monitoring et de preview runtime semblent exposables sans garde dédiée si l’app est lancée en dev/test.

Correction :

auth globale sur tout sauf health/readiness/status minimal.

2. /api/translation/trace renvoie user_input brut

La route retourne :

"user_input": req.text

Contrairement au bus signal qui sanitize les champs texte avec sanitize_user_facing_text().

Risque

écho brut utilisateur

payload potentiellement sensible

token interdit non nettoyé

Correction :

user_input_hash

user_input_excerpt_sanitized

```
raw_echo=false par défaut
```

3. /api/context/from-message avale l’erreur backend

La route catch toute exception et remplace par :

{"status": "BACKEND_STUB", "packet_id": "stub"}

Problème

Un vrai crash du builder contexte devient un stub silencieux.

Correction :

```
context_backend_error=true
```

error_type

```
source=MODULE_ERROR
```

4. Gencoin masque les erreurs de ledger

/api/gencoin tente read_ledger(), puis :

except Exception:

pass

Puis renvoie LIVE_EMPTY_REGISTRY si rien n’est lu.

Problème

Un ledger cassé peut apparaître comme ledger vide.

Correction :

```
LIVE_EMPTY_REGISTRY = aucun ledger
LEDGER_READ_ERROR = backend cassé
```

5. Monitoring Brody scanne tout brody_memory_readonly

brody_monitoring.py fait :

for dp, dn, fn in sorted(os.walk(BRODY_ROOT)):

puis lit/parse chaque .py.

Problème

pas de max_files

pas de timeout

pas de cache

pas de skip backups

exposition structure complète du repo

Correction :

auth obligatoire

cache TTL

max_files

skip *.bak*

compact default

debug only for entries détaillées

P1 — Important

6. OS Map expose inventaire runtime détaillé

/api/runtime-wiring/os-map/query renvoie :

selected_modules

selected_adapters

selected_routes

selected_functions

selected_classes

selected_tests

selected_docs

hydration_plan

source_file_refs

Sans auth visible, c’est une surface d’inventaire très riche.

Correction :

auth obligatoire

```
compact=true par défaut
```

debug/auth pour selected_functions/classes/tests/docs

7. OS Map utilise x108_decision pour du context-only

OS Map retourne :

```
x108_decision = selected_path.get("x108_decision", "ALLOW_CONTEXT_ONLY")
```

puis peut remplacer par :

"BLOCK_OR_HOLD_CONTEXT_ONLY"

Ce n’est pas une décision d’action X108 ; c’est une décision de routage contexte.

Correction :

x108_context_route_gate

```
≠ x108_action_decision
```

8. Runtime wiring preview cache sans invalidation

runtime_wiring_preview.py garde :

_PREVIEW_CACHE

et ne le recharge plus après premier appel.

Problème

Si le registry change, la preview reste stale.

Correction :

cache_ttl_seconds

cache_hash

```
force_refresh=false
preview_stale=false/true
```

9. Runtime wiring preview répète la confusion hydratation

Il pose :

```
real_readonly_hydration_available = _SOURCE_RUNTIME_AVAILABLE
```

Mais _SOURCE_RUNTIME_AVAILABLE signifie seulement que le module est importable, pas que les packs sont localement hydratables.

Correction :

source_runtime_module_available

selectable_families_count

local_pack_hydration_available

real_hydration_available

10. Sigma monitoring est propre mais sensible

Sigma monitoring est bien structuré côté boundary :

```
readonly=true
```

```
advisory_only=true
allowed_to_decide=false
emits_act=false
emits_verdict=false
neo4j_write=false
graphiti_write=false
memory_write=false
```

Mais il expose domaines et évaluations Sigma :

domains

evaluate

bank

trading

ecom

gps-defense-aviation

Correction :

auth obligatoire si démo non locale

P2 — Cohérence

11. /api/translation/trace annonce PARSED alors que c’est un stub

La route renvoie :

os_trad_status: PARSED

mode: BACKEND_STUB

Correction :

os_trad_status: STUB_PARSED

parsed_by: BACKEND_STUB

real_parser_used: false

12. OS Trad routes sans préfixe router

os_trad_ir_reverse.py crée :

```
router = APIRouter(tags=["os-trad-ir-reverse"])
```

puis les routes contiennent directement /api/os-trad/translate, /api/ir/candidate, /api/os-reverse/project.

Ce n’est pas cassé, mais moins propre que :

```
APIRouter(prefix="/api")
```

Correction :

standardiser les prefixes route.

Synthèse nouvelle

NOUVEAUX P0

1. Auth manquante sur routes restantes.

2. Translation renvoie user_input brut.

3. Context route masque erreurs en BACKEND_STUB.

4. Gencoin masque erreurs ledger en LIVE_EMPTY_REGISTRY.

5. Brody monitoring scanne tout brody_memory_readonly sans limite/cache.

NOUVEAUX P1

6. OS Map expose inventaire runtime riche.

7. OS Map mélange x108_context_gate et x108_action_decision.

8. Runtime wiring preview cache sans invalidation.

9. Runtime wiring preview confond module importable et hydratation réelle.

10. Sigma monitoring propre mais sensible sans auth visible.

NOUVEAUX P2

11. Translation annonce PARSED en mode stub.

12. OS Trad route prefix non standardisé.

Patch order ajouté

1. Auth globale.

2. Sanitize/hasher user_input dans translation.

3. Remonter erreurs context/gencoin au lieu de stub silencieux.

4. Limiter/cacher brody-cli-registry.

5. Compact default pour OS Map et source runtime.

6. Renommer x108_decision en x108_context_route_gate sur OS Map/source runtime.

7. Ajouter TTL / force_refresh au runtime preview.

8. Séparer module_available / hydration_available.

9. Renommer STUB_PARSED.

10. Standardiser prefixes API.

Verdict actualisé :

```
KX108_CORE = OK
API_SURFACE = TROP OUVERTE
MONITORING = TROP RICHE SANS AUTH
STUBS = MASQUENT DES ERREURS
READONLY = ENCORE TROP AMBIGU
PUBLIC_READY = NON TANT QUE AUTH + BOUNDARY + WRITE FLAGS NON PATCHÉS
```

Audit continué — scripts “readonly” qui écrivent / réouvrent / intègrent des sources

Nouveaux problèmes remontés :

P0

1. Plusieurs scripts brody_memory_readonly écrivent massivement des fichiers malgré le nom readonly.

2. Session reopen lit Neo4j live et dépend d’un ancien manual apply de 29 records.

3. Memory scheduler est readonly en décision, mais écrit plan/prompt/summary/report.

4. World source intake extrait/cop ie des contenus locaux complets dans normalized_md.

5. Les compteurs 29 restent partout dans la chaîne mémoire récente.

P0 — Critique

1. session_reopen_loop_readonly n’est pas “readonly filesystem”

Le module déclare :

```
readonly: true
```

graphiti_query_read: true

```
neo4j_write_executed: false
```

memory_intake: false

Mais il écrit plusieurs fichiers :

BRODY_SESSION_REOPEN_EXACT_RECORDS.json

BRODY_SESSION_REOPEN_QUERY_RECORDS.jsonl

BRODY_SESSION_REOPEN_QUERY_RECORDS.json

BRODY_SESSION_REOPEN_MISSING_IDS.json

BRODY_SESSION_REOPEN_CONTEXT_PACKET.json

BRODY_SESSION_REOPEN_PROMPT_CONTEXT.md

BRODY_SESSION_REOPEN_LOOP_SUMMARY.json

BRODY_SESSION_REOPEN_LOOP_REPORT.md

via write_text() / write_jsonl().

Correction

```
readonly=true → readonly_decision=true
filesystem_write=true
artifact_generation=true
neo4j_read=true
neo4j_write=false
```

2. session_reopen_loop_readonly dépend d’un apply Graphiti manuel antérieur

Il exige :

CURRENT_BRODY_GRAPHITI_GUARDED_MANUAL_APPLY_EXECUTION_MEMORY_ONLY.txt

Puis exige que les IDs appliqués existent dans Neo4j.

Donc ce module est “query read only”, mais il dépend d’un état issu d’un write manuel précédent.

Correction

```
depends_on_prior_manual_graphiti_apply=true
manual_write_dependency=true
not_pure_readonly=true
```

3. Hardcode 29 encore présent dans session reopen

Le module exige :

```
manual_apply_write_executed_count == 29
exact_found_count == 29
len(expected_ids) == 29
```

Et le pass final exige encore :

```
len(expected_ids) == 29
len(found_ids) == 29
prefix_hits >= 29
```

Correction

```
expected_ids_count = len(applied_ids)
expected_found_count = expected_ids_count
```

4. memory_scheduler_readonly écrit des artefacts

Le scheduler annonce :

```
scheduler_registers_background_task=false
automatic_execution=false
cron_registered=false
requires_human_trigger=true
```

Mais il écrit :

BRODY_MEMORY_SCHEDULER_PLAN_READONLY.jsonl

BRODY_MEMORY_SCHEDULER_PLAN_READONLY.json

BRODY_MEMORY_SCHEDULER_NEXT_SESSION_PROMPT.md

BRODY_MEMORY_SCHEDULER_READONLY_SUMMARY.json

BRODY_MEMORY_SCHEDULER_READONLY_REPORT.md

Correction

```
filesystem_write=true
artifact_generation=true
scheduler_execution=false
background_task_registered=false
```

Readonly ici signifie “pas d’exécution automatique”, pas “pas d’écriture disque”.

5. memory_scheduler_readonly hardcode encore 29

Il vérifie :

```
manual_apply_write_executed_count == 29
smoke exact_found_count == 29
reopen exact_found_count == 29
freeze manual_apply_write_executed_count == 29
```

Correction :

derive_expected_count_from_applied_ids

ou from close_summary.expected_applied_id_count

P1 — Important

6. world_source_intake_readonly copie / normalise du contenu local

Le module prend un --source, traverse les fichiers supportés, extrait texte PDF/DOCX/XLSX/CSV/etc., puis écrit des .md normalisés dans out_root.

Il écrit ensuite :

normalized_md/*.md

BRODY_WORLD_SOURCE_INTAKE_MANIFEST.json

CURRENT_BRODY_WORLD_SOURCE_INTAKE_READONLY.txt

Problème

Le nom readonly est trompeur : il ne décide pas, mais il fait de l’ingestion/extraction et écrit des artefacts.

Correction :

```
readonly_decision=true
source_ingestion=true
filesystem_write=true
normalized_artifact_write=true
memory_decision=false
graphiti_write=false
```

7. world_source_intake_readonly peut aspirer beaucoup de données

Formats supportés :

.txt, .md, .json, .jsonl, .csv, .pdf, .docx, .xlsx, .xls

Le script extrait jusqu’à :

safe_text[:800000]

par fichier dans le Markdown normalisé.

Risque

gros volumes

données sensibles locales

documents privés copiés dans artefacts

risque de commit accidentel

Correction :

max_files obligatoire

max_chars_per_file plus bas

exclude _local_audits/.env/secrets/node_modules/.venv

dry_run_inventory mode par défaut

8. Exclusion Windows-only dans world_source_intake

Le filtrage exclut :

"\\node_modules\\"

"\\.venv\\"

"__pycache__\\"

Sur Linux/macOS, les chemins utilisent /, donc ces exclusions peuvent ne pas marcher.

Correction :

```
parts = set(p.parts)
```

if {"node_modules", ".venv", "__pycache__"} & parts:

skip

9. world_source_intake écrit les chemins originaux complets

Le manifest conserve :

source_original_path

path

run_dir

Risque

chemins locaux personnels

structure machine

fuite de noms de dossiers

Correction :

store relative path if inside allowed root

redact home/user path

source_original_path_hash

P2 — Cohérence SRL

10. Le vocabulaire “readonly” doit être scindé

Dans ces modules, readonly signifie plusieurs choses différentes :

pas de décision

pas d’ACT

pas d’écriture Graphiti

pas d’écriture Neo4j

pas d’écriture mémoire canonique

pas d’exécution automatique

mais écriture d’artefacts locaux autorisée

Donc il faut remplacer le flag unique par une boundary complète :

decision_readonly: true

act_readonly: true

kernel_readonly: true

```
graphiti_write: false
neo4j_write: false
```

canonical_memory_write: false

local_artifact_write: true

filesystem_write: true

source_ingestion: true/false

prior_manual_write_dependency: true/false

Synthèse nouvelle

NOUVEAUX P0

1. session_reopen_loop_readonly écrit 8 artefacts.

2. session_reopen_loop_readonly dépend d’un manual apply Graphiti antérieur.

3. session_reopen_loop hardcode 29.

4. memory_scheduler_readonly écrit 5 artefacts.

5. memory_scheduler hardcode 29.

NOUVEAUX P1

6. world_source_intake_readonly extrait et écrit des documents locaux.

7. world_source_intake peut aspirer 800k chars par fichier.

8. exclusions node_modules/.venv Windows-only.

9. chemins originaux complets stockés dans manifests.

NOUVEAU P2

10. readonly doit être scindé en decision_readonly / filesystem_write / artifact_write / graphiti_write / neo4j_write.

Patch order ajouté

1. Renommer boundary globale :

```
readonly → decision_readonly / no_act / no_kernel_mutation.
```

2. Ajouter partout :

filesystem_write

local_artifact_write

canonical_memory_write

```
graphiti_write
neo4j_write
```

prior_manual_write_dependency

3. Remplacer tous les hardcodes 29 par expected_count dynamique.

4. World source intake :

max_files

max_chars_per_file

path redaction

POSIX-compatible exclusions

dry-run inventory default.

5. Session reopen :

```
marquer prior_manual_write_dependency=true
marquer neo4j_read=true
marquer local_artifact_write=true.
```

6. Scheduler :

```
marquer scheduler_execution=false
mais local_artifact_write=true.
```

Verdict actualisé :

```
KX108_CORE = OK
BRODY_MEMORY_READONLY = NOM TROP LARGE
READONLY_DECISION = OUI
READONLY_FILESYSTEM = NON
READONLY_GRAPHITI = PAS TOUJOURS, SI MANUAL APPLY HORS CHEMIN
SRL_CANON = À BLOQUER TANT QUE LES FLAGS NE SONT PAS SCINDÉS
```

Audit continué — couche logs / middleware / bridges / workflow

Nouveaux problèmes remontés :

P0

1. AuditMiddleware écrit en audit_logs tout en boundary memory_write=false.

2. BrodyBridge écrit dans audit/brody_llm_pseudo_output.jsonl mais déclare memory_write=false.

3. Plusieurs outils CLI “readonly” écrivent si --out est fourni.

4. Le terme readonly est définitivement trop ambigu : il cache des écritures locales d’audit/artifact.

P0 — Critique

1. AuditMiddleware écrit localement mais se déclare readonly

audit_middleware.py annonce :

Persists to audit_logs/

```
decision_authority = KX108_ONLY
```

Puis crée directement :

```
_LOG_DIR = Path("audit_logs")
_LOG_DIR.mkdir(parents=True, exist_ok=True)
```

Et écrit chaque entrée dans :

audit_logs/http_audit_<date>.jsonl

via :

```
with open(_log_path(), "a", encoding="utf-8") as fh:
```

fh.write(line)

Mais sa boundary dit seulement :

```
readonly=true
memory_write=false
```

Correction

```
memory_write=false
canonical_memory_write=false
local_audit_write=true
filesystem_write=true
audit_log_write=true
```

2. AuditMiddleware log les query params

L’entrée audit inclut :

"query": str(request.query_params) or ""

Risque

Si une clé, token, URL sensible ou paramètre privé passe dans la query, il est écrit dans audit_logs.

Correction :

```
query_params_redacted=true
```

redact token/key/password/secret/auth/bearer

ou query_hash seulement

3. BrodyBridge écrit un pseudo-output local

Le bridge déclare :

No ACT. No kernel mutation. No memory write. KX108_ONLY.

Mais si env.topic == "GENCOIN_TRAD_CONTEXT", il écrit dans :

audit/brody_llm_pseudo_output.jsonl

avec :

```
with open(_BRODY_OUTPUT_LOG, "a", encoding="utf-8") as f:
```

f.write(...)

Correction

```
memory_write=false
brody_memory_write=false
local_audit_write=true
pseudo_output_log_write=true
filesystem_write=true
```

4. BrodyBridge print des payloads complets

Le bridge fait :

print(f"[BRODY_DEBUG] RE?U: {env.topic} | Payload: {env.payload}")

Risque

console leak

payload sensible

logs terminal non maîtrisés

Correction :

```
debug_print=false par défaut
```

payload_hash

payload_keys seulement

P1 — Important

5. workflow_governance_readonly/cli.py écrit si --out

Le CLI prend :

--out Optional output JSON path

puis écrit :

```
Path(args.out).write_text(text, encoding="utf-8")
```

C’est normal pour un CLI, mais pas strictement filesystem readonly.

Correction :

```
readonly_decision=true
filesystem_write=true if --out
artifact_write=true if --out
```

6. workflow_governance_readonly/utils.py contient un helper d’écriture

write_json() crée le dossier puis écrit :

```
p.parent.mkdir(parents=True, exist_ok=True)
```

p.write_text(...)

Correction : ajouter boundary ou nommer clairement :

write_json_artifact()

pas “readonly” implicite.

7. BRODY_ROOT scanné en relatif

brody_monitoring.py définit :

```
BRODY_ROOT = "periphery/brody_memory_readonly"
```

puis fait os.walk(BRODY_ROOT).

Problème

Selon le working directory, le scan peut échouer ou scanner le mauvais chemin.

Correction :

```
ROOT = Path(__file__).resolve().parents[3]
BRODY_ROOT = ROOT / "periphery" / "brody_memory_readonly"
```

P2 — Standardisation

8. Routes avec préfixes non homogènes

On a au moins :

/bus/*

/api/*

/api/runtime-wiring/*

/api/periphery/*

/api/os-trad/*

/api/ir/*

/api/os-reverse/*

```
bus.py n’a pas de prefix /api. os_trad_ir_reverse.py met APIRouter(tags=...) sans prefix, puis encode les chemins complets dans chaque décorateur.
```

Correction :

tout sous /api

router prefix standard

public routes explicitement listées

Nouvelle synthèse

NOUVEAUX P0

1. AuditMiddleware écrit dans audit_logs malgré readonly/memory_write=false.

2. AuditMiddleware log les query params sans redaction visible.

3. BrodyBridge écrit pseudo-output local malgré memory_write=false.

4. BrodyBridge print payload complet en debug.

NOUVEAUX P1

5. Workflow governance CLI écrit si --out.

6. Utils readonly contient write_json.

7. Brody monitoring scanne un chemin relatif.

NOUVEAU P2

8. Routes API préfixées de manière non homogène.

Patch order ajouté

1. Ajouter flags globaux :

filesystem_write

local_audit_write

artifact_write

canonical_memory_write

query_params_redacted

2. AuditMiddleware :

redact query params

```
marquer audit_log_write=true
```

ne pas appeler readonly strict.

3. BrodyBridge :

supprimer print payload brut

log hash/keys seulement

```
marquer pseudo_output_log_write=true.
```

4. Workflow CLI :

renommer output write en artifact_write.

documenter --out comme écriture locale.

5. Monitoring :

chemins absolus repo-root

max_files / cache / skip backups.

6. Routes :

standardiser tout sous /api

auth globale.

Verdict mis à jour :

```
KX108_CORE = OK
```

PROBLÈME CENTRAL :

le repo utilise “readonly” pour dire “ne décide pas / ne mute pas le kernel”,

mais pas pour dire “n’écrit jamais sur disque”.

Donc avant SRL canonique :

il faut remplacer readonly par une boundary granulaire.

Boundary cible :

decision_readonly: true

act_readonly: true

```
kernel_mutation: false
```

x108_mutation: false

```
graphiti_write: false
neo4j_write: false
```

canonical_memory_write: false

filesystem_write: true/false

local_audit_write: true/false

artifact_write: true/false

pseudo_output_log_write: true/false

source_ingestion: true/false

prior_manual_write_dependency: true/false

auth_required: true

query_params_redacted: true

Audit continué — couche serveur / worldcalls / OS3 / context consumer

Nouveaux problèmes remontés :

P0

1. Rate limit désactivé par défaut en dev.

2. Docs/OpenAPI accessibles en dev.

3. Worldcalls expose audit/world_action_bus.jsonl et sovereign_tickets.jsonl sans auth visible.

4. Context packet consumer peut lire des fichiers locaux depuis un path fourni dans packet JSON.

5. Context packet consumer expose path/source_ref/excerpt dans la réponse.

P0 — Critique

1. Rate-limit désactivé par défaut hors prod

Dans main.py, le rate-limit est actif seulement si :

_IS_PROD

```
or OBSIDIA_RATE_LIMIT_ENABLED=true
```

En dev/test, il ne s’applique pas. Or l’auth aussi est désactivée par défaut en dev/test dans auth.py, vu plus haut.

Risque

```
APP_ENV=dev lancé publiquement
```

```
→ routes sans auth
→ pas de rate limit
→ docs ouvertes
→ inventaires exposés
```

Correction :

```
APP_ENV=prod obligatoire pour toute exposition réseau
OBSIDIA_RATE_LIMIT_ENABLED=true par défaut sauf explicitement local
```

2. Docs/OpenAPI accessibles en dev

main.py active :

/docs

/redoc

/openapi.json

hors prod.

Normal pour local. Dangereux si le serveur dev est exposé.

Correction :

```
docs_enabled=false par défaut
OBSIDIA_DOCS_ENABLED=true seulement local
```

3. Worldcalls expose des journaux d’audit réels

worldcalls.py lit :

audit/sovereign_tickets.jsonl

audit/world_action_bus.jsonl

Puis expose les 20 derniers world action bus et 50 derniers sovereign tickets.

Problème

Même si c’est readonly, ça expose :

event_id

action_id

intent

domain

world_call_class

action_risk_class

block_reason

ticket_id

os3_ticket_id

x108_gate

hash

issued_at

Sans auth visible, c’est trop riche.

Correction :

```
auth_required=true
compact=true par défaut
```

hash-only mode public

debug/audit role pour détails

4. Context packet consumer lit des fichiers locaux depuis le packet

context_packet_consumer_readonly prend un packet_json, charge les items, puis si l’excerpt est vide il lit :

read_local_excerpt(item.get("path"))

read_local_excerpt() accepte .md, .txt, .json, .jsonl, .py, .ps1, .csv, .lean, .yaml, .toml, .html, .css, .js, .ts, .docx.

Risque

packet JSON contrôlé / pollué

```
→ path vers fichier local sensible
→ excerpt lu
→ réponse imprime contenu local
```

Correction :

path allowlist obligatoire

path must be inside repo_root / allowed context roots

no absolute path

no parent traversal

max_chars strict

```
local_file_read=true déclaré
```

5. Context packet consumer expose chemins + extraits

La réponse inclut :

path

source_ref

excerpt

selected_items

response_md

Correction

```
redact_paths=true par défaut
excerpt_public=false sauf debug/auth
```

path_hash + relative_path seulement

P1 — Important

6. OS3 routes sont des viewers documentaires, pas du replay réel

/api/os3/tickets liste les .md dans docs/runtime.

/api/os3/replay/{ticket_id} renvoie :

replay_status: NOT_RUN

Problème

C’est un viewer documentaire, pas un replay engine.

Correction wording :

OS3_DOCUMENTARY_VIEWER

not OS3_REPLAY_RUNTIME

7. Worldcalls masque les erreurs de lecture JSONL

_read_jsonl() retourne [] sur exception.

Donc fichier absent, JSON cassé, permission denied ou audit vide donnent tous le même résultat.

Correction :

read_status: OK / FILE_MISSING / PARSE_ERROR / PERMISSION_ERROR

8. Root endpoint trop rassurant

/ retourne :

mode: readonly_dryrun

```
decision_authority: KX108_ONLY
```

Mais on a identifié plusieurs écritures locales d’audit/artifacts.

Correction :

mode: decision_readonly_dryrun

filesystem_writes_possible: true

P2 — Cohérence boundary

9. CORS autorise tous les headers

CORS :

```
allow_headers=["*"]
```

avec origins localhost par défaut.

Acceptable localement, mais à verrouiller en prod :

```
allow_headers=["Authorization", "Content-Type"]
```

10. Middleware rate-limit stocke en mémoire sans purge globale

_hits est un defaultdict(list) par IP. Il purge seulement les timestamps pour l’IP courante.

Risque faible, mais dette :

```
beaucoup d’IPs uniques → croissance mémoire
```

Correction :

global purge périodique

max_ip_entries

Synthèse nouvelle

NOUVEAUX P0

1. Rate-limit off en dev.

2. Docs/OpenAPI ouvertes en dev.

3. Worldcalls expose audit logs réels sans auth visible.

4. Context packet consumer lit des chemins locaux fournis par packet.

5. Context packet consumer expose paths/excerpts.

NOUVEAUX P1

6. OS3 est documentaire, pas replay réel.

7. Worldcalls masque erreurs JSONL.

8. Root endpoint dit readonly_dryrun alors que filesystem writes existent.

NOUVEAUX P2

9. CORS allow_headers="*" à durcir en prod.

10. Rate-limit memory store peut grossir avec IPs uniques.

Patch order ajouté

1. Auth globale + prod guard obligatoire.

2. Rename global mode:

```
readonly_dryrun → decision_readonly_dryrun_with_local_audit_writes.
```

3. Context packet consumer:

allowed_roots

no absolute paths

no parent traversal

redact path

excerpt only under debug/auth.

4. Worldcalls:

compact mode default

auth required

read_status explicit.

5. OS3:

rename routes/status as documentary viewer until replay is real.

6. CORS:

prod headers strict.

7. Rate-limit:

enable outside local explicit

add max IP cache.

Verdict actualisé :

```
CORE_X108 = OK
```

SURFACE À CORRIGER AVANT PUBLIC :

AUTH

READONLY WORDING

FILESYSTEM WRITE FLAGS

LOCAL PATH READS

AUDIT LOG EXPOSURE

CONTEXT EXCERPT EXPOSURE

SRL peut avancer seulement après séparation stricte :

```
decision_readonly ≠ filesystem_readonly ≠ local_audit_write.
```

Audit continué — scripts Sigma / packs de charge / outils CLI

Nouveaux problèmes remontés :

P0

1. Les packs Sigma exécutent des subprocess massifs.

2. Ils génèrent énormément d’artefacts/payloads.

3. Ils peuvent écrire 1000 / 10000 / 100000 payloads JSON locaux.

4. Ils ne portent pas de boundary explicite filesystem_write/artifact_write/subprocess_execute.

5. Les erreurs capturent stdout/stderr complets dans les rapports.

P0 — Critique

1. run_bank_scale_pack.py exécute le pipeline par subprocess

Le script lance :

subprocess.run(

[sys.executable, str(RUN_PIPELINE), "bank", str(payload_path)],

...

)

Ce n’est pas une faille en soi, mais ce n’est pas “readonly” au sens strict d’exécution.

Correction boundary

subprocess_execute: true

pipeline_execute: true

world_action: false

```
kernel_mutation: false
```

artifact_write: true

payload_generation: true

2. run_bank_scale_pack.py génère jusqu’à 100000 payloads

Le script accepte :

```
--size choices=[1000, 10000, 100000]
```

Puis écrit chaque payload dans :

artifacts/p2_bank_scale/<size>_cases_<workers>_workers/_payloads/

Chaque cas est écrit via :

payload_path.write_text(...)

Risque

gros volume disque

pollution repo/artifacts

commit accidentel

temps d’exécution long

Correction :

default --size 1000 OK

10000/100000 exigent --confirm-large-run

artifacts/ dans .gitignore

max_disk_estimate avant run

3. run_bank_scale_pack.py écrit rapports complets

Le script écrit :

bank_scale_report.json

bank_scale_summary.json

bank_scale_report.csv

Correction boundary :

```
report_write=true
artifact_write=true
filesystem_write=true
```

4. run_bank_replay_pack.py exécute deux fois le pipeline par cas

Le replay fait :

```
first = run_pipeline_once(case, payload_dir)
second = run_pipeline_once(case, payload_dir)
```

Et run_pipeline_once() appelle aussi subprocess.run(...).

Donc pour 100000 cas :

200000 exécutions pipeline potentielles

Correction :

large_run_guard

```
subprocess_execute=true
pipeline_execute_count_estimate=size*2
```

5. run_bank_replay_pack.py écrit aussi payloads + rapports

Il écrit les payloads :

payload_path.write_text(...)

Puis :

bank_replay_report.json

bank_replay_report.csv

bank_replay_summary.json

Même correction :

```
artifact_write=true
payload_generation=true
```

```
subprocess_execute=true
```

P1 — Important

6. stdout/stderr complets capturés dans les rapports

En cas d’erreur, run_bank_scale_pack.py stocke :

stdout

stderr

dans l’objet error.

run_bank_replay_pack.py fait pareil.

Risque

trace locale

chemins machine

variables ou outputs sensibles

rapport commit accidentel

Correction :

stderr_excerpt

stdout_excerpt

```
max_chars=2000
```

redact paths/tokens

full logs seulement en _local_audits non commit

7. Les packs Sigma ne déclarent pas leur mode de preuve

Ces scripts produisent des preuves utiles, mais ils ne distinguent pas :

simulation generated payloads

subprocess execution

runtime pipeline execution

artifact report

no external egress

no real bank action

Correction : ajouter un bloc summary commun :

{

"decision_authority": "KX108_ONLY",

"subprocess_execute": true,

"pipeline_execute": true,

"external_egress": false,

"real_bank_action": false,

"artifact_write": true,

"filesystem_write": true,

"payload_generation": true

}

P2 — Cohérence repo

8. scripts/ et sigma/tools/ sont hors logique API mais influencent la preuve

La CI / manifest / docs doivent distinguer :

runtime API

proof scripts

audit generators

load-test generators

manual tools

legacy archive

Actuellement tout se mélange dans le discours “readonly”.

Correction :

TOOL_CLASS:

- API_ROUTE

- READONLY_VIEWER

- LOCAL_ARTIFACT_GENERATOR
- SUBPROCESS_TEST_RUNNER
- LIVE_SERVICE_READER
- MANUAL_WRITE_APPLIER
- LEGACY_ARCHIVE

Nouvelle synthèse

NOUVEAUX P0

1. Sigma scale pack exécute subprocess.

2. Sigma scale pack génère jusqu’à 100000 payloads.

3. Sigma scale pack écrit rapports JSON/CSV.

4. Sigma replay pack exécute 2 subprocess par cas.

5. Sigma replay pack écrit payloads + rapports.

NOUVEAUX P1

6. stdout/stderr complets stockés dans rapports d’erreur.

7. Pas de boundary explicite subprocess_execute/artifact_write.

NOUVEAU P2

8. scripts/tools doivent être classés séparément de runtime API.

Patch order ajouté

1. Ajouter boundary commune aux outils Sigma :

subprocess_execute

pipeline_execute

payload_generation

artifact_write

filesystem_write

```
external_egress=false
real_bank_action=false
```

2. Ajouter --confirm-large-run pour 10000/100000.

3. Redacter stdout/stderr dans rapports publics.

4. Mettre artifacts/ sous règle stricte :

non public par défaut

manifest seulement si freeze volontaire.

5. Créer TOOL_CLASS_MATRIX :

API_ROUTE / LOCAL_ARTIFACT_GENERATOR / SUBPROCESS_TEST_RUNNER / LIVE_READER / MANUAL_WRITE_APPLIER.

Verdict actualisé :

```
KX108_CORE = OK
```

MAIS :

le repo mélange encore preuves, outils de génération, viewers, API et scripts de charge sous le même vocabulaire readonly.

Pour audit RSSI :

il faut une matrice de classes d’outils avant SRL canonique.

Audit continué — .graph-memory / graph scan / mémoire candidate

Nouveaux problèmes remontés :

P0

1. Les scripts .graph-memory exécutent git via subprocess.

2. Ils écrivent des rapports locaux malgré “dry-run”.

3. Le scan dépend de Path.cwd(), donc fragile selon le dossier d’exécution.

4. Les deny patterns contiennent des règles trop larges : *root*, *seal*, *hash*, *freeze*.

5. Les scripts écrivent dans .graph-memory/reports sans boundary filesystem_write.

P0 — Critique

1. .graph-memory/scripts/dry_run_scan.py exécute git ls-files

Le script appelle :

```
subprocess.check_output(["git", "ls-files"], cwd=ROOT, ...)
```

Donc ce n’est pas seulement un scan mémoire abstrait : c’est une commande Git locale.

Correction boundary :

subprocess_execute: true

git_read_command: true

filesystem_write: true

artifact_write: true

external_egress: false

2. .graph-memory écrit des rapports locaux

Le script écrit dans :

.graph-memory/reports/

avec :

REPORTS.mkdir(...)

(REPORTS / name).write_text(...)

Il produit :

dry_run_candidates.md

denylist_hits.md

conditional_candidates.md

token_budget_estimate.md

Même problème que le reste du repo :

```
dry-run ≠ no filesystem write
```

Correction :

```
dry_run=true
artifact_write=true
filesystem_write=true
memory_write=false
graphiti_write=false
```

3. Path.cwd() rend le scan fragile

Le script définit :

```
ROOT = Path.cwd()
```

Donc si le script est lancé depuis un mauvais dossier, il scanne le mauvais repo ou échoue silencieusement selon le contexte.

Correction :

```
ROOT = Path(__file__).resolve().parents[2]
```

ou argument obligatoire :

--repo-root

4. Deny patterns trop larges

La denylist contient :

root

seal

hash

anchor

freeze

Risque :

faux positifs massifs

documents légitimes exclus

audit mémoire amputé

Exemple : un fichier de doc contenant freeze dans son nom peut être exclu même s'il devrait être visible dans un audit documentaire.

Correction :

remplacer glob large par chemins explicites

ajouter deny_reason

séparer security_deny vs noise_deny

5. Phase0 strict corrige docs/*.md, mais garde les mêmes limites

Le script strict retire docs/*.md des allowed candidates, ce qui réduit l'exposition documentaire.

Mais il garde :

```
ROOT = Path.cwd()
```

subprocess.check_output(["git", "ls-files"])

write_report(...)

Donc le mode strict est meilleur côté sélection, mais pas côté boundary.

P1 — Important

6. Conditional allow peut faire remonter du code sensible

Les scripts autorisent conditionnellement :

sigma/*.py

sigma/tests/*.py

connectors/**/*.py

qa/**/*.py

.github/workflows/*.yml

Ce n'est pas mauvais, mais ça doit être marqué :

```
requires_human_approval=true
conditional_memory_candidate=true
not_indexed_by_default=true
```

Le script le dit en texte dans le rapport, mais pas comme champ machine exploitable.

Correction :

{

"conditional_phase": true,

"human_approval_required": true,

"indexed_by_default": false

}

7. .graph-memory exclut artifacts/**, mais d’autres scripts écrivent beaucoup dans artifacts/

La denylist exclut artifacts/**.

Mais les outils Sigma écrivent massivement dans artifacts/p2_bank_scale et artifacts/p2_bank_replay.

Donc :

```
artifacts = hors graph-memory
```

mais pas forcément hors repo local

Correction :

artifacts/ doit être explicitement non commit

et non memory-indexed

Nouvelle synthèse

NOUVEAUX P0

1. .graph-memory exécute git via subprocess.

2. .graph-memory écrit des rapports locaux.

3. .graph-memory dépend de Path.cwd().

4. denylist trop large : *root*, *seal*, *hash*, *anchor*, *freeze*.

5. phase0_strict garde les mêmes problèmes boundary.

NOUVEAUX P1

6. conditional candidates doivent avoir un champ machine human_approval_required.

7. artifacts/ est bien exclu de graph-memory, mais reste un risque local/commit.

Patch order ajouté

1. Ajouter boundary .graph-memory :

```
subprocess_execute=true
git_read_command=true
artifact_write=true
filesystem_write=true
external_egress=false
```

2. Remplacer Path.cwd() par repo_root explicite.

3. Séparer denylist :

SECURITY_DENY

PROOF_DENY

NOISE_DENY

ARTIFACT_DENY

4. Remplacer globs trop larges par chemins précis.

5. Ajouter JSON report machine :

human_approval_required

indexed_by_default

conditional_phase

deny_reason_type

6. Vérifier .gitignore :

artifacts/

.graph-memory/reports/

_local_audits/

audit_logs/

Verdict actualisé :

```
GRAPH_MEMORY = BONNE INTENTION
```

MAIS PAS ENCORE AUDIT-BOUNDARY PROPRE

Elle est dry-run côté indexation,

mais pas readonly côté filesystem,

et sa denylist peut exclure trop large.

Conclusion cumulée :

Le problème transversal est maintenant clair :

readonly est utilisé pour 5 réalités différentes :

1. pas de décision

2. pas d’ACT

3. pas de mutation kernel

4. pas d’écriture Graphiti/Neo4j

5. parfois pas d’écriture mémoire canonique

Mais il ne couvre pas :

- filesystem writes
- artifact writes
- audit log writes
- subprocess execution
- local file reads
- prior manual write dependencies

Boundary canonique à imposer partout avant SRL :

decision_readonly: true

act_readonly: true

```
kernel_mutation: false
```

x108_mutation: false

```
graphiti_write: false
neo4j_write: false
```

canonical_memory_write: false

filesystem_write: true/false

artifact_write: true/false

local_audit_write: true/false

subprocess_execute: true/false

git_read_command: true/false

local_file_read: true/false

source_ingestion: true/false

prior_manual_write_dependency: true/false

auth_required: true/false

external_egress: false

Statut actuel :

```
KX108_CORE = OK
PERIPHERY = AUDITABLE MAIS PAS ENCORE CLEAN
GRAPH_MEMORY = À RECADRER
SRL_CANON = À BLOQUER TANT QUE BOUNDARY GRANULAIRE NON POSÉE
```

Audit continué — dépendances réseau / connecteurs live

Nouveaux problèmes remontés :

P0

1. requirements inclut requests/httpx, mais le discours “no external egress” n’est pas toujours vrai.

2. connectors/trading_live.py utilise ccxt + Binance live.

3. connectors/trading_live.py boucle infinie et POST vers API locale.

4. aviation_robo.py et bank_normal_flow.py bouclent infiniment et POST vers API.

5. Les connecteurs ne portent pas de boundary explicite external_market_read/network_post/infinite_loop.

P0 — Critique

1. requirements.txt inclut des clients réseau

requirements.txt inclut :

```
requests>=2.31.0
httpx>=0.27.0
```

Ce n’est pas un problème seul. Le problème est que plusieurs modules se présentent comme readonly/dry-run alors que certains connecteurs peuvent faire du réseau.

Correction boundary globale :

network_client_available: true

external_egress_possible: true

external_egress_default: false

2. connectors/trading_live.py lit Binance via ccxt

Le connecteur trading importe requests, puis dans stream_to_kernel() importe ccxt, crée :

```
exchange = ccxt.binance()
```

et appelle :

```
ticker = exchange.fetch_ticker(symbol)
```

Donc ce module fait un external market read réel si exécuté.

Correction boundary :

external_market_read: true

external_network_egress: true

exchange: binance

real_trade_execution: false

3. trading_live.py boucle infiniment

Le connecteur a :

while True:

...

time.sleep(2)

Il POST ensuite vers :

/api/periphery/monitoring/adapters/trading

Correction :

--once par défaut

--loop explicit

--max-events

--interval

4. aviation_robo.py boucle infiniment et POST vers API

aviation_robo.py utilise requests.post() vers :

/api/periphery/monitoring/adapters/gps

Puis boucle :

while True:

...

time.sleep(4)

Même correction :

```
simulated_data=true
network_post=true
infinite_loop=true
real_aircraft_action=false
```

5. bank_normal_flow.py boucle infiniment et POST vers API

Le connecteur banque utilise requests.post() vers :

/api/periphery/monitoring/adapters/bank

Puis boucle :

while True:

...

time.sleep(10)

Correction :

```
simulated_bank_payload=true
network_post=true
infinite_loop=true
real_bank_action=false
```

P1 — Important

6. Les connecteurs ne passent pas d’API key

```
Les connecteurs postent vers l’API avec requests.post(url, json=..., timeout=...).
```

Si l’auth devient API-wide, ces connecteurs casseront sauf ajout :

Authorization: Bearer <OBSIDIA_API_KEY>

ou :

X-API-Key: <OBSIDIA_API_KEY>

Patch :

OBSIDIA_API_KEY optional local

```
required if api_base not localhost or APP_ENV=prod
```

7. Connecteurs nommés “live” / “robo” peuvent être mal interprétés

Même si les payloads restent simulation/monitoring, les noms :

trading_live

aviation_robo

bank_normal_flow

peuvent être lus comme intégration réelle.

Correction naming / metadata :

connector_mode: SIMULATED_OR_MONITORING

real_execution: false

external_read: true/false

network_post: true

P2 — Cohérence Docker/env

8. Docker prod dit no cloud, mais connecteurs permettent egress si exécutés

Le Docker guide dit :

No cloud deployment is active. Docker is for local and internal validation.

C’est vrai pour le déploiement, mais pas une garantie “no network” globale si les connecteurs sont lancés.

Correction :

Docker API default: no external egress from API routes.

Connectors are separate optional scripts and may perform network POST / external reads if manually executed.

Nouvelle synthèse

NOUVEAUX P0

1. requests/httpx présents : réseau possible.

2. trading_live lit Binance via ccxt.

3. trading_live boucle infinie + POST API.

4. aviation_robo boucle infinie + POST API.

5. bank_normal_flow boucle infinie + POST API.

NOUVEAUX P1

6. Connecteurs sans auth header.

7. Noms live/robo à encadrer.

NOUVEAU P2

8. Docker no-cloud ≠ no-network si connecteurs exécutés.

Patch order ajouté

1. Ajouter CONNECTOR_BOUNDARY commune :

network_post

external_read

external_market_read

```
real_execution=false
simulated_payload=true/false
```

infinite_loop

max_events_supported

auth_header_supported

2. Modifier connecteurs :

--once par défaut

--loop explicite

--max-events

--api-key-env OBSIDIA_API_KEY

3. trading_live :

```
external_market_read=true
binance_read=true
no_order_execution=true
```

4. aviation/bank :

```
simulated_payload=true
no_real_aircraft_action=true
no_real_bank_action=true
```

5. Docs :

```
connecteurs = optional local simulation/monitoring tools,
```

not part of default API runtime.

Verdict actualisé :

```
CORE_X108 = OK
API_RUNTIME = À DURCIR
CONNECTORS = À CLASSER COMME NETWORKED OPTIONAL TOOLS
NO_EXTERNAL_EGRESS = FAUX SI CONNECTEURS LANCÉS
SRL_CANON = ENCORE À BLOQUER TANT QUE TOOL_CLASS_MATRIX NON POSÉE
```

Plan intégral des audits restants — obsidia-x108-proofs

SCOPE UNIQUE :

Eaubin08/obsidia-x108-proofs

BUT :

passer de “repo qui prouve beaucoup de choses”

à “repo audit-ready, RSSI-readable, boundary-clean”.

RÈGLE :

audit d’abord,

patch ensuite,

freeze seulement après preuve.

0. Audit socle — REPO_SURFACE_BASELINE_AUDIT

Objectif

Faire l’inventaire exact de toutes les surfaces du repo :

API routes

scripts

connectors

sigma tools

brody_memory_readonly

graph-memory

docker

workbench

docs/runtime

docs/freeze

local audit writers

manual write modules

À produire

docs/runtime/OBSIDIA_REPO_SURFACE_BASELINE_AUDIT_V0.md

docs/runtime/OBSIDIA_REPO_SURFACE_BASELINE_AUDIT_V0.json

Métriques

route_count

script_count

connector_count

`readonly_named_module_count`

filesystem_writer_count

subprocess_runner_count

network_client_count

manual_write_module_count

legacy_module_count

1. Audit boundary global — READONLY_SEMANTIC_SPLIT_AUDIT

Problème

Le mot readonly est utilisé pour trop de choses :

pas de décision

pas d’ACT

pas de mutation kernel

pas d’écriture Graphiti

pas d’écriture mémoire canonique

mais parfois écriture disque / logs / artefacts

On l’a vu sur output_envelope.py, où les flags de boundary peuvent être écrasés par payload.update(data).

À auditer

```
readonly
memory_write
graphiti_write
neo4j_write
emits_act
```

emits_verdict

allowed_to_decide

```
kernel_mutation
x108_merge
```

Nouveau modèle boundary cible

decision_readonly: true

act_readonly: true

```
kernel_mutation: false
```

x108_mutation: false

```
graphiti_write: false
neo4j_write: false
```

canonical_memory_write: false

filesystem_write: true/false

artifact_write: true/false

local_audit_write: true/false

pseudo_output_log_write: true/false

subprocess_execute: true/false

git_read_command: true/false

local_file_read: true/false

source_ingestion: true/false

network_post: true/false

external_read: true/false

prior_manual_write_dependency: true/false

auth_required: true/false

À produire

docs/runtime/OBSIDIA_BOUNDARY_SEMANTIC_SPLIT_AUDIT_V0.md

docs/runtime/OBSIDIA_BOUNDARY_SEMANTIC_SPLIT_MATRIX_V0.json

2. Audit API auth — API_AUTH_COVERAGE_AUDIT

Problème

main.py inclut beaucoup de routers :

status

brody

translation

context

memory

gencoin

graphiti

x108

os3

worldcalls

blockchain

audit

periphery_ops

brody_monitoring

runtime_freeze

bus

sigma_monitoring

runtime_wiring_preview

source_runtime_status

os_map

Mais l’auth visible est concentrée sur Brody. Le mode dev désactive aussi rate-limit/docs selon environnement.

À auditer

routes publiques réelles

routes sensibles non protégées

routes hors /api

routes qui exposent logs/paths/excerpts

routes qui écrivent localement

Public allowlist cible

/

/api/health

/api/readiness

/api/status minimal

Tout le reste :

```
auth_required=true
```

À produire

docs/runtime/OBSIDIA_API_AUTH_COVERAGE_AUDIT_V0.md

docs/runtime/OBSIDIA_API_ROUTE_SECURITY_MATRIX_V0.json

3. Audit filesystem writes — FILESYSTEM_WRITE_AUDIT

Problème

Beaucoup de modules déclarent readonly, mais écrivent :

audit_logs/*.jsonl

audit/*.jsonl

_local_audits/*.jsonl

artifacts/**

.graph-memory/reports/**

CURRENT_*

reports md/json/csv

Exemples déjà remontés :

AuditMiddleware écrit audit_logs/http_audit_<date>.jsonl.

BrodyBridge écrit audit/brody_llm_pseudo_output.jsonl.

World source intake écrit normalized_md + manifests.

Sigma packs écrivent payloads + rapports.

À auditer

write_text

open(..., "w")

open(..., "a")

mkdir

jsonl append

csv write

artifact dirs

absolute paths

user-controlled paths

À produire

docs/runtime/OBSIDIA_FILESYSTEM_WRITE_AUDIT_V0.md

docs/runtime/OBSIDIA_FILESYSTEM_WRITE_MATRIX_V0.json

Décision attendue

Chaque écriture doit être classée :

CANONICAL_MEMORY_WRITE

LOCAL_AUDIT_WRITE

ARTIFACT_WRITE

PSEUDO_OUTPUT_LOG_WRITE

REPORT_WRITE

SOURCE_NORMALIZATION_WRITE

DANGEROUS_PATH_WRITE

4. Audit local file reads / path exposure — LOCAL_FILE_READ_EXPOSURE_AUDIT

Problème

Certains modules lisent des chemins locaux depuis des packets ou des arguments.

Exemple : context_packet_consumer_readonly lit item.get("path") si l’excerpt est vide, puis peut exposer path, source_ref, excerpt, selected_items, response_md.

À auditer

Path(user_input)

Path(packet value)

read_text

read_bytes

docx/pdf/xlsx extraction

absolute path accepted

parent traversal

path leaked in response

À produire

docs/runtime/OBSIDIA_LOCAL_FILE_READ_EXPOSURE_AUDIT_V0.md

docs/runtime/OBSIDIA_PATH_EXPOSURE_MATRIX_V0.json

Règles cibles

no absolute path by default

allowed_roots only

relative_path only

path_hash in public response

excerpt requires auth/debug

5. Audit network / egress — NETWORK_EGRESS_AUDIT

Problème

Le repo inclut requests et httpx.

Les connecteurs font du réseau :

trading_live.py lit Binance via ccxt + POST API.

aviation_robo.py POST vers API en boucle.

bank_normal_flow.py POST vers API en boucle.

À auditer

requests.post

requests.get

httpx

ccxt

urllib

localhost calls

external exchange reads

infinite loops

API posts without key

À produire

docs/runtime/OBSIDIA_NETWORK_EGRESS_AUDIT_V0.md

docs/runtime/OBSIDIA_CONNECTOR_BOUNDARY_MATRIX_V0.json

Boundary connecteur cible

network_post: true/false

external_read: true/false

external_market_read: true/false

real_execution: false

simulated_payload: true/false

infinite_loop: true/false

max_events_supported: true/false

auth_header_supported: true/false

6. Audit Docker / image hygiene — DOCKER_IMAGE_HYGIENE_AUDIT

Problème

Docker fait :

COPY . .

Mais .dockerignore est beaucoup plus faible que .gitignore. .dockerignore n’exclut pas clairement artifacts/, _local_audits/, audit_logs/, CURRENT_*, GAVAGE_*, *.docx, etc. Alors que .gitignore les exclut explicitement.

À auditer

docker context

dockerignore vs gitignore

env files

local audits

generated zips

docx

CURRENT pointers

audit logs

graph-memory reports

À produire

docs/runtime/OBSIDIA_DOCKER_IMAGE_HYGIENE_AUDIT_V0.md

docs/runtime/OBSIDIA_DOCKER_CONTEXT_DIFF_MATRIX_V0.json

Décision attendue

dockerignore_aligned_with_gitignore: true/false

local_artifacts_excluded: true/false

secrets_excluded: true/false

generated_reports_excluded: true/false

7. Audit CI / manifest coverage — CI_MANIFEST_COVERAGE_AUDIT

Problème

Le manifeste récursif exclut apps/, alors que apps/obsidia_api contient la surface API réelle. La CI ignore aussi trop de docs/audits selon les règles déjà vues.

À auditer

CI triggers

paths-ignore

apps/ coverage

docs/runtime coverage

docs/freeze coverage

_local_audits exclusion

manifest recursive exclusions

protected files

forbidden content scanner

À produire

docs/runtime/OBSIDIA_CI_MANIFEST_COVERAGE_AUDIT_V0.md

docs/runtime/OBSIDIA_CI_COVERAGE_MATRIX_V0.json

Tests attendus

```
apps/ changed → CI runs
docs/runtime changed → CI runs
```

```
docs/freeze changed → CI runs
boundary file changed → CI runs
```

8. Audit Graphiti / Neo4j / manual apply — GRAPHITI_MANUAL_WRITE_BOUNDARY_AUDIT

Problème

Certains modules “readonly/memory” peuvent écrire dans Neo4j avec --apply + token manuel. On a aussi des modules qui lisent Neo4j live et dépendent d’un état écrit précédemment.

À auditer

GraphDatabase

execute_write

session.run MERGE/SET/CREATE/DELETE

manual token

--apply

neo4j read

graphiti read

graphiti write

manual write dependency

À produire

docs/runtime/OBSIDIA_GRAPHITI_MANUAL_WRITE_BOUNDARY_AUDIT_V0.md

docs/runtime/OBSIDIA_GRAPHITI_NEO4J_MODULE_CLASS_MATRIX_V0.json

Classes cibles

FROZEN_JSON_READER

LIVE_NEO4J_READER

GRAPHITI_CANDIDATE_EXPORTER

GRAPHITI_REVIEW_GATE

MANUAL_WRITE_APPLIER

POST_WRITE_VERIFIER

OUT_OF_SRL_READONLY

9. Audit SRL / session memory — SRL_CANONICAL_READINESS_AUDIT

Problème

La SRL existe en morceaux, mais pas en couche canonique.

Présent :

session ledger

presave buffer

followup resolver

auto triage

review gate

post-human triage

replay query

scheduler

Manquant :

SRL_SESSION_CARD

ACTIVE / SEMI_ACTIVE / COLD / GHOST

GHOST_SIDE_TABLE

COLD_INDEX

symbolic_hash

context_reload_pack

deep_recall_policy

À auditer

session_memory_ledger

session_presave_buffer

auto_triage

post_human_review

session_reopen

memory_scheduler

followup runtime

adapter

À produire

docs/runtime/OBSIDIA_SRL_CANONICAL_READINESS_AUDIT_V0.md

docs/runtime/OBSIDIA_SRL_GAP_MATRIX_V0.json

Décision attendue

```
SRL_STATUS = FOUNDATION_PRESENT_NOT_CANONICAL
```

10. Audit stubs / fake readiness — STUB_AND_READINESS_CLAIM_AUDIT

Problème

Certaines routes annoncent :

READY

REAL_BACKEND

LIVE

PARSED

AUDITED

alors que :

backend stub

doc viewer

not run

not collected

mock fallback

readiness statique

Exemple : OS3 replay retourne NOT_RUN, mais la surface peut être lue comme replay runtime.

À auditer

BACKEND_STUB

NOT_RUN

not_collected

mock fallback

declared ready

measured ready

documentary viewer

runtime proof

À produire

docs/runtime/OBSIDIA_STUB_AND_READINESS_CLAIM_AUDIT_V0.md

docs/runtime/OBSIDIA_CLAIM_ALIGNMENT_MATRIX_V0.json

Statuts cibles

DECLARED_READY

MEASURED_READY

DOCUMENTARY_READY

RUNTIME_READY

MOCK_READY

BACKEND_STUB

NOT_RUN

11. Audit Workbench / UI proof surface — WORKBENCH_PROOF_SURFACE_AUDIT

Problème

Workbench peut fonctionner sans backend via mock fallback, et pointe vers deux backends différents :

```
8011 = ObsidiaShell / agent_bridge
8000 = engine API optional
```

Il contient aussi un password Neo4j dev en clair dans le runbook.

À auditer

mock fallback

live backend status

backend 8011 role

backend 8000 role

proof mode

demo mode

env.local

passwords in docs

À produire

docs/runtime/OBSIDIA_WORKBENCH_PROOF_SURFACE_AUDIT_V0.md

docs/runtime/OBSIDIA_WORKBENCH_MODE_MATRIX_V0.json

Modes cibles

MOCK

LOCAL_BRIDGE

ENGINE_API

LIVE_BACKEND

PROOF_READY

NOT_PROOF_READY

12. Audit Sigma / load tools — SIGMA_TOOLCLASS_AUDIT

Problème

Les outils Sigma font :

subprocess execution

payload generation

1000/10000/100000 cases

CSV/JSON reports

stdout/stderr capture

Exemple : run_bank_scale_pack.py exécute subprocess.run, génère des payloads et écrit des rapports.

À auditer

run_bank_scale_pack

run_bank_replay_pack

run_bank_enterprise_pack

run_bank_regulatory_proxy_pack

run_bank_truth_proxy_pack

run_bank_robo_scenario_benchmark

À produire

docs/runtime/OBSIDIA_SIGMA_TOOLCLASS_AUDIT_V0.md

docs/runtime/OBSIDIA_SIGMA_TOOL_BOUNDARY_MATRIX_V0.json

Boundary cible

subprocess_execute: true

pipeline_execute: true

payload_generation: true

artifact_write: true

external_egress: false

real_bank_action: false

large_run_guard: true/false

stdout_stderr_redacted: true/false

13. Audit .graph-memory — GRAPH_MEMORY_BOUNDARY_AUDIT

Problème

.graph-memory fait un dry-run, mais exécute git ls-files, écrit des rapports, dépend de Path.cwd(), et possède une denylist trop large :

root

seal

hash

anchor

freeze

À auditer

dry_run_scan.py

dry_run_scan_phase0_strict.py

allow patterns

deny patterns

conditional allow

reports

token budget

À produire

docs/runtime/OBSIDIA_GRAPH_MEMORY_BOUNDARY_AUDIT_V0.md

docs/runtime/OBSIDIA_GRAPH_MEMORY_POLICY_MATRIX_V0.json

Correction attendue

SECURITY_DENY

PROOF_DENY

NOISE_DENY

ARTIFACT_DENY

HUMAN_APPROVAL_REQUIRED

INDEXED_BY_DEFAULT

14. Audit secrets / paths / logs — SECRETS_PATHS_LOGS_AUDIT

Problème

On a vu :

password dev dans doc

chemins Windows personnels

query params loggés

payloads printés

stdout/stderr capturés

source_original_path dans manifests

À auditer

password

token

secret

api_key

bearer

C:\Users

/home/

query params

payload print

source_original_path

stdout/stderr

À produire

docs/runtime/OBSIDIA_SECRETS_PATHS_LOGS_AUDIT_V0.md

docs/runtime/OBSIDIA_REDACTION_MATRIX_V0.json

Règles cibles

no raw query params

no raw payload print

no absolute local path in public report

no password in docs

stdout/stderr max excerpt + redaction

15. Audit docs / claims alignment — PUBLIC_CLAIM_ALIGNMENT_AUDIT

Problème

Le repo dit souvent :

```
readonly
```

dry-run

no write

no external egress

real backend

runtime ready

audit ready

Mais l’audit montre que ces claims doivent être affinés.

À auditer

README

docs/freeze

docs/runtime

external_pack

Docker guide

ENV reference

Workbench docs

architecture docs

À produire

docs/runtime/OBSIDIA_PUBLIC_CLAIM_ALIGNMENT_AUDIT_V0.md

docs/runtime/OBSIDIA_PUBLIC_CLAIM_MATRIX_V0.json

Claim cible

decision_readonly

not filesystem_readonly

local_artifact_writes_possible

connectors optional network tools

Graphiti manual apply quarantined

Workbench mock not proof

OS3 documentary viewer unless replay executed

16. Audit tests manquants — TEST_GAP_AUDIT

Objectif

Transformer tous les risques en tests.

Tests à ajouter

test_output_envelope_boundary_not_overridable

test_all_protected_routes_require_auth

test_no_route_outside_api_except_allowlist

test_no_raw_query_params_in_audit_logs

test_no_user_controlled_log_path

test_dockerignore_covers_gitignored_sensitive_patterns

test_context_consumer_rejects_absolute_path

test_context_consumer_rejects_parent_traversal

test_graphiti_manual_apply_not_in_srl_path

test_auto_triage_no_verdict_no_hold_block

test_readonly_modules_declare_filesystem_write_when_writing

test_connectors_have_once_mode

test_sigma_large_run_requires_confirm

test_claim_matrix_no_forbidden_claim

À produire

docs/runtime/OBSIDIA_TEST_GAP_AUDIT_V0.md

docs/runtime/OBSIDIA_TEST_BACKLOG_V0.json

17. Audit tool class matrix — TOOL_CLASS_MATRIX_AUDIT

Objectif

Classer chaque fichier exécutable dans une famille claire.

Classes

API_ROUTE

READONLY_VIEWER

DOCUMENTARY_VIEWER

LOCAL_ARTIFACT_GENERATOR

LOCAL_AUDIT_WRITER

SUBPROCESS_TEST_RUNNER

LIVE_SERVICE_READER

NETWORK_CONNECTOR

MANUAL_WRITE_APPLIER

GRAPHITI_REVIEW_GATE

MEMORY_TRIAGE_TOOL

SRL_FOUNDATION_MODULE

LEGACY_ARCHIVE

MOCK_UI

PROOF_CORE

À produire

docs/runtime/OBSIDIA_TOOL_CLASS_MATRIX_AUDIT_V0.md

docs/runtime/OBSIDIA_TOOL_CLASS_MATRIX_V0.json

C’est le document central avant SRL canonique.

Ordre d’exécution recommandé

PALIER 1 — Sécurité immédiate

1. API_AUTH_COVERAGE_AUDIT

2. FILESYSTEM_WRITE_AUDIT

3. SECRETS_PATHS_LOGS_AUDIT

4. DOCKER_IMAGE_HYGIENE_AUDIT

PALIER 2 — Boundary propre

5. READONLY_SEMANTIC_SPLIT_AUDIT

6. TOOL_CLASS_MATRIX_AUDIT

7. NETWORK_EGRESS_AUDIT

8. LOCAL_FILE_READ_EXPOSURE_AUDIT

PALIER 3 — Mémoire / SRL

9. GRAPHITI_MANUAL_WRITE_BOUNDARY_AUDIT

10. SRL_CANONICAL_READINESS_AUDIT

11. GRAPH_MEMORY_BOUNDARY_AUDIT

PALIER 4 — Preuve publique

12. CI_MANIFEST_COVERAGE_AUDIT

13. STUB_AND_READINESS_CLAIM_AUDIT

14. PUBLIC_CLAIM_ALIGNMENT_AUDIT

15. WORKBENCH_PROOF_SURFACE_AUDIT

PALIER 5 — Tests de verrouillage

16. SIGMA_TOOLCLASS_AUDIT

17. TEST_GAP_AUDIT

Freeze cible après tous les audits

OBSIDIA_X108_PERIPHERY_AUDIT_CONSOLIDATED_V0

Statut attendu

KX108_CORE_OK

API_SURFACE_CLASSIFIED

READONLY_SPLIT_DONE

FILESYSTEM_WRITES_DECLARED

NETWORK_EGRESS_DECLARED

GRAPHITI_MANUAL_WRITE_QUARANTINED

SRL_FOUNDATION_AUDITED_NOT_CANONICAL

DOCKER_CONTEXT_CLEAN

AUTH_API_WIDE_REQUIRED

PUBLIC_CLAIMS_ALIGNED

TEST_BACKLOG_READY

Verdict actuel

On n’est pas encore à “patch final”.

On est à “audit consolidation”.

La prochaine vraie étape n’est pas de brancher SRL.

La prochaine vraie étape est de produire la matrice centrale :

OBSIDIA_TOOL_CLASS_MATRIX_V0

+

OBSIDIA_BOUNDARY_SEMANTIC_SPLIT_MATRIX_V0

Ces deux matrices deviennent le socle. Ensuite seulement, SRL peut être canonisée proprement.

Oui. Là, le reste à auditer sur GitHub n’est plus “les bugs déjà trouvés dans le code”. C’est la surface GitHub complète autour du repo + les zones du repo qu’on n’a pas encore passées en profondeur.

Reste à auditer — GitHub uniquement

1. Audit GitHub settings / sécurité repo

À faire dans l’interface GitHub ou API GitHub si accès complet :

branch protection sur main

required status checks

required PR reviews

force-push disabled

delete branch protection

rulesets GitHub

merge methods autorisées

signed commits

secret scanning

push protection

Dependabot alerts

Dependabot security updates

code scanning

private vulnerability reporting

GitHub Pages désactivé/actif

environments / deployment protection

Actions permissions

workflow token permissions

Point actuel : repo public, main est la branche par défaut, mais il faut encore vérifier les protections réelles côté GitHub settings.

2. Audit branches / tags / releases

À faire :

branches actives

branches anciennes non mergées

branches locales poussées oubliées

tags freeze

tags P1/P2/P40/P41

releases GitHub

draft releases

assets attachés aux releases

branches contenant artefacts locaux

branches avec ancien état non sécurisé

But :

savoir si main est propre

savoir si une vieille branche expose un état dangereux

savoir quels tags sont vraiment auditables

3. Audit Pull Requests / review history

J’ai vérifié : pas de PR ouvert visible actuellement.

Mais il reste à auditer :

PR fermées

PR mergées

reviews

comments

resolved/unresolved threads

anciens diffs

CI failures historiques

merge commits

PR #12 et assimilées

Les commits mentionnent déjà des corrections liées à PR #12 / CI, notamment des tests legacy rendus hermétiques, des artifacts absents en CI, et des assertions partial-safe.

Audit restant :

rejouer la logique PR par PR

vérifier que les commentaires n'ont pas laissé de dette ouverte

vérifier que les changements CI n'ont pas seulement contourné les erreurs

4. Audit issues GitHub

J'ai vérifié : pas d'issue ouverte visible actuellement.

Reste à auditer :

issues fermées

issues non planifiées

issues liées à sécurité

issues liées à CI

issues liées à Graphiti / Brody / runtime

issues sans label

issues fermées sans patch réel

But :

retrouver les dettes masquées dans l'historique GitHub

5. Audit GitHub Actions / workflows

On a touché un workflow, mais pas l'ensemble GitHub Actions.

À auditer :

.github/workflows/*

triggers push / PR / manual

paths-ignore

permissions: contents read/write

actions/cache

upload-artifact

download-artifact

secrets utilisés

matrix

branches ciblées

tests réellement lancés

tests skip

coverage réelle

Point déjà critique : la CI actuelle doit être vérifiée parce que des changements de docs/audits peuvent ne pas lancer les bons tests, et les commits montrent que la CI a déjà été adaptée plusieurs fois pour accepter des états PARTIAL ou des artefacts absents.

6. Audit GitHub dependency graph

À faire côté GitHub Security / Dependabot :

Python dependencies

Node dependencies

Dependabot alerts

licences

vulnérabilités transitive

packages non pinnés

lockfiles présents/absents

> `Côté Python, requirements.txt utilise des versions minimales >=, pas des versions figées.`

Côté Workbench, package.json utilise aussi beaucoup de versions ^, donc audit de lockfile + dépendances nécessaire.

À vérifier :

package-lock.json présent ?

npm audit

pip-audit

licence compatibility

supply-chain risk

7. Audit historique Git / commits récents

À faire :

commits depuis le dernier freeze propre

commits co-authored Claude

commits avec “all tests pass”

commits qui changent les compteurs

commits qui ajoutent des familles source

commits qui modifient les boundaries

commits qui changent CI

commits qui ajoutent routes API

> Les commits récents montrent beaucoup de paliers P24 → P41, dont source runtime, OS map, Workbench, freeze runtime capability, et plusieurs assertions “tests pass”.

Audit restant :

vérifier que les claims de commit correspondent au code actuel

vérifier que les tests passés couvrent vraiment les routes ajoutées

vérifier que chaque palier a un artifact de preuve

8. Audit source packs / _source_packs / _source_discovery

À auditer encore :

_source_packs/

_source_discovery/

source_file_registry.json

source_file_registry.csv

registry_loader

source_pack_resolver

source_adapters

source_family_selector

pack ZIP vs directory packs

fichiers exclus DO_NOT_IMPORT_RUNTIME

Les commits montrent que les familles source ont beaucoup bougé : 7 familles, puis 8 familles, puis OS_TRAD_REVERSE_OS, puis Reverse OS Interlanguage Canon.

Audit restant :

vérifier tous les packs réellement présents sur GitHub

vérifier les fichiers ignorés/exclus

vérifier les compteurs registry

vérifier que les adapters ne chargent jamais du code non autorisé

9. Audit tests GitHub

À auditer :

tests/api/

tests/integration/

tests/sigma/

tests runtime-wiring

tests Workbench

tests source runtime

tests Brody

tests memory

tests CI skips

pytest.skip

xfail

tests partial-safe

tests qui passent sans artefacts locaux

Point important : les commits indiquent que certains tests ont été rendus “CI-safe” en sautant ou acceptant l’absence d’artefacts locaux.

Audit restant :

quels tests prouvent vraiment ?

quels tests évitent juste de casser CI ?

quels tests ne tournent qu’en local ?

quels tests acceptent PARTIAL ?

10. Audit Workbench GitHub

À auditer encore :

apps/obsidia-workbench/src/**

Vite config

proxy config

env variables

mock fallback

views runtime wiring

OS map view

source runtime view

API calls frontend

build output

package-lock

npm audit

package.json montre une app Vite/React avec scripts dev/build/preview et dépendances frontend.

À vérifier :

aucune clé dans frontend

aucun endpoint hardcodé public

mock fallback visible

mode preuve vs mode mock

build reproductible

11. Audit Docker / GitHub packaging

On a commencé, mais le GitHub-level reste à faire :

Dockerfile

docker-compose.yml

.dockerignore

GitHub Container registry éventuel

images publiées ?

packages GitHub ?

release assets Docker ?

Le Dockerfile fait COPY . ., donc la qualité réelle dépend de .dockerignore. .dockerignore est plus faible que .gitignore, donc audit packaging complet à faire.

12. Audit docs publiques GitHub

À auditer :

README.md

docs/

docs/runtime/

docs/freeze/

docs/architecture/

external_pack/

apps/obsidia-workbench/*.md

PUBLIC_DEPLOY.md

DOCKER_GUIDE.md

ENV_REFERENCE.md

But :

détecter claims faux / trop forts

détecter mots de passe dev

détecter chemins locaux

détecter “ready” non prouvé

détecter “readonly” trop large

> On a déjà vu que ENV_REFERENCE documente dev = docs exposed, no auth, no rate limit.

13. Audit GitHub code search — secrets / chemins / artefacts

À faire avec recherches systématiques :

password

token

secret

api_key

bearer

C:\Users

Desktop

localhost

127.0.0.1

admin1234

obsidia-graphiti-dev

GAVAGE

LOBOTOMIE

FORCE PASS

TODO

FIXME

HACK

STUB

BACKEND_STUB

REAL_BACKEND

READY

But :

scanner tout GitHub, pas seulement les fichiers qu’on a ouverts

14. Audit fichiers suivis par Git qui devraient être ignorés

À faire :

fichiers .bak suivis

CURRENT_* suivis

rapports runtime suivis

source discovery csv suivis

archives docs énormes

generated json suivis

local run reports suivis

On sait déjà que .gitignore contient une exception volontaire pour CURRENT_BRODY_SIGMA_READONLY_CHAIN_F68.txt, suite à un problème CI. Donc il faut vérifier toutes les exceptions de ce type.

15. Audit branch protection vs permissions

À faire côté GitHub settings :

admin peut push direct main ?

push direct autorisé ?

review obligatoire ?

CI obligatoire ?

status checks obligatoires ?

linear history ?

signed commits ?

force push désactivé ?

delete branch désactivé ?

C’est un audit GitHub pur, pas un audit code.

Ordre GitHub recommandé

1. GitHub settings / branch protection / Actions permissions

2. Open + closed PRs / issues

3. Workflows GitHub Actions

4. Commits depuis dernier freeze stable

5. Tests + skips + partial-safe

6. Dependency graph / Dependabot / npm audit / pip-audit

7. Source packs + registry

8. Workbench frontend

9. Docker packaging + .dockerignore

10. Docs publiques / claims

11. GitHub code search secrets/paths/stubs

12. Fichiers suivis qui devraient être ignorés

13. Tags / releases / assets

Version ultra nette

Ce qu’il reste à auditer sur GitHub :

- GitHub settings
- branch protection
- Actions permissions
- open/closed PRs
- open/closed issues
- commit history depuis freeze
- workflows

- tests/skips/partial-safe
- dependencies Python/Node
- source packs/registry
- Workbench frontend
- Docker packaging
- docs publiques
- secrets/paths/stubs via code search
- tracked generated files
- tags/releases/assets

Le prochain audit le plus rentable :

GITHUB_REPO_GOVERNANCE_AUDIT_V0

Avec sortie :

docs/runtime/OBSIDIA_GITHUB_REPO_GOVERNANCE_AUDIT_V0.md

docs/runtime/OBSIDIA_GITHUB_REPO_GOVERNANCE_MATRIX_V0.json

Parce que maintenant, le code a été largement ouvert. Le reste critique, c'est la gouvernance GitHub autour du code : qui peut merger, ce qui déclenche CI, ce qui est protégé, ce qui est exposé, ce qui est traçable.

Oui. Là, plan restant côté code uniquement — pas GitHub settings, pas PR/issues, pas gouvernance.

Plan des audits code restants — obsidia-x108-proofs

SCOPE :

code source uniquement

HORS SCOPE :

branch protection

issues

PR

GitHub settings

releases

permissions

1. Audit proofs/ / preuve formelle

À auditer

proofs/

proofs/lean/

proofs/tla/

proofs/V18_*

formal/tla/

merkle_root.json

merkle_seal.json

rfc3161_anchor.json

Questions

- Les preuves Lean/TLA sont-elles encore alignées avec le runtime actuel ?
- Les fichiers de preuve sont-ils encore appelés par CI ?
- Les seals / hashes correspondent-ils aux bons artefacts ?
- Les proofs V18 sont-ils legacy ou encore autorité ?
- Le repo mélange-t-il preuve canonique et preuve ancienne ?

Sortie attendue

OBSIDIA_FORMAL_PROOF_ALIGNMENT_AUDIT_V0

2. Audit sigma/ complet

On a regardé quelques outils Sigma, mais pas tout le moteur Sigma.

À auditer

sigma/

sigma/run_pipeline.py

sigma/contracts*

sigma/examples/

sigma/tools/

sigma/tests/

sigma/domain adapters

Questions

- Les gates ALLOW / HOLD / BLOCK sont-ils purement simulés ?
- Les métriques Sigma sont-elles cohérentes par domaine ?
- Les seuils sont-ils hardcodés ?
- Les exemples bank/trading/ecom/gps sont-ils réalistes ou seulement démo ?
- Les tests Sigma couvrent-ils toutes les familles ?
- Les tools Sigma écrivent-ils tous leurs artefacts proprement ?

Risque connu

Les scripts Sigma exécutent des subprocess et génèrent des payloads/rapports lourds ; il faut auditer tout le reste de sigma/tools, pas seulement bank scale/replay.

Sortie attendue

OBSIDIA_SIGMA_ENGINE_FULL_AUDIT_V0

3. Audit runtime_wiring/

À auditer

runtime_wiring/

runtime_wiring/source_runtime/

runtime_wiring/source_runtime_cache.py

runtime_wiring/source_pack_resolver.py

runtime_wiring/source_adapters.py

runtime_wiring/registry_loader.py

runtime_wiring/registry_types.py

Questions

- Les familles source sont-elles réellement hydratables ou seulement sélectionnables ?
- Les adapters chargent-ils du contenu ou du code ?
- Les compteurs registry sont-ils fiables ?

- Les packs ZIP et directory packs suivent-ils les mêmes règles ?

- Les fichiers DO_NOT_IMPORT_RUNTIME sont-ils respectés partout ?

Risque connu

Des commits récents ont ajouté des familles source, étendu le registry, et modifié les adapters/runtime. Il faut auditer cette chaîne à froid.

Sortie attendue

OBSIDIA_SOURCE_RUNTIME_DEEP_AUDIT_V0

4. Audit _source_packs/ / familles source

À auditer

_source_packs/

docs/source_packs/

source_file_registry.json

source_file_registry.csv

source_registry_summary.json

Questions

- Quels packs sont vraiment présents dans GitHub ?
- Quels packs sont seulement locaux ?
- Les fichiers `.py`, `.ps1`, `.bat`, `.sh` sont-ils exclus ?
- Les contenus RSSI / External Signals / OS_TRAD sont-ils sûrs à exposer ?
- Les compteurs registry correspondent-ils aux fichiers réels ?

Sortie attendue

OBSIDIA_SOURCE_PACKS_CONTENT_AUDIT_V0

5. Audit apps/obsidia_api complet

On a audité plusieurs routes, mais il faut passer fichier par fichier.

À auditer

apps/obsidia_api/

apps/obsidia_api/routes/

apps/obsidia_api/bus/

apps/obsidia_api/*.py

Axes

auth

rate-limit

boundary flags

local file read

filesystem write

raw payload echo

stub masking

Graphiti/Neo4j calls

subprocess

path exposure

Priorité

routes non encore ouvertes :

- routes restantes dans apps/obsidia_api/routes/*
- modules bus internes
- adapters Brody non lus
- true_voice / response composers
- backend response chain complet

Sortie attendue

OBSIDIA_API_CODE_FULL_AUDIT_V0

6. Audit periphery/ hors brody_memory_readonly

On a surtout creusé brody_memory_readonly. Il reste toute la périphérie générale.

À auditer

periphery/

periphery/memory/

periphery/world_calls/

periphery/interface/

periphery/ingestion/

periphery/workflow_governance_readonly/

periphery/core_registry.py

periphery/event_bus.py

periphery/brody_bridge.py

Questions

- Quels modules écrivent localement ?
- Quels modules appellent le kernel ou simulent un appel ?
- Quels modules sont legacy ?
- Quels modules doivent être classés LOCAL_AUDIT_WRITER ?
- Quels modules sont encore réellement READONLY_DECISION ?

Sortie attendue

OBSIDIA_PERIPHERY_NON_BRODY_AUDIT_V0

7. Audit periphery/brody_memory_readonly restant

On en a ouvert beaucoup, mais pas tout.

À auditer

periphery/brody_memory_readonly/**/*

Axes

- scripts jamais ouverts
- manifests vs code
- READONLY dans nom vs écriture réelle
- hardcodes 29 / 51
- CURRENT_* pointer dependency

- Graphiti apply dependency
- Neo4j live dependency
- session close / reopen / scheduler / replay chain complète

Sortie attendue

OBSIDIA_BRODY_MEMORY_REMAINING_MODULES_AUDIT_V0

8. Audit connectors/

On a vu trading_live.py, aviation_robo.py, bank_normal_flow.py. Il reste les autres.

À auditer

connectors/

Questions

- Quels connecteurs font du réseau ?
- Quels connecteurs bouclent à l'infini ?
- Quels connecteurs postent sans API key ?
- Quels connecteurs simulent vs lisent du réel ?
- Quels connecteurs peuvent être confondus avec exécution réelle ?

Risque connu

trading_live.py lit Binance via ccxt et poste vers l'API ; les autres connecteurs postent aussi en boucle vers l'API locale.

Sortie attendue

OBSIDIA_CONNECTORS_FULL_AUDIT_V0

9. Audit apps/obsidia-workbench code frontend

On a regardé package.json et le runbook, pas le code React.

À auditer

apps/obsidia-workbench/src/

apps/obsidia-workbench/vite.config.*

apps/obsidia-workbench/package.json

apps/obsidia-workbench/package-lock.json si présent

Questions

- Quelles routes API sont appelées ?
- Y a-t-il des endpoints hardcodés ?
- Le mock fallback est-il visible clairement ?
- Le mode proof/live/mock est-il distingué ?
- Des données sensibles sont-elles affichées ?
- Le frontend envoie-t-il l'API key ?
- Les erreurs backend sont-elles masquées ?

Note

package.json montre une app Vite/React avec build TypeScript, mais il faut auditer les appels API réels dans src/.

Sortie attendue

OBSIDIA_WORKBENCH_FRONTEND_CODE_AUDIT_V0

10. Audit dépendances Python / Node

À auditer

requirements.txt

package.json

package-lock.json

pyproject.toml si présent

Dockerfile

Questions

- Versions pinnées ou flottantes ?
- Dépendances réseau non justifiées ?
- Dépendances inutilisées ?
- Dépendances vulnérables ?
- Lockfiles présents ?
- Build reproductible ?

Point connu

```
requirements.txt utilise des minimums >=, pas des versions figées. package.json utilise aussi des plages ^ côté frontend.
```

Sortie attendue

OBSIDIA_DEPENDENCY_CODE_AUDIT_V0

11. Audit tests

À auditer

tests/

tests/api/

tests/integration/

tests/sigma/

tests/runtime*

Questions

- Quels tests prouvent vraiment ?
- Quels tests acceptent PARTIAL ?
- Quels tests skip si artefact absent ?
- Quels tests valident seulement des flags déclaratifs ?
- Quels tests couvrent les routes sensibles ?
- Quels tests doivent être ajoutés pour les failles trouvées ?

Point connu

Des commits indiquent que certains tests ont été rendus “CI-safe” en sautant ou acceptant l’absence d’artefacts locaux.

Sortie attendue

OBSIDIA_TEST_SUITE_DEEP_AUDIT_V0

12. Audit scripts scripts/

À auditer

scripts/

Questions

- Quels scripts modifient des fichiers ?
- Quels scripts génèrent des manifests ?
- Quels scripts scannent secrets ?
- Quels scripts excluent `apps/` ou `_local_audits` ?
- Quels scripts sont encore legacy ?

Priorité

generate_recursive_manifest.py

verify_recursive_manifest.py

check_forbidden_content.py

check_protected_files.py

tous scripts f23/fxx récents

Sortie attendue

OBSIDIA_SCRIPTS_DEEP_AUDIT_V0

13. Audit .github/workflows côté code

Pas gouvernance GitHub, mais contenu YAML.

À auditer

.github/workflows/*.yml

Questions

- Quelles commandes tournent vraiment ?
- Quels dossiers sont exclus ?
- Quels tests sont lancés ?
- Quels artefacts sont uploadés ?
- Les permissions sont-elles minimales ?
- Les docs/runtime déclenchent-elles CI ?

Sortie attendue

OBSIDIA_WORKFLOW_CODE_AUDIT_V0

14. Audit archives / legacy suivis par Git

À auditer

archive/

docs/runtime/archive/

proofs/V18_*

legacy files

*.bak suivis

Questions

- Est-ce que du code legacy est encore importable ?
- Est-ce qu'un vieux fichier dangereux peut être pris par Docker COPY ?
- Est-ce qu'un vieux module contredit les claims actuels ?
- Est-ce que les archives doivent rester dans GitHub ?

Sortie attendue

OBSIDIA_LEGACY_ARCHIVE_CODE_AUDIT_V0

15. Audit imports dynamiques / loaders

À auditer

importlib

__import__

dynamic module loading

runtime_loader

source adapters

registry loaders

Questions

- Quels modules peuvent être chargés dynamiquement ?
- Les chemins sont-ils allowlistés ?
- Les adapters peuvent-ils importer du code source packs ?

- Les modules legacy peuvent-ils être chargés par erreur ?

Sortie attendue

OBSIDIA_DYNAMIC_IMPORT_LOADER_AUDIT_V0

16. Audit erreurs silencieuses

À auditer

except Exception: pass

except Exception: return []

except Exception: fallback stub

except Exception: BACKEND_STUB

Questions

- Où les erreurs sont-elles masquées ?
- Quels backends cassés deviennent “empty” ?
- Quels modules retournent READY malgré erreur ?
- Quels endpoints devraient retourner MODULE_ERROR ?

Sortie attendue

OBSIDIA_SILENT_FAILURE_AUDIT_V0

17. Audit hardcodes / compteurs figés

À auditer

29

51

3448

3505

3547

3667

15298

15844

READY

PARTIAL

FOUNDATION_B_READY

Questions

- Quels compteurs sont encore hardcodés ?
- Quels tests cassent si un record change ?
- Quels nombres doivent être dérivés dynamiquement ?
- Quels “tests pass” sont anciens et non recalculés ?

Sortie attendue

OBSIDIA_HARDCODED_COUNTS_AUDIT_V0

18. Audit claims runtime dans le code

À auditer

READY

LIVE

REAL_BACKEND

AUDITED

PROOF_READY

runtime_allowed_now

```
emits_act
```

KX108_ONLY

```
readonly
```

dry_run

Questions

- Le code prouve-t-il ce qu’il annonce ?

```
- READY = mesuré ou déclaré ?
- LIVE = live service ou simple import ?
- REAL_BACKEND = backend réel ou fallback ?
- readonly = décision seulement ou filesystem aussi ?
```

Sortie attendue

OBSIDIA_RUNTIME_CLAIM_CODE_AUDIT_V0

Ordre recommandé côté code

PALIER A — Code critique sécurité

1. apps/obsidia_api full audit

2. periphery non-brody audit

3. connectors full audit

4. filesystem/local read/network audit transversal

PALIER B — Code preuve/moteur

5. proofs/formal audit

6. sigma full audit

7. runtime_wiring/source packs audit

8. dynamic imports/loaders audit

PALIER C — Code qualité/auditabilité

9. tests deep audit

10. scripts audit

11. workflows YAML audit

12. hardcoded counts audit

13. silent failure audit

PALIER D — Interfaces/legacy

14. Workbench frontend audit

15. brody_memory_readonly remaining audit

16. archive/legacy audit

17. runtime claims code audit

Version courte

Ce qu’il reste à auditer côté code :

1. proofs/formal

2. sigma complet

3. runtime_wiring complet

4. _source_packs / registry

5. apps/obsidia_api complet

6. periphery hors brody_memory

7. brody_memory_readonly restant

8. connectors complet

9. Workbench frontend

10. dependencies Python/Node

11. tests

12. scripts

13. workflows YAML

14. archives/legacy

15. dynamic imports/loaders

16. silent failures

17. hardcoded counts

18. runtime claims dans le code

Priorité immédiate

1. apps/obsidia_api complet

2. tests complet

3. runtime_wiring/source packs

4. proofs/formal

5. sigma complet

Parce que ces cinq blocs répondent à la vraie question :

Est-ce que le repo prouve réellement ce qu'il annonce,

ou est-ce qu'il annonce plus que ce que le code garantit ?

Synthèse finale - Fusion des deux documents

Couche	Rôle	Peut décider ?	Peut ACT ?	Statut
Kernel X108	autorité	oui	selon verdict	souverain
Sigma	surveillance	non	non	advisory / proof signal
Agents	observation	non	non	signal
Audit	preuve	non	non	trace / replay
SRL	mémoire sessionnelle	non	non	périphérique
Cold / Ghost	conservation hors contexte actif	non	non	rappel conditionnel

Chemin décisionnel X108 : Domaine → Agents → Aggregate → Sigma → X108 → Envelope → Audit.
Chemin mémoire SRL : Session → Card → Triage → Active / Semi-active / Cold / Ghost → Reload Pack.
Différence finale : X108 autorise. SRL rappelle.